A Postmortem of a Withdrawn Result

Supplementary Material for "Faster-than-Light: Latency Measurement Artifacts in Streaming Benchmarks"

Gustavo Pedro Ricou

This document is the supplementary material for the paper named above, and it is not part of the main submission. It is written as a postmortem by the lead author. Part I — *what we set out to measure, and what broke* — retells the work in the order it happened, with everything withdrawn along the way. Part II — *the mechanism, and the law it produces* — is the argument. Part III — *apparatus, data and tables* — is reference material, meant to be looked things up in rather than read through. Part IV — *the literature, and what it already requires* — places the result. Main-text references of the form "Supplement Sn" resolve to the correspondingly numbered section here, and the artifact's `docs/supplement_index.md` records where each section came from.

CONTENTS

Part I. What we set out to measure, and what broke 4

S1. How these results were reached: the chronology, and everything withdrawn 4
S1.1. The phase excluded from every result . . . 5
S1.2. The answer we obtained first, and why it was wrong . . . 5
S1.3. The claim about the benchmark that we withdrew . . . 5
S1.4. The causality framing, withdrawn in this version . . . 5
S1.5. The M/G/1 form, withdrawn . . . 5
S1.6. The traced tail-index cross-check, withdrawn . . . 6
S1.7. Pre-registration outcomes . . . 6
S1.8. The achieved send rate was taken from a flag, not measured . . . 6
S1.9. A source we read wrong, and how we found out . . . 6
S1.10. The distributed-mode gap . . . 6
S1.11. The first result set . . . 6
S1.12. The first corpus's prior corrections . . . 6
S1.13. The result . . . 7
S1.14. The checks it had already passed . . . 7
S1.15. Open questions from the comprehensive campaign . . . 7

S2. What the broker comparison rests on, and what it does not 8

S3. The twentyfold end-to-end gap was send lag, and we withdrew the claim 9
S3.1. The delay sweep failed its manipulation check . . . 10

S4. Our own replay-rate record did not survive the audit 11
S4.1. Provenance-gap episode and the co-location withholding . . . 14
S4.2. No artifact records the earliest corpus's replay rate . . . 14

S5. E1's shift is confined to the first seven events 14
S5.1. Where the knee sweep placed the load conditions . . . 15

S6. The even-denominator exclusion was wrong, and what replaced it 15

Part II. The mechanism, and the law it produces 16

S7. Priority and geometry each move the rate on their own 16
S7.1. The overnight campaign, prediction by prediction . . . 16

G. P. Ricou is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie. This document accompanies the paper by G. P. Ricou (Trinity College Dublin) and R. Duvignau (Chalmers University of Technology, Gothenburg, Sweden); the retelling and its judgments are the lead author's.

S8. What the payload sweep fits, and what it does not support 16
S8.1. Tail recovery without a reference clock . . . 19

S9. The two-state model commentary and the tracer checks 19

S10. Retention sits on the grid, and the spread rule says when 21

S11. Grid membership, configuration by configuration 22

S12. Every real-time-priority pair, the ladder they make, and the exposure curve 23

S13. What the brokers actually do 26
S13.1. Equivalence on gated artifacts . . . 26
S13.2. The condition that reverses the ordering . . . 26
S13.3. The falsifier, registered in advance . . . 26
S13.4. Why the standard testbed cannot see it . . . 26

S14. The classical statistics of a discarded zero, and what the guard deletes 27
S14.1. The two floored clocks behind the $-1000\,\mu$s signature . . . 27
S14.2. The permutation null, constructed . . . 28

S15. Which clocksource, bounded from a measurement 29

S16. Why the acknowledgment timestamp is late, and why the broker is not the reason 29
S16.1. Where each timestamp is actually taken . . . 29
S16.2. Why one span cannot invert and the other can . . . 30
S16.3. The second thread is not the mechanism . . . 30
S16.4. Two rivals: the scheduler itself, and the hypervisor . . . 31
S16.5. Why averaging does not repair either mode . . . 31
S16.6. The redesign that does not help . . . 32
S16.7. Where the repairs belong . . . 32
S16.8. The scheduler constants, derived . . . 32

S17. The 1970 counter note, line by line, and the grid geometry it explains 32

Part III. Apparatus, data and tables 33

S18. The experiment map, the claim-evidence table, and the audit threshold figure 34
S18.1. What the audit gate selects, bounded . . . 34

S19. Threats to the scheduling attribution, and what rules them out 35

S20. The workload is a live football feed, described as far as the results require 38

S21. Every setting that can move a latency number 38

S22. This harness has discarded latency data before 39
S22.1. Distributed mode never completed a run in eleven attempts . . . 40

S23. Where the check transfers, and what it costs to run 41

S24. The campaign ledger, and what distributed mode did instead 42
S24.1. OpenMessaging distributed mode, failure diagnostics . . . 42
S24.2. The statistical inventory . . . 43
S24.3. The metric map . . . 43

S25. The evidence behind the count of tools that dispose silently 43

S26. Tables 45
S26.1. The mechanism campaigns' tables . . . 45
S26.2. The audit, timestamping-mode and injected-delay tables . . . 46

Part IV. The literature, and what it already requires 46

S27. Where our irreproducibility sits in the variability literature 46

S28. The resolution literature knew the problem; the deletion failure is new 48
S28.1. The payload sweep's underpowered rows, and the comprehensive campaign's duration and plateau paragraphs (moved from the main text) . . . 49

S29. Two neighboring literatures, and what they do about the second clock 50
S29.1. "One-way latency" that is half a round trip . . . 50
S29.2. Drift-aware synchronization in MPI benchmarking . . . 50

S30. Where the timestamp is written, in other runtimes 50

S31. Neighboring rules, and what they already require 51

S32. What standardized benchmarks require about resolution 52

S33. Where our reading parts from the concurrent report 53
S33.1. The survey behind the two related-work sentences . . . 53
S33.2. Production tracing reached the conclusion without the mechanism . . . 53
S33.3. A framework that does not have the problem . . . 54
S33.4. How large the literature is that misses this . . . 54
S33.5. Where scheduling delay comes from, and what the queueing account gets right . . . 55

References 56

LIST OF FIGURES

S1 Window sweep, as a picture . . . 12
S2 Withdrawn: the end-to-end gap as first decomposed . . . 13
S3 Longer delivery inverts less often . . . 17
S4 Two-state model, drawn . . . 19
S5 Campaign's confirmed prediction . . . 21
S6 Grid membership, as a set . . . 23
S7 All 8 pairs, as a ladder . . . 24
S8 Every manipulation separates; the broker does not . . . 25
S9 Where a path sits on the exposure curve . . . 26
S10 What the benchmark deletes, beside what it then reports . . . 28
S11 Why a sample disappears, and why the survivors are not a sample . . . 33
S12 Experiment map . . . 34
S13 Audit on Testbed A . . . 36

LIST OF TABLES

S1 First result set, later rejected . . . 6
S2 Historical corpus; the scheduling-lag columns are withdrawn . . . 8
S3 Powered transport replication at a verified true-real-time rate, over a median of 125 events per run (not the seven of the main text), audit-gated (Section S18). Medians in ms; the Hodges–Lehmann shift is $Kafka - Redis$ with a percentile-bootstrap 90% interval. The brokers are equivalent within 1 ms at every N (all three estimators) yet cleanly distinguishable: Redis is ≈ 0.41 ms faster, and the shift does not grow with concurrency. . . . 9
S4 Observation-window sweep separates a per-run cost from a per-event one . . . 11
S5 Why the delay sweep is not an effect-size manipulation . . . 11
S6 One set of runs reproduces both published answers . . . 14
S7 Grid membership against a continuum null . . . 24
S8 Every matched pair . . . 24
S9 What the displacement costs at longer paths, and how wide the cost is . . . 25
S10 What each claim rests on . . . 35
S11 Audit gate applied to the powered transport campaigns . . . 35
S12 Workload, from 3,315 matches . . . 39
S13 Every configuration that can move a latency number, and why it holds that value . . . 39

S14 Audited harnesses, one row per span rather than per tool . . . 43
S15 Replicate spread against the phase denominator q . . . 45
S16 Retention against the producer's send interval, with message size, background load, duration and host held fixed. Only the commensurate rate is unstable, and the two incommensurate rates agree with each other despite intervals differing by 19%. Predictions were recorded before the runs. . . . 45
S17 Retention against payload at 457 msg/s, incommensurate with the millisecond grid so samples stay dephased. Implied T_{true} inverts the main text. The final column is the increment in implied latency per byte of added payload, relative to the 200 B baseline; the two smallest steps are noise-dominated and marked. . . . 46
S18 H2: negative-span rate against achieved CPU utilization, no core pinning, background load swept from idle to oversubscription. The withdrawn queueing form fits at $R^2 = 0.945$ against 0.640 for a straight line. . . . 46
S19 Bounding the unequal-retention selection effect . . . 47
S20 Mixture, measured across nine load levels (25 runs each, pre-registered) . . . 47
S21 Occupancy, not utilization . . . 48
S22 Same utilization, different rate . . . 48
S23 Slower delivery is a more reliable measurement, twice . . . 49
S24 Kernel tracing predicts the negative-span rate, unfitted, in every configuration that has one . . . 49
S25 Consistency audit of every run behind a reported result . . . 50
S26 H3: median broker transport under the two timestamping modes, both configurations at one in-flight request under matched load. `callback` is the default asymmetric timestamping mode; `inline` timestamps Kafka on the calling thread as Redis already does. The between-system gap shrinks, and the shrinkage is entirely on Kafka's side. . . . 50
S27 Injected one-way broker delay, applied identically to both systems, five feeds . . . 51

On review. This manuscript has not been submitted to, or reviewed by, any journal. Where a section below records a correction, the correction was found in the course of the work rather than asked for by a referee.

On the vocabulary. This document uses the paper's terms, defined here once. A *condition* is one point of the experimental design; a *cell* is a condition together with a workload; a *run* is one execution of a cell and a *replicate* a further run of the same cell; where a manipulation compares settings that differ in one variable, each setting is a *configuration*. The *admission condition* is the benchmark's filter that keeps an end-to-end sample only when its millisecond-quantized difference is strictly positive; the audit registry (Section S25) uses *guard* for a tool's own conditional of that kind, where the word names a code path rather than the benchmark's. The *timestamp resolution* τ is the smallest difference a timestamp can express, one millisecond here. The *scheduling delay* is the time a thread that is ready to run spends queued before a core runs it; the *send lag* is how late the send call ran after the event was due. The paper's two failures are the *scheduling failure* (Section V of the paper: a negative span from a late acknowledgment timestamp) and the *deletion failure* (Section VI: samples shorter than the timestamp resolution discarded uncounted).

PART I. WHAT WE SET OUT TO MEASURE, AND WHAT BROKE

This part is the postmortem proper, in the order the work happened. Section S1 retells the chronology and everything withdrawn along the way. Sections S2 and S3 hold the two results we took back: what the broker comparison rests on once the wrong corpus is removed, and the twentyfold end-to-end gap that turned out to be send lag. Section S4 turns the same scrutiny on our own record-keeping, which did not survive it. Sections S5 and S6 close the two reconciliations — where E1's shift comes from, and the exclusion we got wrong.

S1. HOW THESE RESULTS WERE REACHED: THE CHRONOLOGY, AND EVERYTHING WITHDRAWN

The main text is organized around what the evidence supports. This section holds what that reorganization displaced: the order in which the work happened, every claim withdrawn, and the pre-registrations that failed. It is kept in full because an audit's credibility rests on its own failures being visible, and because a reader checking whether the corrections were real should not have to take the summary on trust.

We are not alone in needing the practice, and the field has not settled how to provide it. In 2026 Bostanci et al. showed that a conference paper carrying all three artifact badges, including *Reproducible*, had reported incorrect simulator results through configuration errors, and that its artifact could not rebuild its own headline claims [S1]. Their closing recommendation is the one this section is an attempt at: that the community needs mechanisms for correcting results and artifacts *after* publication, rather than only gates before it. A withdrawal recorded in the document that made the claim is the cheapest such mechanism we know of.

S1.1. The phase excluded from every result

Measurement perturbs, and one early phase perturbed enough to be unusable: it pinned the load generator to a subset of cores while utilization was measured across all of them, diluting ρ. Every number in the main text excludes it, and the campaigns that remain measure utilization with no core pinning. It is recorded here rather than dropped because a phase removed without saying so is the same omission this paper is about.

S1.2. The answer we obtained first, and why it was wrong

The study began as a broker comparison on a live sports feed. The first complete result set was monotone in concurrency, significant after correction ($p = 9.0 \times 10^{-11}$), and had a tidy architectural story: a twentyfold end-to-end gap between the two brokers. It passed replicate counts, effect sizes, corrected significance and sanity plots.

Two independent defects destroyed it. The sign check of the main text's consistency check rejected 862 of the $1{,}382$ runs on that testbed, including every run behind the headline: only 8 of that testbed's 76 conditions survive it. Separately, the twentyfold gap itself was a per-run start-up cost read as a per-event constant: the median was taken over as few as seven events per run, and a deterministic per-run artifact reproduces across a hundred runs beautifully. The second defect is the more instructive one, because the run data were causally consistent throughout. Causal consistency is necessary, not sufficient.

S1.3. The claim about the benchmark that we withdrew

An earlier version of this work read the OpenMessaging Benchmark's discards as causality violations. They are not. The instrumented counter we had added could not distinguish sign, and separating it showed the Kafka-driver corpus's $10{,}913{,}263$ discarded samples contain not one negative: they are resolution failures, samples whose true delivery was faster than the timestamp resolution. The refuting evidence had been in our own artifact from the day we committed it, in a field we had printed and not read. The same separated channel then caught $41{,}403$ genuine negatives in the Redis-driver replication, which is the result the main text reports.

We also called the mechanism behind those Redis-driver negatives a *candidate* — a sub-millisecond skew between two views of a virtualized clock — and hedged it as such through the round-1 internal draft. The hedge was unnecessary, and reading the driver removes it. `RedisBenchmarkConsumer.java` takes the publish timestamp from `entry.getID().getTime()`, the stream-entry identifier the Redis *server* assigns on `XADD`, while the Kafka driver uses a producer-set timestamp. The Redis span is therefore a difference between two millisecond-floored clocks on two hosts, for which $-1000\,\mu$s is the only negative value expressible; the Kafka span floors two timestamps differenced inside one JVM, and never goes negative. The corpus matches both predictions exactly. Hedging a claim the source code settles is its own kind of imprecision, and the main text now states the mechanism.

S1.4. The causality framing, withdrawn in this version

Through version 2 this paper described its first failure mode as a causality violation, on the argument that an acknowledgment causally precedes a receipt. That argument is wrong, and a reader pointed it out. The broker's append precedes both the producer's receipt of the acknowledgment and the consumer's receipt of the record; those are two branches of one cause, and neither precedes the other. A negative difference between them is therefore possible without any physical impossibility, and is in fact routine when the producer's callback thread is descheduled.

We settled the question on the corpus rather than by argument. Every archived run that joins producer and consumer events was recounted, span by span (`scripts/recount_spans.py`, output committed as `docs/results/span_recount.csv`). Over $738{,}730$ events in $5{,}913$ runs, the acknowledgment-referenced span is negative $62{,}264$ times and the send-referenced span is negative 0 times. Nothing in the corpus arrived before it was sent.

What changed as a result: the framing, the justification of the gate, and the claim that the failure is self-detecting. What did not change: the rejection counts, which are unaltered as numbers and now described as reference-timestamp failures; and every manipulation result in the Section V of the main text, which rests on moving scheduling and observing the rate move, not on what the negatives are called.

S1.5. The M/G/1 form, withdrawn

We adopted an M/G/1 waiting-time framing as a leading-order account of the stall, following its use for benchmark bias elsewhere, and pre-registered it as a hypothesis with a falsifier. Extended to the region where the candidate forms diverge ($\rho = 0.881$–0.990), the M/G/1 form fitted *worse than the mean*: $R^2 = -0.05$, against a fitted exponential's 0.93. Our own registered bracket also missed, predicting 2.45–$3.07\times$ growth against a measured $1.44\times$. Both candidates were parameterized in ρ, and the real-time-priority experiment shows ρ is not the variable. The Pollaczek–Khinchine result remains the motivation for expecting a heavy waiting tail; it is not a fitted result here.

TABLE S1
FIRST RESULT SET, LATER REJECTED. BROKER TRANSPORT (MEDIAN, MS) BY CONCURRENCY, TESTBED A, $10\times$ REPLAY. EVERY CELL FAILS THE CHECK OF THE MAIN TEXT.

N	Kafka	Redis	Difference	p_{Holm}
1	0.999	1.006	0.007	1.0×10^{-4}
5	1.001	1.197	0.196	6.5×10^{-6}
10	1.002	1.346	0.344	9.0×10^{-11}

S1.6. The traced tail-index cross-check, withdrawn

The traced tail index was re-estimated with an interval rather than read off a slope. Doing so refuted the cross-check it was meant to strengthen, and the claim is withdrawn; S8 gives the numbers and the argument. In summary: two proper estimators on the same histogram disagree sixfold because the survival is not a power law over the window we quoted, and the agreement we reported between the traced index and the payload-sweep exponent was a coincidence of window and estimator. The payload-sweep fit itself is unchanged and is now reported in S8 rather than in the main text, since four points do not earn an equation there. The mechanism results are untouched: none of them rests on the value of an exponent, only on the direction the rate moves when T_{true} moves, which is measured and monotone.

This is the third occasion on which a number in this work stood for months and failed the first time it was estimated with an interval attached. We record that pattern because it is the argument of the paper turned on its author: a statistic that carries no uncertainty is not yet a measurement.

S1.7. Pre-registration outcomes

Six predictions were registered with falsifiers before the comprehensive campaign. One was confirmed (the payload flip, the main text's payload-flip figure); three falsifiers fired; one was not evaluable once smeared configurations appeared, because the branch-count statistic it relied on has no value on a continuum; and one split. The jitter-kernel conjecture advanced after that campaign was killed by a subsequent alternation test. The plateau observation at 1053 and 1219 msg/s refuted the linear extrapolation that one registered prediction had assumed.

S1.8. The achieved send rate was taken from a flag, not measured

For part of the campaign the achieved send rate was recorded from the configuration flag rather than measured against elapsed wall time, and for a subset of runs the two disagreed. The affected conditions were re-derived from per-event timestamps and the ledger corrected; the incident is the reason rule (10) of the main text's checklist exists in the form it does. It is also the reason the main text reports measured pacer jitter rather than a configured rate.

S1.9. A source we read wrong, and how we found out

Section III-A of the main text places a published broker comparison inside the regime the deletion law binds on. An earlier draft of that placement attributed to it [S2] a Redis p99 of 0.8 ms against Kafka's 12.5 ms. Those figures are not on that page. Re-reading it on 2026-09-03 found only Kafka given a percentile — "sub-10ms latency at p99" — with Redis and NATS given "sub-millisecond" and no percentile at all, which is the sharper exhibit and the one the paper now uses. The figures we had attached to it belong to a different 2026 comparison [S3], which a search had returned in the same result set.

We record it here rather than in the reference note because it is the same class of error this paper is about, one level up: a number carried from one source to another because both arrived together, and nothing in the citation would have shown it. Both documents are now placed separately in S33.4, and the registry the main text counts from (`docs/results/external/literature_regime.csv`) holds one row each.

S1.10. The distributed-mode gap

The OpenMessaging Benchmark's distributed mode failed to complete a run on every attempt we made, across two release versions. The cross-host exposure of that particular tool therefore rests on reading its source together with our own two-host harness, not on running the tool itself cross-host. Diagnostics are in the upstream issue draft accompanying this artifact.

S1.11. The first result set

Table S1 is the first, rejected result set in full.

S1.12. The first corpus's prior corrections

We present the first result set in full rather than describing it, because a reader cannot otherwise judge how convincing an invalid corpus looks.

S1.13. The result

On Testbed A, replaying distinct real matches at ten times real speed across one, five and ten feeds with 15–30 runs per cell and both producers pipelined, broker transport separated the two systems cleanly (the main text). Every comparison was significant after correction and at $N \geq 5$ the run distributions did not overlap.

S1.14. The checks it had already passed

The corpus was not naive. It was the product of five earlier corrections, each of which had reversed the apparent winner and each found by investigating a result that looked wrong:

1) *A process-relative clock.* Cross-process timestamps taken with `perf_counter_ns`, whose origin is per-process, injected each run's consumer start-up offset into every measurement. Fixed with a shared wall-clock epoch.
2) *A saturating replay speed.* At $120\times$ the producer could not keep pace, inflating send lag to tens of seconds.
3) *An asymmetric load generator.* The Kafka producer blocked on a broker round trip per event while the Redis producer dispatched asynchronously. Median TTI at one feed fell from $1{,}644$ ms to 16 ms once the Kafka producer was pipelined.
4) *A concurrency axis that was covertly a throughput axis*, from the merged-plan design described in the main text.
5) *A heavy tail contaminating every mean.* Kafka's end-to-end 95th percentile sat at 99–105 ms at every concurrency level while its median moved from 1.8 to 7.2 ms. A 95th percentile that ignores load is not system behavior; decomposition located it in send lag, not transport.

We also verified the comparison was not biased by message size or protocol: payloads are near-identical (median $\approx$170 bytes) and serialization costs are negligible and comparable.

A sixth problem shaped our later protocol. Our first run of the acknowledgment experiment of the main text reported no improvement whatsoever — a clean refutation of our own hypothesis. The treatment had never reached the consumer: the orchestration script accepted the option but, through a failed edit, never passed it on. We found this only by deliberately supplying an invalid option and observing that the consumer did not reject it. A null produced by an unapplied treatment is indistinguishable in the data from a genuine one, so every intervention reported here carries a manipulation check that aborts the campaign if the treatment is not accepted.

The point of the list is not the corrections. It is that a corpus which had survived six of them, a fairness audit, and a full battery of rank tests, effect sizes and multiplicity correction, was still entirely invalid, and nothing in its output said so.

S1.15. Open questions from the comprehensive campaign

Three observations from the campaign are consistent with a single conjecture — a per-send jitter kernel whose width in cell units grows with the numerator p of Δ/τ — and none of them is established. We separate them from the results above deliberately: they are post-hoc, the pacer they implicate is measured only in the non-JVM replication of the main text, and one of our own follow-up tests cuts against the conjecture.

Displacement toward θ. Replicates sit off their grid vertices toward the continuous value, from under a point to complete detachment. The two configurations that detached outright (a majority of replicates more than 3 points from every vertex) are 300 msg/s in the evening pass and 700 msg/s — both with $p = 10$; no configuration with $p \leq 8$ detached, and the same evening that 300 msg/s detached, 625 msg/s ($p = 8$) produced its cleanest two-branch split of the study. Suggestive; a census of two is not a law.

Mobility between passes, now measured under control. The same 300 msg/s configuration was branch-adjacent in the morning and detached in the evening; passes were hours apart, so epoch and configuration were confounded. The controlled test interleaves the two configurations A/B/A/B within one session — four pairs of (300, 625) msg/s — and **both configurations hold their grids**: 300 msg/s at 37.40, 38.21, 39.86 on the lower vertex and 67.41 on the upper; 625 msg/s at 42.97 and 55.91–60.18 (the `ultimate_alt` cells of the artifact). No configuration detached. The detachment is therefore a property of the *epoch*, not of the configuration, and the numerator census above loses its same-evening contrast — which cuts the kernel conjecture's strongest support and is reported accordingly. What remains established is that some sessions produce a detached regime and others do not, at fixed configuration; the disciplined clock is exonerated throughout (residual 0.013 ppm), and the pacing loop's state remains the open suspect.

Branch suppression. The formal residual analysis (the main text) finds mean retention *below* θ at every $q \geq 3$ configuration, and the rank correlation of |residual| with p is negative (-0.8), not the positive association a displacement-scaling kernel predicts — the sign is dominated by branch-weight sampling at $q = 1$. What the residuals do support is minority-branch suppression: a kernel narrower than the half-cell keeps runs on the nearest vertex, depressing the upper-branch weight below $\mathrm{frac}(q\theta)$, which would also account for the 1250 msg/s miss above. A quantitative displacement prediction from a measured jitter statistic does not yet exist; the harness of the main text records the statistic per run and is where one would be built.

TABLE S2

HISTORICAL CORPUS; THE SCHEDULING-LAG COLUMNS ARE WITHDRAWN. END-TO-END DELAY AND ITS DECOMPOSITION, TESTBED B, AFTER THE CONSISTENCY CHECK (164 OF 201 RUNS RETAINED). MEDIANS, MS. **THE TTI AND SCHEDULING-LAG COLUMNS RECORD A PER-RUN START-UP COST, NOT A PER-EVENT PROPERTY, AND ARE RETAINED ONLY AS EVIDENCE FOR THE WITHDRAWAL IN THE MAIN TEXT; THEY SHOULD NOT BE READ AS SYSTEM BEHAVIOR.** THE TRANSPORT COLUMNS ARE COMPUTED OVER THE SAME SEVEN-EVENT OPENING BURST, AND SO MEASURE START-UP RATHER THAN STEADY STATE: SUPPLEMENTARY MATERIAL S5 REPRODUCES THEM FROM THE POWERED RUNS BY RESTRICTING THOSE RUNS TO THE SAME WINDOW. FOR STEADY-STATE TRANSPORT SEE TABLE S3. THE REPLAY RATE WAS NOT RECORDED AND IS RECOVERED AS TRUE REAL TIME IN THE MAIN TEXT.

	TTI		Sched. lag		Transport	
N	Kafka	Redis	Kafka	Redis	Kafka	Redis
1	105.3	5.0	103.0	1.4	0.792	0.721
9	105.5	5.4	103.0	1.9	0.802	0.807
10	105.3	5.8	102.8	2.0	1.000	0.855
12	105.7	5.3	102.9	1.7	0.839	0.844

S2. WHAT THE BROKER COMPARISON RESTS ON, AND WHAT IT DOES NOT

What this claim rests on. We state the evidential basis first, because an earlier version of this section rested it on the wrong corpus. The load-bearing measurement is a powered replication at a replay rate that is *derived from the plan and verified against wall time before the campaign runs* — a documented setting, not an inferred one — over a median of 125 audit-surviving events per run. The original E1 corpus (Table S2) is reported afterwards, as the historical measurement it is, for two reasons that disqualify it from carrying the claim: its replay rate is an inference — recovered in the main text as true real time, but inferred from the runs rather than recorded — and it matched a *median of seven events per run*, the same seven-event opening burst as the send lag we withdraw in the main text. A sub-millisecond difference cannot be resolved on seven events, and E1 does not resolve one: its Hodges–Lehmann shifts wander (0.021, 0.116, 0.053 ms) and its $N{=}1$ estimators disagree. That E1 finds the two systems "equal" is as much a statement about seven events as about the brokers. What E1 does establish, and what the powered campaign confirms, is the *concurrency* result: Kruskal–Wallis finds no significant effect of concurrency in either backend ($p = 0.061$ for Kafka, $p = 0.091$ for Redis), and neither system degrades across the range.

The powered measurement: We measured transport directly, at a replay rate derived from the plan and verified against wall time, over a median of 125 events per run rather than seven, at $N \in \{1, 9, 12\}$ with 15 replicates each (Table S3). The numbers are computed over audit-surviving runs only — the campaign's own retention, the gate's effect on every estimate (at most 0.003 ms on the shift) and the worst-case imputation flip point are Table S11 in S18 — and the harness timestamps in microseconds on one clock, so neither failure mode of this paper touches the numbers that follow. The picture is sharper and it is not the one E1 painted. Broker transport is Kafka ≈ 0.54 ms against Redis ≈ 0.11 ms: a Hodges–Lehmann shift of 0.41 ms, Kafka slower, with a bootstrap 90% interval of width ± 0.006 ms and $p < 10^{-26}$ at every N. The two systems are *not* statistically indistinguishable; Redis's in-memory `XADD` really is several times faster per operation than Kafka's replicated-log append, which is the direction the gray-literature comparisons report.

And yet the same three estimators — Welch, percentile bootstrap and Hodges–Lehmann — agree that the difference is *equivalent within one millisecond* at every N, because that shift is comfortably inside that margin. Both statements are true and neither is idle: against the seconds-scale human annotation latency that dominates any live sports feed's end-to-end budget, a broker difference is parts in 100,000 of the end-to-end path, so for choosing a broker it is noise; but it is a real, tight, reproducible effect that a properly powered measurement resolves and a seven-event one cannot. The shift is also flat across concurrency (0.408, 0.416, 0.417 ms at $N = 1, 9, 12$, audit-gated; S18), so neither system degrades. A part of even this residual is the harness rather than the broker: the main text shows that 0.07 ms of the gap is the asymmetric acknowledgment timestamp — a different quantity from the inter-host clock offset of Section IV-A, which happens to be the same size —, leaving a true broker-transport difference near 0.34 ms. That 0.07 ms is a concrete instance of the blind spot of the main text: it is a bias smaller than the quantity measured, so it never turns a value negative and the consistency check passed both corpora that carried it. The check guarantees a corpus is not *catastrophically* wrong; resolving a 0.07 ms harness artifact still took a controlled experiment.

An independent replication confirms it. A second powered campaign, run days apart with the same protocol, gives Hodges–Lehmann shifts of 0.397, 0.412 and 0.413 ms at $N = 1, 9, 12$ — inside the 90% interval of the first campaign at every level, with overlapping intervals throughout. The 0.41 ms advantage is not a fluctuation of one campaign.

This refines E1 rather than contradicting it, and it sharpens the contradiction with the first-answer table in the main text: the accelerated corpus reported Redis *degrading* with concurrency; the powered real-time measurement finds Redis uniformly *faster* and flat. The reversal is the audit's doing, and the measurement's.

Why E1 saw near-equality, tested directly: If the disagreement between E1 (< 0.1 ms apart) and the powered campaign (0.41 ms apart) is the seven-event window and nothing else, then restricting the powered runs to their first seven events must reproduce E1, and the full runs must reproduce the powered answer. Both hold: full-length shifts 0.381, 0.417, 0.414 ms against the powered 0.41; first-seven shifts 0.088–0.248 ms against E1's 0.021–0.116, with absolute medians matching as well. The

TABLE S3
POWERED TRANSPORT REPLICATION AT A VERIFIED TRUE-REAL-TIME RATE, OVER A MEDIAN OF 125 EVENTS PER RUN (NOT THE SEVEN OF THE MAIN TEXT), AUDIT-GATED (SECTION S18). MEDIANS IN MS; THE HODGES–LEHMANN SHIFT IS $Kafka - Redis$ WITH A PERCENTILE-BOOTSTRAP 90% INTERVAL. THE BROKERS ARE EQUIVALENT WITHIN 1 MS AT EVERY N (ALL THREE ESTIMATORS) YET CLEANLY DISTINGUISHABLE: REDIS IS ≈ 0.41 MS FASTER, AND THE SHIFT DOES NOT GROW WITH CONCURRENCY.

N	Events/run	Kafka	Redis	HL shift [90% CI]
1	148	0.512	0.102	$0.408\ [0.389, 0.419]$
9	112	0.542	0.120	$0.416\ [0.408, 0.423]$
12	121	0.540	0.114	$0.417\ [0.408, 0.424]$

window explains the disagreement with one parameter fixed in advance; the full test, including the start-up decomposition it rests on, is supplementary material S5.

S3. THE TWENTYFOLD END-TO-END GAP WAS SEND LAG, AND WE WITHDREW THE CLAIM

TTI and transport give opposite answers in Table S2. Of Kafka's 105.5 ms, 102.9 ms is producer send lag against Redis's 1.8 ms, while broker transport is 0.84 and 0.80 ms. An earlier version of this paper reported that twentyfold end-to-end gap as a finding, attributed it to the client's sender thread waking from idle, and built a practitioner recommendation on it.

That gap does not survive re-measurement, and the reason is a second instance of this paper's own thesis. We report it at length for the same reason we reported the first, and we retain the figure that once carried it (Figure S2) rather than deleting the evidence.

The offset is a start-up cost, not a per-event constant: Re-measuring at $N{=}1$ on the same driver and the same broker, at a replay rate derived from the plan and verified against elapsed wall time, gives a median producer send lag of 1.59 ms for Kafka, not 103 ms — with a *maximum* of 103.5 ms. Two independent instrumentation paths agree: the per-event loop trace and the per-run summary. The 103 ms is real, but it is paid once per run, not once per event.

We cannot call this a repetition of E1's condition, because E1's replay rate is not recoverable (the main text). That is why the argument below does not rest on the comparison between the two campaigns at all. It rests on a sweep that is internally controlled — one campaign, one rate, one match, only the window varying — and on an arithmetic relation that holds at every rate E1 could have used.

A controlled test that separates the two: A median cannot distinguish a per-run cost from a per-event one, because how much of the cost the median sees depends on how many events the run contains — which is precisely what misled us. The quantity that *does* separate them is the *number* of affected events per run. A per-run cost predicts that number is fixed however long the run gets; a per-event constant predicts it grows with the run. We therefore replayed the same match at true real time under three observation windows, holding everything else fixed, and counted events waking more than 50 ms late (Table S4).

The count does not move (Figure S1). Going from the 60 s window to the 600 s window multiplies the events emitted by 8.9 — 57 to 507 — while the number waking more than 50 ms late stays at exactly four and exactly one send blocks, so the share of events paying the cost falls from 7.0% to 0.8%. The median is flat at 1.6 ms throughout and the maximum flat at 103.5 ms: the cost is present in every run and dilutes in every run, which is the signature of something paid once. Had it been a per-event constant, the count would have grown with the window and the median would have stayed at 103 ms.

A once-per-producer cost of this kind is documented upstream: a Kafka producer performs a metadata lookup before its first send to a topic it has not yet cached [S4]. We did not isolate that call, so we do not claim it *is* the mechanism — the window sweep constrains the cost's *shape*, not its origin. We note it because the shape we measure and the shape the client documents agree, and because a reader deciding whether this generalize beyond our testbed should know the behavior is a property of the client rather than of our rig.

Redis was run in the same sweep on the same events, with the same loop trace, and has *no* events waking more than 50 ms late and *no* blocking send at any window. Its worst single event anywhere is under 25 ms. Instrumenting both configurations identically matters here for a reason internal to this paper: an earlier version of this table reported Redis's counts as zero when its producer in fact carried no trace, which is the absence of a measurement dressed as one — exactly the failure the paper is about. We added the trace to the Redis producer and re-ran both configurations together.

The mechanism, read directly off the loop trace: The per-event trace shows the same five-event signature in every Kafka run, at every window:

event	wake late (ms)	send blocked (ms)
0	0.17	**102.60**
1	**102.96**	0.07
2	**103.08**	0.05
3	**103.17**	0.04
4	**103.30**	0.05
5	0.94	0.27
6	1.42	0.06

The first `produce()` blocks for 102.6 ms fetching metadata and creating the topic. The replay loop is single-threaded, so every event due while it is blocked wakes roughly 103 ms late and then sends in tens of microseconds. There are exactly four of them, and the reason is the sport rather than the harness: this plan's first five events all carry $t_{\mathrm{sim}}{=}0$, because a kickoff is recorded as a pass, a ball receipt and a carry inside the same second. That is not particular to this match — every one of the eleven plans in our corpus opens with at least five events at $t_{\mathrm{sim}}{=}0$. A football feed therefore delivers its densest burst precisely when the producer is coldest, which is the worst possible alignment for a start-up cost and the reason it lands in the median rather than the tail. From event 5 the loop is in steady state at about 1 ms. One cost, paid once, is charged to five events; the sixth onward pays nothing. Redis shows no such prologue because `XADD` to a new stream creates it in the same round trip.

That closes it. Five events carry the cost, seven were matched, and the median of seven values of which at least four are 103 ms *is* 103 ms. The published figure is the arithmetic of its own prologue.

Why the original corpus could not see that, and how we know: The E1 runs behind Table S2 replayed the first 600 s of match time, which in this plan holds **507 events**. They matched a **median of seven** — 745 events across 100 Kafka runs, a match rate near one percent. The window was not the problem; the join was. And the seven that survived it are not a random seven.

That last claim is arithmetic rather than inference. Those runs report a median producer send lag of 102.93 ms over seven values, and a median of seven values at 102.93 ms requires *at least four of the seven* to be 102.93 ms or greater. The loop trace says exactly how many events per run are ever that late: four (Table S4), and always the same four — those due while the first send blocks. The matched set must therefore consist almost entirely of the prologue. The plan's first five events are all timestamped at $t_{\mathrm{sim}}{=}0$, so they are the first candidates a consumer sees and the first a lossy join keeps.

The failure thus compounds: a sample of seven is small, but a sample of seven *selected towards the contaminated events* is worse than small. Redis reports 1.75 ms on the same events because opening a stream with `XADD` does not pay what Kafka's first send pays in metadata fetch and topic creation, so the same selection costs it nothing.

Why not knowing E1's replay rate does not matter here: The blocking first send lasts 102.6 ms of *wall* time, so at a replay rate of R times real time it spans $0.103R$ seconds of *match* time. Evaluating that against the plan for each rate E1 could have run at:

Replay rate	Match time spanned	Events inside the prologue
1×	0.10 s	5
120×	12.3 s	16
1200×	123.1 s	112

E1 matched a median of seven events per run. At every one of the three candidate rates the prologue already contains at least four events — enough to carry the median of seven — so the matched sample's median is the start-up cost, which is what the corpus reports. The conclusion is therefore insensitive to the replay rate; the main text goes on to recover that rate from the same start-up cost, but the withdrawal does not depend on the recovery.

This also explains the three properties we previously offered as evidence, and each of them is equally consistent with a start-up cost: a per-run constant looks *constant* within a run; it is *concurrency-invariant* because every run pays exactly one regardless of N; and it is *rate-dependent* because accelerated replay packs more events into the window and dilutes it. We read three signatures of a per-event mechanism into measurements that a per-run mechanism explains at least as well.

The general lesson: The integrity check of the main text does not catch this. Every one of those runs is causally consistent; nothing is negative; the medians are stable to three significant figures across a hundred runs. What went wrong is that a median over seven events was reported as a property of the system rather than of the window, and the tight agreement across runs made it look robust precisely because the artifact is deterministic. A benchmark can be internally consistent, gate-clean, and still measure its own start-up. We therefore add to the main text the requirement to report events-per-run alongside any latency percentile, since a percentile over single-digit samples is a statement about the harness.

What we claim instead: On broker transport — the quantity that survived the main text and is measured over the same events for both systems — Kafka and Redis remain equivalent within one millisecond (the main text). We make no end-to-end claim. The twentyfold figure is withdrawn.

The benchmark's own output corroborates the insensitivity: The benchmark's own output corroborates this without our instrumentation: across eight runs all 32 reported percentiles are whole milliseconds (only the averages, means of integers, are fractional), and three runs report $\mathrm{p50} = \mathrm{p95} = \mathrm{p99} = \max = 1.0$ — not a narrow distribution but one with a single value in it, printed to three decimal places.

S3.1. The delay sweep failed its manipulation check

We withdraw the intermediate points, and the reason is worth more than they were. An earlier version supported H1 additionally with a `netem` delay sweep, reporting a rank correlation across injected delays of 0 to 50 ms. We then re-ran that sweep at a verified real-time rate with a manipulation check, and the check failed — not because the treatment was too weak, but because it does not act on the quantity at all. Injecting delay at the broker moves end-to-end TTI almost exactly as commanded (3.72 → 23.61 ms across 0 → 20 ms) while broker transport stays flat (0.535 → 0.482 ms; Table S5).

TABLE S4

OBSERVATION-WINDOW SWEEP SEPARATES A PER-RUN COST FROM A PER-EVENT ONE. SAME MATCH, SAME DRIVER, SAME BROKER, $N{=}1$, TRUE REAL-TIME REPLAY; ONLY THE WINDOW CHANGES. BOTH PRODUCERS ARE LOOP-TRACED, SO THE COUNTS FOR THE TWO SYSTEMS COME FROM THE SAME TRACER. EVENTS EMITTED GROW $8.9\times$ ACROSS THE SWEEP, BUT THE NUMBER OF KAFKA EVENTS WAKING MORE THAN 50 MS LATE DOES NOT MOVE, AND EXACTLY ONE SEND BLOCKS IN EVERY RUN; REDIS HAS NEITHER AT ANY WINDOW. COUNTS ARE MEDIANS OVER THREE REPLICATES FROM THE PER-EVENT TRACE; PERCENTILES COME FROM THE PER-RUN SUMMARY.

	Window	Events		Scheduling lag (ms)		Events > 50	Blocking
	(s)	emitted	matched	median	max	ms late	sends
Kafka	60	57	57	1.56	103.4	4	1
	180	148	148	1.61	103.5	4	1
	600	507	465	1.58	103.5	4	1
Redis	60	57	57	1.50	17.4	0	0
	180	148	148	1.56	20.4	0	0
	600	507	465	1.53	24.8	0	0

TABLE S5

WHY THE DELAY SWEEP IS NOT AN EFFECT-SIZE MANIPULATION. INJECTED ONE-WAY DELAY AT THE BROKER, VERIFIED PRESENT AT THE INTERFACE, AT A VERIFIED TRUE REAL-TIME RATE. TTI TRACKS THE INJECTION; BROKER TRANSPORT DOES NOT MOVE, BECAUSE THE SAME DELAY IS ADDED TO BOTH ARRIVALS WHOSE DIFFERENCE DEFINES IT.

Injected delay	TTI median	Transport median	Transport IQR
0 ms	3.72 ms	0.535 ms	0.162 ms
1 ms	4.67 ms	0.508 ms	0.166 ms
5 ms	8.66 ms	0.494 ms	0.159 ms
20 ms	23.61 ms	0.480 ms	0.150 ms

The cause is that the delay is **common-mode**. It delays the acknowledgment travelling broker-to-producer and the record travelling broker-to-consumer by the same amount, and transport is the *difference* of timestamps taken at those two arrivals, so the injected delay cancels exactly. A manipulation can be applied correctly, verified present at the interface, and still not touch the quantity under study — which is the same lesson as the rest of this paper, arriving this time in our own experimental design rather than in our harness. The original sweep's correlation was therefore never an effect-size relationship, and we report the sweep only as the negative result it is.

S4. OUR OWN REPLAY-RATE RECORD DID NOT SURVIVE THE AUDIT

Auditing the corpus turned the same scrutiny on our own record-keeping, and it did not survive it either. The main text summarizes this episode; this section is the full record, kept because the protocol rule it produced is weaker without it.

Replay plans built from StatsBomb carry a **baked-in** $120\times$ **time compression**: a plan's emission offset is already the match time divided by 120. The producer's `--speedup` flag then multiplies that. So `--speedup 1` against such a plan is *not* real time, it is $120\times$; true real time needs $1/120$. The same flag means different things depending on which plan it is pointed at, and getting it wrong is silent: the run completes, the wall time looks unremarkable, and the numbers stay plausible. We lost a campaign to it before adding a pre-flight that derives the value from the plan and then checks the achieved rate against elapsed wall time.

The part we cannot repair is retrospective. **No surviving artifact records the achieved replay rate of any run we report.** Several superseded Testbed A scenarios record the nominal `--speedup` *flag*, which is not the same thing: converting a flag to a rate requires the plan's compression, and every plan those files name has since been deleted. Everywhere else the orchestrator wrote `speedup` into a per-campaign summary that lived beside the raw per-run data, and for the earliest corpora only the aggregated CSVs were kept. For the E1 corpus of Section VIII-A of the main text the surviving evidence is contradictory: the commit that introduced it states the runs were made at true real time, the campaign script reconstructed afterwards specifies `--speedup 10`, and the flag's meaning was corrected 21 hours later the same day. E1 was replayed at $1\times$, $120\times$ or $1200\times$, and we cannot say which.

The rate is recoverable from the measurements, if not from the metadata: Having argued all paper that a physical constraint can decide what record-keeping cannot, we applied the same move here, and it works. The start-up cost of Section S3 is a clock: it lasts a fixed 102.6 ms of *wall* time, so the number of events it swallows depends directly on the replay rate. Counting how many events per run carry it therefore reads the rate back out of the data.

The E1 runs sort into cells by how many events they matched, and one cell is diagnostic. Fifteen Kafka runs matched exactly eight events, and their median send lag is $\mathbf{52.34}$ ms (range 51.60–53.01) — not 103 ms, and not 1.6 ms. A median of eight values is the mean of the fourth and fifth. The only way to land midway between the two modes is for exactly four of the eight to be late, so the fourth value is a fast event and the fifth a slow one: $(1.59 + 103.5)/2 = 52.6$ ms against 52.34

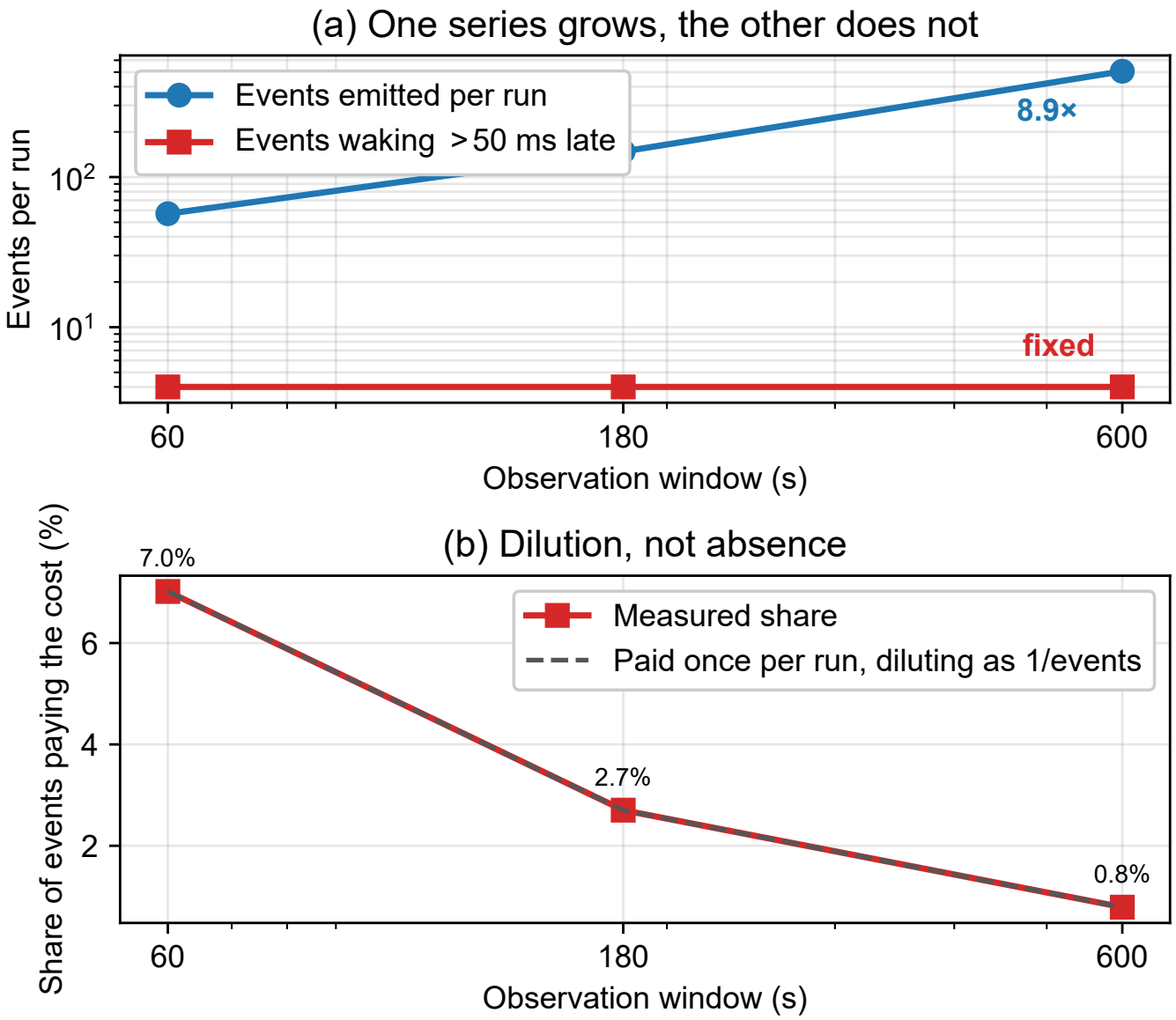

Fig. S1. Window sweep, as a picture. (a) Events emitted per run grow 8.9× while the number waking more than 50 ms late stays at four — a per-event constant would put the two series on parallel lines. (b) The same fact as a proportion: the share of events paying the cost falls as $1/\text{events}$, the dilution a once-per-run cost predicts. Redis, on the same events, has no affected events at any window.

observed. Four is also exactly what the loop trace shows at a verified real-time rate (the window table in the main text), and it is what the plan predicts, since five events share $t_{\text{sim}}{=}0$ and the first of them pays the block rather than inheriting it.

That cell excludes the accelerated alternatives outright. At 120× the prologue spans 12.3 s of match time and holds 16 events; at 1200×, 123 s and 112 events. Under either, all eight matched events would be inside it and the cell would read 103 ms. It reads half that. **E1 was replayed at true real time**, the commit record was right, and the campaign script reconstructed afterwards describes a different, earlier campaign. The remaining cells agree: runs matching five or seven events read 103 ms, as four late events out of five or seven must give.

The inference has since been checked against the original script, and it holds. Archiving the cloud driver before releasing it turned up the campaign as actually invoked, in a shell script in the operator's home directory that no part of the repository had ever referenced. It specifies `--speedup 0.008333` — 1/120, against a plan already compressed 120×, which is true real time — and its log's first line, `E1 N=1 reps=8 00:39:40`, names the run directory the E1 artifact's first row still points at. The arithmetic above and the flag agree.

We report that as a check rather than as the answer, because the order matters. The rate was determined from the measurements while the metadata was genuinely unavailable, and the script surfaced afterwards, by luck, from a machine that was about to be switched off. What the agreement establishes is not that E1's provenance was fine all along — it was not — but that the

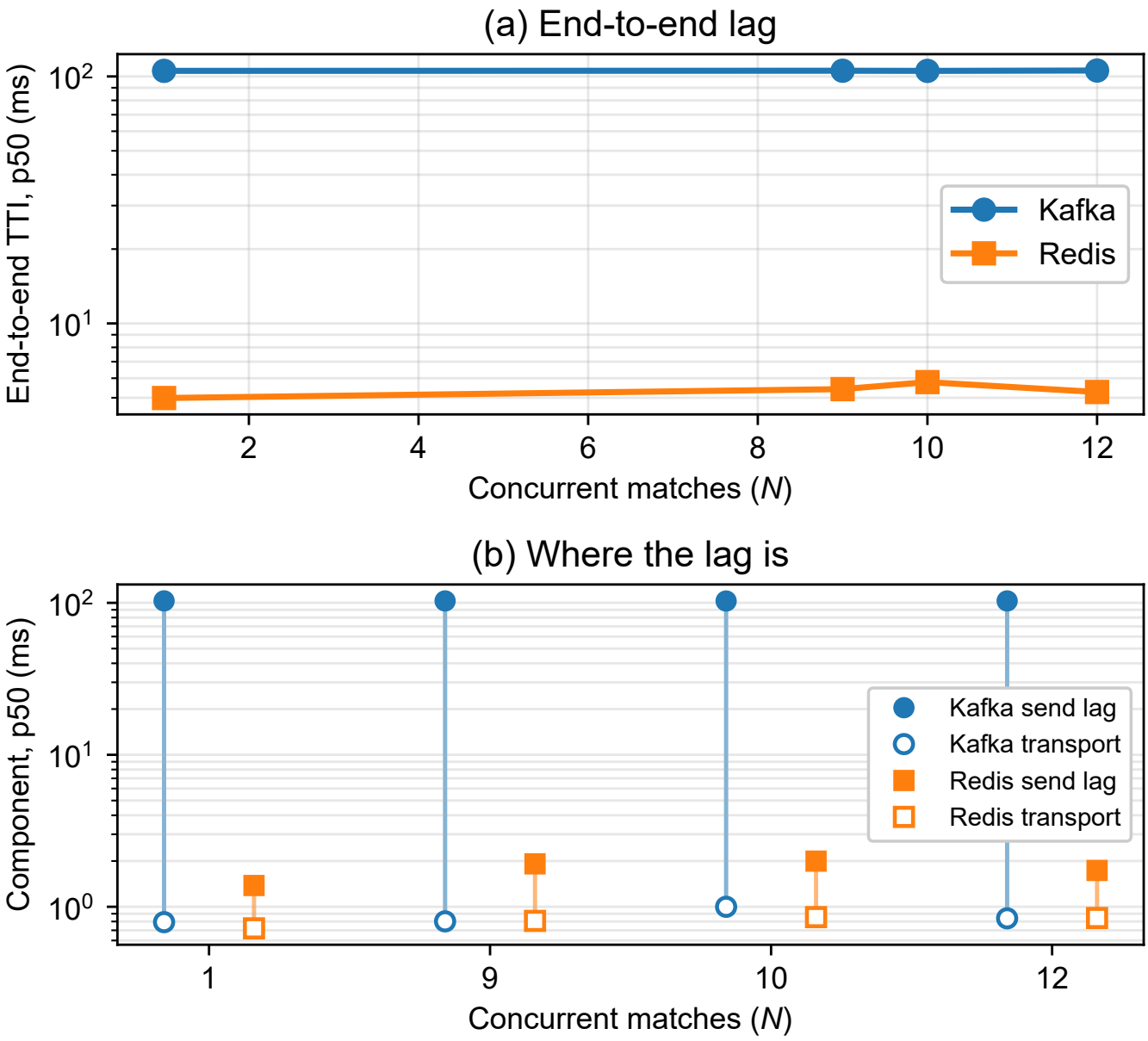

Fig. S2. **Withdrawn.** (a) End-to-end delay against concurrency as originally measured. (b) The decomposition that appeared to place the whole gap in producer send lag. Both panels are retained as evidence for the main text: the runs behind them matched a median of seven events each, so the apparent per-event offset is a per-run start-up cost. Broker transport, the lower series in (b), is unaffected and is what the paper claims.

recovery method returns the right answer when there is an answer to check it against. That is the one thing a physical-constraint argument can rarely demonstrate about itself.

The same script also explains a feature we had described and not accounted for. It passes `--max-t-sim 2`: two seconds of match time per run. The median of seven matched events that makes E1 under-powered, and that Section S2 treats as an opening burst, is that window. The burst is real and is why those seven events are the densest in the match; the reason there were only seven is that the campaign asked for two seconds.

We leave the disclosure standing rather than deleting it, for two reasons. The recovery was available only because the artifact we were chasing happened to be a wall-clock constant with a sharp signature, and the confirmation only because a virtual machine had not yet been destroyed; a reader should not have to reconstruct an experimental parameter by arithmetic on its results, and next time there may be nothing to reconstruct it from and nothing left to check it against. And the exercise makes the protocol rule concrete: what was missing was one number, written down once.

What the gap touched: The audit is unaffected either way: a negative transport is causally impossible at any replay rate, and the rejection counts are arithmetic on surviving per-event data. The withdrawal in Section V-A of the main text is unaffected by construction — we give the argument in a form that holds at every candidate rate. The rules of Section VIII-B

TABLE S6
ONE SET OF RUNS REPRODUCES BOTH PUBLISHED ANSWERS. TRANSPORT MEDIANS (MS) FROM THE POWERED CAMPAIGN, COMPUTED OVER EVERY EVENT AND OVER EACH RUN'S FIRST SEVEN EVENTS IN EMISSION ORDER — THE WINDOW E1 SAW. THE ALL-EVENTS COLUMNS REPRODUCE THE POWERED 0.41 MS SHIFT; THE PROLOGUE COLUMNS REPRODUCE E1'S NEAR-EQUALITY (THE MAIN TEXT) IN BOTH SHIFT AND ABSOLUTE LEVEL. THE WINDOW, NOT THE BROKERS, ACCOUNTS FOR THE DISAGREEMENT.

		All events			First 7 (prologue)		
N	Runs	Kafka	Redis	Shift	Kafka	Redis	Shift
1	5	0.476	0.095	0.381	0.831	0.743	0.088
9	45	0.533	0.116	0.417	0.939	0.810	0.129
12	55	0.527	0.113	0.414	1.018	0.769	0.248

of the main text were measured after the pre-flight existed, at a derived and verified rate. Section VIII-A of the main text's external validity is restored by the recovery above, and the recovered script now documents it as well — though the inference stood on its own before the script appeared, and is what we would still be relying on had the machine been released a day earlier. Section VIII-B of the main text carries the rule that would have made all of this unnecessary.

S4.1. Provenance-gap episode and the co-location withholding

S4.2. No artifact records the earliest corpus's replay rate

Auditing the corpus turned the same scrutiny on our own record-keeping, and it did not survive it either: no surviving artifact records the achieved replay rate of the earliest corpus. Replay plans carry a baked-in $120\times$ compression that the producer's flag then multiplies, so the candidate rates are $1\times$, $120\times$ and $1200\times$, and the records that exist disagree. The rate was recovered from a physical constraint: the client's start-up cost is a wall-clock constant, so the number of events it swallows encodes the rate, and one diagnostic cell whose median reads 52.34 ms (range 51.60–53.01 across fifteen runs) — midway between the two modes at 1.6 and 103.5 ms — fixes it at four late events and hence at true real time. What the gap touches is bounded: the audit is unaffected, since causal impossibility holds at any rate; the withdrawal of the main text is stated in a form that holds at every candidate rate; and the main text's external validity is restored by the recovery. The full episode — recovery arithmetic, per-claim exposure, the disagreeing records — is supplementary material S4. The rule it produced — write the number down — is in the main text.

A manipulation that did not manipulate, and what it cost: We also withhold a co-location campaign (E-A8). The intent was to shorten T_{true} by removing the network hop — the opposite of the payload sweep, and a useful check that the T_{true} result is not an artifact of how we lengthened it. It did not work. Moving the broker onto the driver made transport *longer* at idle, $0.512 \to 0.573$ ms, because it traded network latency for CPU contention on the same cores; under 88% load it moved transport by $1.03\times$, which is no manipulation at all. The negative-span rates duly failed to separate (intervals overlapping in both cells). Reporting those rates as a null would misdescribe the experiment: nothing was manipulated, so nothing was tested. We record it because the pattern is itself informative. Two of our three attempts on this axis failed for structurally similar reasons — netem delayed both paths and canceled in the difference, co-location traded one delay for another — which suggests T_{true} is a quantity that resists manipulation in a shared-resource system, because every route to shortening it lengthens something else. The payload sweep worked because padding adds serialization without contending for the scheduler, and we were fortunate that it does.

S5. E1'S SHIFT IS CONFINED TO THE FIRST SEVEN EVENTS

Moved from Section VIII-A of the main text: the complete windowed re-analysis showing E1's near-equality is the observation window and nothing else.

Why E1 saw near-equality, tested directly: The two campaigns disagree by more than power: E1 puts both systems near 0.8 ms and within 0.1 ms of each other, while the powered campaign puts them at 0.53 and 0.11 ms and 0.41 ms apart. Attributing that to seven events is an explanation only if it survives being checked, so we checked it. If the difference is the observation window and nothing else, then restricting the *powered* runs to their first seven events — in emission order, the same events E1 would have caught — must reproduce E1, and the same runs must give the powered answer over their full length. That is a prediction with one free parameter fixed in advance, and it can fail.

It does not fail (Table S6). Over all events the runs give shifts of 0.381, 0.417 and 0.414 ms, reproducing the powered 0.41 ms. Over their first seven they give 0.088, 0.129 and 0.248 ms — E1's wandering near-equality, and of the same magnitude as E1's own 0.021, 0.116 and 0.053 ms. The *absolute* medians match as well, which the shift alone does not force: the prologue puts Kafka at 0.83–1.02 ms and Redis at 0.74–0.81 ms, against E1's 0.79–1.00 and 0.72–0.86. One set of runs, one rate, one protocol, two windows, both published answers.

So E1 is not wrong and the campaigns do not contradict each other. E1 measured the opening burst, during which both systems are slow for reasons this section sets out — topic auto-creation, connection setup, and the first blocking `produce()`

— and that shared start-up cost swamps a 0.41 ms difference. As the run lengthens the cost dilutes, and the brokers' steady-state separation emerges. The seven-event corpus answered a question about start-up while appearing to answer one about brokers, which is the same failure of window selection as the end-to-end withdrawal of Section S3, one level up.

S5.1. Where the knee sweep placed the load conditions

Moved from the main text's mixture development: the load-axis post-mortem in full. The main text keeps the admissions and the numbers.

Both load-axis predictions failed, including ours: The knee sweep placed conditions at achieved ρ of 0.881, 0.920, 0.950, 0.970 and 0.990, where the candidate forms diverge most; the earlier ladder stopped at 0.878 and everything above it collapsed onto a degenerate $\rho = 1.000$, which is why Section VIII-B of the main text could not settle the question. With those points in hand, M/G/1 fits *worse than the mean* ($R^2 = -0.05$ against a fitted exponential's 0.93). The withdrawal does not merely stand: the forms are now separated, and M/G/1 is the one ruled out.

The bounded prediction fared better but also failed. Over $\rho = 0.881$ to 0.990 the rate grew by a factor of 1.44. M/G/1 predicted 13.8; the bracket we registered before the sweep predicted 2.45–3.07. The rate saturates, as a bounded mechanism requires — but harder than our own extrapolation allowed, and a prediction that misses low is still a prediction that missed. We report it as a failure rather than as the nearer of two misses.

The two failures have one cause, and E-A5 names it. Our bracket was still parameterized in ρ, through $p = \rho^C$; E-A5 shows ρ does not determine occupancy. What survives is the mechanism — established by manipulation — and not any curve in ρ, ours included.

S6. The even-denominator exclusion was wrong, and what replaced it

We report the correction because the first version was weaker and we published it to ourselves: Treating $100/q$ as a point prediction made the even denominators look like failures, and we introduced an exclusion for them — an configuration cannot test a grid the continuous value sits on, so $q = 2$ and $q = 4$ were set aside. That exclusion was defensible and pre-registered: it was derived from the $q = 4$ result and committed at 14:09Z on 2026-07-27, and the first cell of the first odd-q configuration was measured at 15:06Z, so the odd denominators were run *because* the rule said they would separate. It was also unnecessary. Under the main text an on-grid configuration is a prediction of a flat configuration, so $q = 2$ and $q = 4$ are evidence *for* the account rather than cases to be excused. We had reached for a law that excluded its exceptions when one that explains them was available.

What exposed it was checking whether the exclusion survived re-estimating one of its own inputs. The continuous value is not constant across rate — it runs 51.2% at 333 msg/s down to 47.5% at 889, a trend of 4.15 points against a replicate noise of 1.55 — and under a rate-local estimate $q = 4$ ceased to be degenerate and became a failure. A classification that flips on that choice is not a classification. Measuring position against the cell half-width instead of a fixed noise floor removes the sensitivity: every call in the main text is the same under either estimate.

A second prediction of ours failed, and no reading of it rescues the number: We recorded, before running it, that $q = 5$ would put 2 of 5 phases across a boundary and so retain about 40%. The ten replicates gave a median of 51.04% — inside the incommensurate range and 11.0 points from what we wrote down. The error was again naming one branch of a two-branch prediction: with T_{true}/τ at a half, 2 *or* 3 phases cross depending on where a run's initial phase falls, so a replicate is predicted at 40% or 60% and the average of many is near 50. The values — 40.71, 41.07, 41.96, 42.11, 46.58, 55.50, 57.02, 58.60, 59.05 and 60.04 — are bimodal about exactly those two branches, five on each, which is what the account requires and what no continuous account produces. The miss grew when the configuration did: at five replicates the median stood 6.6 points from the registered value and at 10 it stands 11.0, because the upper branch was under-sampled in the smaller set. The grid held; the point we picked on it did not.

Where the account is approximate, stated rather than trimmed: Two observations sit outside it, and a third did until the configuration was filled. One replicate at 500 msg/s sits at 18.0, which a run confined to a single phase cannot produce. We read this as the limit of the idealization: a nominal rate is $q = 1$ only if the pacing is exact for the whole run, and over three minutes a drift of a few parts per million walks the phase across a boundary part-way through. The grid that governs retention is the one the pacing realizes, not the one the nominal rate implies. Second, $q = 2$'s spread of 17.61 rests on a single replicate at 68.03; the other four span 3.10 points, which is the incommensurate noise level and exactly what an on-grid configuration should show. We report the spread over all five rather than the four that agree.

The one that resolved is worth recording, because it is an argument for one of our own rules. The $q = 1$ configuration at 250 msg/s was listed here as a departure from the account when it had three replicates and spread little more than half its cell. At five it spreads 98.73 of 100, which is the full cell width the model predicts, and the anomaly is gone. It was never a departure; it was an configuration reported before it had enough replicates to show what it was doing — which is exactly what Section VIII-B asks benchmark authors to count before they quote.

PART II. THE MECHANISM, AND THE LAW IT PRODUCES

This part is the argument. Sections S7 to S9 establish the scheduling failure by manipulation: priority and geometry each moving the rate on their own, what the payload sweep does and does not support, and the two-state model with its tracer checks. Sections S10 to S13 establish the deletion failure — retention on the grid, membership configuration by configuration, every real-time-priority pair with the exposure curve they make, and what the brokers actually do. Sections S14 to S17 derive both: the statistics of a discarded zero, the clocksource bounded from a measurement, why the acknowledgment timestamp is late, and the 1970 counter note behind the grid geometry.

S7. PRIORITY AND GEOMETRY EACH MOVE THE RATE ON THEIR OWN

The experiment that settles it: Move one of the two and hold the other fixed: raising the timestamping processes to real-time priority makes them preempt the background load rather than queue behind it, so occupancy falls sharply while utilization does not move — and unlike our delay injection, which canceled in the subtraction (the main text), priority acts on the timestamping threads themselves, where the model locates the failure. An occupancy mechanism predicts collapse toward the running-state floor; any account in which the rate is a function of ρ predicts no change, because ρ is unchanged by construction. The result is unambiguous (Section S26): utilization held to 0.001 (the pre-registered manipulation check passes), and the negative-span rate fell from 0.132 to 0.0034 at $\rho = 0.75$ and from 0.305 to 0.0057 at $\rho = 0.88$ — factors of 39 and 54, disjoint Wilson intervals, both residual rates on the floor the model names in advance, $C_0 \approx 0.004$.

Utilization is not the variable, shown at identical utilization: Two loads can reach the same ρ by different means: k cores flat out leaves $C - k$ genuinely free, while duty-cycling every core leaves none. If the mechanism is whether the timestamping thread can find a core, the second should be worse at equal ρ — and it is (Section S26): at $k = 6$ both configurations sit at $\rho = 0.7531$, identical to four decimals, same machine, same hour, and the negative-span rates differ by $2.07\times$ ($z = 10.3$ over 2 985 matched events per cell, disjoint Wilson intervals). A function of ρ returns one value for one input. The gap narrows at $k = 7$ ($1.06\times$) — the mechanism showing itself: one free core out of eight is nearly as useless as none. *A full replication (E-A6b) corrects us on one point*: it reproduces the load-bearing cell almost exactly ($\rho = 0.7531$ in all four configurations; $2.05\times$ against $2.07\times$), reproduces $k = 5$ in direction but not size, and at $k = 7$ resolves a small difference ($1.19\times$, $z = 3.46$) where the original could not — so we withdraw an earlier draft's reading of that convergence as a null. Both campaigns agree on the shape: twofold at $k = 6$, near-parity at $k = 7$.

The other side of the inequality, and it moves the rate the wrong way: The mechanism predicts that a *slower* path is a *more reliable* measurement, because the same stalls have a longer interval to outrun. Padding the payload lengthens T_{true} without touching the scheduler (Section S26): transport rose $76.9\times$ while the negative-span rate *fell* $4.1\times$, monotonically, with utilization held to a 0.012 spread — and pushed slightly *up* by serialization, against the observed fall, so the effect is measured against its own confound. The same numbers constrain the stall distribution sharply: a $77\times$ larger threshold bought only a $4.1\times$ lower rate, so the tail barely thins across two orders of magnitude — which is why mean-based counters could not see the effect and the direct trace was necessary.

S7.1. The overnight campaign, prediction by prediction

Section VI-B reports the outcome in a sentence. This is the tally, because a pre-registration is worth what its failures are worth and three of these failed.

Sixty-three cells ran at $n \geq 5$ with six predictions and their falsifiers registered in advance. Every cell completed validly. One prediction was confirmed, three falsifiers fired, one was not evaluable and one split.

Duration invariance, which killed the drift account: If retention were set by an accumulating drift, longer runs would sit further from the grid than shorter ones. Intermediate counts over one, three and ten minutes are 1, 1 and 0, with the vertices pinned throughout. Nothing accumulates. What remains is per-send jitter, which is the mechanism the retention law of Section VI assumes and not one we chose after seeing the data.

The linear extrapolation that failed: One registered prediction assumed retention would extrapolate linearly in rate across the incommensurate configurations. At 1053 and 1219 msg/s the measured values are 47.2 and 48.2% — flat where the prediction wanted a slope. The grid law does not predict a trend in rate, only a position against the tick, so the failure is of the extrapolation rather than of the law; we record it because it was registered and it did not come true.

S8. WHAT THE PAYLOAD SWEEP FITS, AND WHAT IT DOES NOT SUPPORT

The payload-sweep fit, demoted here from the main text: Over a $77\times$ payload span, least squares of $\log \Pr$ on $\log T_{\mathrm{true}}$ over 4 payload levels gives

$$\Pr[\text{span} < 0] = 0.238\, T_{\mathrm{true}}^{-0.339}, \qquad R^2 = 0.990, \tag{S1}$$

with a Student-t 95% interval on the exponent of 0.234–0.443 at 2 degrees of freedom. E-A10b repeats the sweep on a different day and returns 0.344 against 0.339; the prefactor drifts (0.238 against 0.216). Four points do not earn an equation in a main text, which is why this one is here, and Fig. S3 plots them.

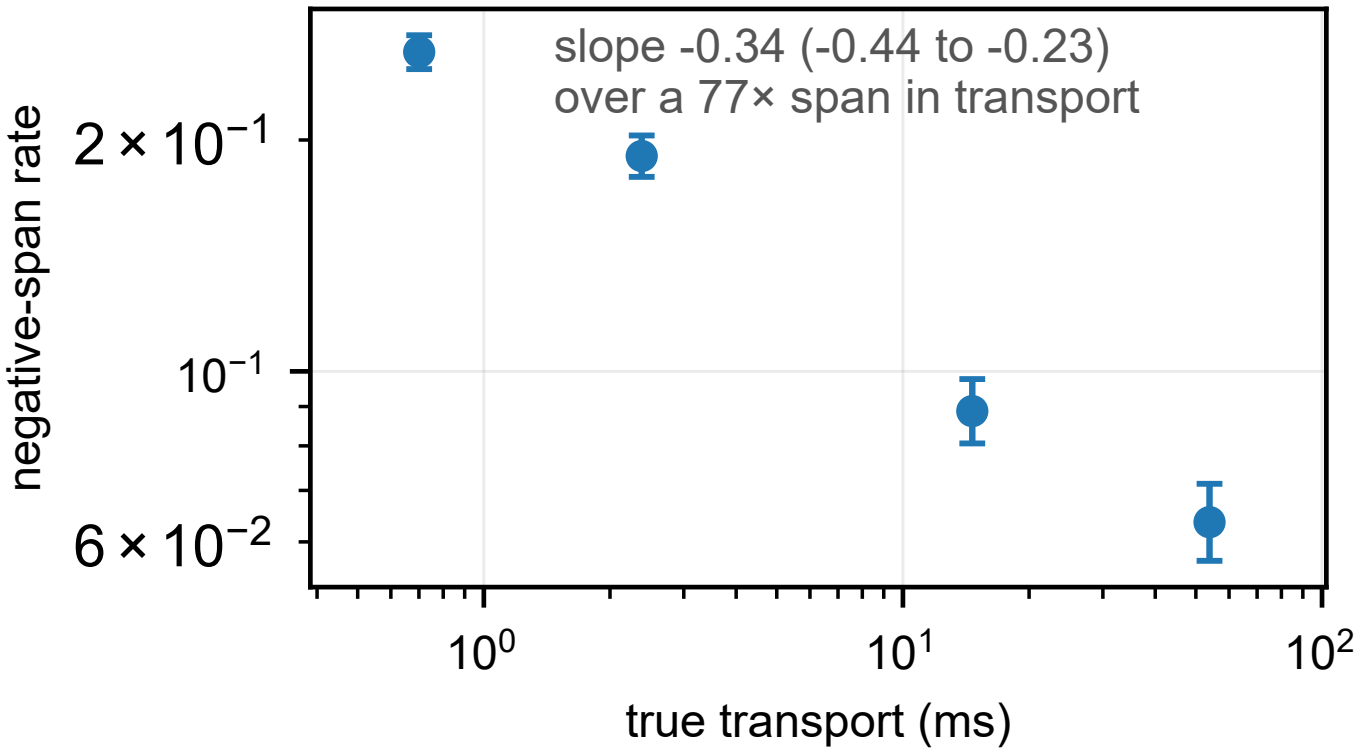

Fig. S3. **Longer delivery inverts less often.** Negative span rate against true transport over the payload sweep, Wilson 95% intervals. The same stall distribution overlaps a short delivery almost entirely and a long one hardly at all, which is the delivery factor $G(T_{\mathrm{true}})$ of Equation S2 of the main text as an experiment: lengthening what is being measured lowers the rate, where a load-driven account predicts no change. The figure sits here rather than in the main text for the same reason Equation S1 above does — it draws four points.

What we withdraw about it, and why: Two claims previously made around Equation S1 do not survive, and both were withdrawn in the round-2 internal revision.

The first is the infinite-moment reading — "the exponent is below one, so the stall distribution has neither a finite mean nor a finite variance". A slope fitted to four application-level points over one decade of T_{true} does not license a statement about the moments of the underlying stall distribution, which is a different object measured over a different range. What we should have written, and now do, is the weaker and sufficient claim: over the decade that matters the survival declines so slowly that a mean-based counter is a poor measure. That is enough to explain the observation it was invoked for — cumulative scheduler counters moving by factors of two to five against a fortyfold change in the negative-span rate — without asserting that a moment fails to exist.

The second is the traced cross-check. We reported that the per-wakeup survival's windowed log–log index, 0.332 over 0.25–2 ms, was "indistinguishable" from the fitted 0.339 and therefore confirmed it independently. It is not, and it does not. Estimated properly on the same histogram of $551{,}956$ wakeups (`scripts/tail_index_traced.py`), the exceedance index over 0.5–2 ms is 0.21 (0.20–0.21, Wilson on the nested counts) while grouped maximum likelihood over the same decade returns 1.19 (1.18–1.20, profile likelihood). Two estimators of one quantity cannot differ sixfold unless the model is wrong, and it is: the per-octave indices over that window run 1.96 and then 0.03, where a power law would give the same value twice. A bootstrap goodness-of-fit drawn from the fitted model itself agrees: $p < 0.0004$ over $2{,}500$ replicates. The 0.332 was ordinary least squares through four nested survival points, which returns a number whether or not the points lie on a line. The agreement with 0.339 was a coincidence of window and estimator, and we should not have offered it as a confirmation.

The prediction check against the trace is unaffected and stands: the main text predicts $\Pr[\text{stall} > 0.5\,\text{ms}] = 0.301$ against the traced 0.181, over-predicting by $1.66\times$ ($1.52\times$ on the replication), the expected direction since not every stall lands on a timestamping instant. That comparison is between two probabilities, not between two fitted exponents, and it needs no power law to be meaningful.

What the histogram says instead, which is better: Asking what shape the data do have, rather than which index to quote, gives the mechanism its most direct evidence. The bucket counts are not monotone, so no power law can describe them: there are 3 local maxima. The first is the jitter core of a running thread; the second a broad hump in the hundreds of microseconds; the third is a *mode* at 2–4 ms carrying 10.5% of all wakeups and standing $4.5\times$ above its lower neighbor. Beyond it the counts collapse rather than trail: they fall $3.4\times$ and $4.7\times$ over the next two octaves and then $357\times$, where a power law falls by one factor every time.

The second index, withdrawn in round 56, and why it took eight rounds: We had ended that paragraph with a number: grouped maximum likelihood above 4 ms gives 2.04 (2.00–2.07), quoted with the words "a finite variance". Three things were wrong with it and only the third is interesting. The moment claim needs an index above two and the interval's lower limit *is* two, so the data do not separate a finite variance from an infinite one. The last populated bucket falls $357\times$ where the fit implies about four, which is a truncation and not a tail. And the bootstrap goodness-of-fit rejects the fit outright: $p < 0.0004$ over $2{,}500$ replicates, the same verdict, from the same estimator, that we use six lines earlier to withdraw the payload-sweep index.

The interesting part is why it survived. `estimate()` in `tail_index_traced.py` ran `gof_pvalue` on the 256–$2048\,\mu$s window and on nothing else, so the test that condemned one index was never pointed at the other. Every gate we own passed the sentence, because none of them asks whether a published fit was judged by the test the same file already contains. That is this paper's own thesis arriving uninvited, and the repair is the one the paper recommends to others: run the check on everything it can judge, and publish what it says. The estimator now returns the tail's verdict whatever it is, the emitter carries it into the ledger beside the index it judges, and `test_the_paper_meets_its_own_rules.py` refuses a moment claim whose interval crosses the boundary it depends on.

The mode's position is not arbitrary. On this machine — one host, 8 online CPUs, recorded as "8 cores" in the campaign log itself and recovered independently from the geometry conditions' own k/ρ rather than from a shape description – the EEVDF base slice is $0.75\,\mathrm{ms} \times (1 + \lfloor \log_2 8 \rfloor) = 3$ ms; run-to-parity holds a woken thread off the CPU until the incumbent has exhausted that slice unless it wins the wakeup-preemption check, and with the high-resolution tick disabled the expiry is observed at the ordinary tick. A timestamping thread descheduled at the wrong moment therefore reads its clock roughly a slice late. That is the second state of the main text's two-state model, observed directly, at the constant the scheduler is configured with — which is a stronger statement than any tail index would have been, and it is what replaced the withdrawn one. The constants should be read off the image under test (`base_slice_ns` and `CONFIG_HZ`) rather than assumed from the kernel version, and we report them as the explanation the mode's location supports rather than as an independently measured cause.

The confirmation is the one that manipulates the physics: Payload moves T_{true}, so it moves θ's grid position, and the main text dictates the configuration's class with nothing else changed: measured $\theta(32\,\mathrm{KB}) = 0.685$ and $\theta(64\,\mathrm{KB}) = 0.853$ give, at $q = 3$, $\mathrm{frac}(q\theta) = 0.055$ (on the $2/3$ point: flat, pinned there) and 0.56 (mid-cell: full width back). The registered prediction was that one manipulation would move the configuration onto a grid point and off it. It did (figure in supplementary material S1): at 32 KB the five replicates spanned 62.06–75.69% — spread 13.6 points, three replicates agreeing to 0.06 — and at 64 KB they spanned 73.06–99.78% — spread 26.7, the upper vertex hit to a quarter point. The flat-full boundary at half the cell width (16.7 points) is crossed in the predicted direction by both halves. One registered detail failed: the 32 KB arm pins at 69.2%, not on the vertex at 66.67% — a displacement toward θ that the next paragraph makes systematic.

The other side of the inequality, and it moves the rate the wrong way: A *slower* path should be a *more reliable* measurement, and it is: payload padding lengthened transport $76.9\times$ while the negative-span rate *fell* $4.1\times$, monotonically, utilization held to a 0.012 spread and pushed slightly *up* against the observed fall (supplementary material S26). A $77\times$ larger threshold buying only a $4.1\times$ lower rate means the tail barely thins across two orders of magnitude — why mean-based counters could not see the effect and the direct trace was necessary.

Closing the loop: measuring the stall distribution directly: Every result so far infers the scheduler's behavior from the negative-span rate it produces. The mechanism says something stronger and directly checkable — that the negative-span rate should equal the probability that a run-queue stall outlasts the interval being measured — so we measured that probability without reference to the negative spans at all. Using `bpftrace` on the `sched_wakeup` and `sched_switch` tracepoints, we recorded the delay between a timestamping thread becoming runnable and reaching a core, for every such transition in the run, and computed $\Pr[\mathrm{stall} > 0.5\,\mathrm{ms}]$ against the measured transport of that cell (supplementary material S26).

The two quantities share no probe, no estimator and no data: one is a count of negative transports in application CSVs, the other a kernel histogram of scheduler transitions. Across three configurations at two load levels they agree to within a third — ratios 0.78, 1.06, 1.32 (supplementary material S26), nothing fitted between them. The residual has no consistent sign (22% low, 6% high, 32% high), so what one configuration had suggested as a directional effect was scatter; a traced scheduler quantity predicts an application-level failure rate to within a third, unfitted, and no bias is resolvable inside that.

The two-state reading accounts for the clustering (a preemption is an interval, so the events inside it are corrupted together) and predicts tails that shift vertically rather than rescale horizontally — measured close to parallel: median log-ratio spread 0.29, worst 0.91, set against the $23\times$ failure of the scale-family collapse. We found this by looking at the data — the separability test is exploratory — so we pre-registered a confirmatory version and ran it: the campaigns below. And read as a function of load alone the model is a tautology, with no content until it is pinned on the threshold axis, where it passed: the form that restores the divided-out σ reaches $R^2 = 0.9905$ against 0.9811 (exponential) and 0.8863 (power law) at equal parameters, but freezing σ at its mean improves the fit to $R^2 = 0.9982$, so σ and p move together on this ladder and neither can be credited over the other; the commentary in full is supplementary material S9.

The experiment that settles it: Raise the timestamping processes to real-time priority: occupancy falls sharply while utilization does not move, and priority acts on the timestamping threads themselves — where the model locates the failure — rather than canceling in the subtraction as our delay injection did. The result is unambiguous (supplementary material S26):

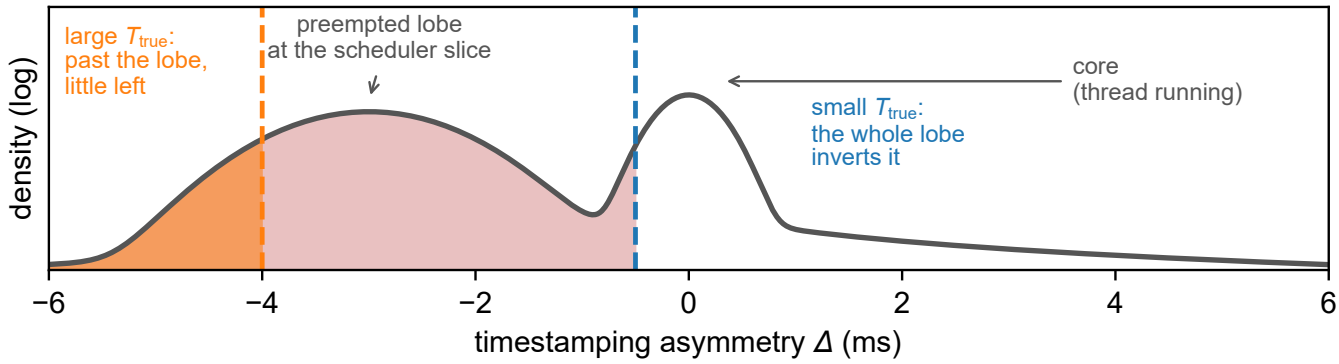

Fig. S4. **Two-state model, drawn.** The timestamping asymmetry Δ has a narrow core when the timestamping thread is running and a lobe at -3 ms, the scheduler's base slice, when it is not, so a small T_{true} is inverted by the whole lobe and a large one only by what lies beyond it — which is why the check bites hardest on small effects. The shape is schematic, two components placed at the measured mode's location rather than a fit; the measured distribution is the traced stall spectrum of the main text.

utilization held to 0.001, and the negative-span rate fell by factors of 39 and 54 at the two load levels, with disjoint Wilson intervals, both residual rates landing on the floor the model names in advance, $C_0 \approx 0.004$.

Utilization is not the variable, shown at identical utilization: Two loads can reach the same ρ with $C-k$ cores genuinely free or with none; at $k=6$ both configurations sit at $\rho = 0.7531$, identical to four decimals, and the negative-span rates differ by $2.07\times$ ($z=10.3$; supplementary material S26). A function of ρ returns one value for one input. A full replication reproduces the load-bearing cell ($2.05\times$), reproduces $k=5$ in direction but not size, and resolves a small $k=7$ difference ($1.19\times$, $z=3.46$) the original could not — so we withdraw an earlier draft's reading of that convergence as a null; both campaigns agree on the shape, twofold at $k=6$ collapsing toward parity at $k=7$, one free core in eight being nearly as useless as none.

S8.1. Tail recovery without a reference clock

Recovering the tail without a reference clock, and where it reproduces: Because negative-span rate at a threshold *is* a quantile of Δ's tail, the tail can be read off negative span counts alone — an measurement-quality estimate needing no reference clock. The honest test of it is not internal consistency but reproduction: do the recovered tail quantiles agree between two independent campaigns at matched utilization? Below saturation they do — 12 of 17 matched quantiles overlap at 90%, and every one of the deep pre-knee quantiles does. The failures are all at $\rho = 1$, where the coordinate is degenerate — eight, ten and twelve background workers all read as full utilization while representing different degrees of overload — so the two campaigns are not in fact at matched load there. The recovered curve reproduces wherever utilization is a meaningful coordinate and stops when it stops being one, which is the correct boundary for the method rather than a defect in it.

S9. The two-state model commentary and the tracer checks

The form, and why it is stated here: Conditioning the identity $S<0 \iff A>D$ on the delivery, and writing p for the timestamping thread's off-CPU occupancy and G for the survival function of the residual delay in that state, gives to leading order

$$\Pr[S<0 \mid T_{\mathrm{true}}] \;=\; p\,G(T_{\mathrm{true}}). \tag{S2}$$

Round 59 moved this out of the main text, and the reason is worth stating rather than hiding. *Nothing in the paper computes with it.* Every reference the main text makes to it names the two-state picture — the preempted state, the state a pinned thread removes — and the three consequences we test are attributable without it: the direction under payload is what the identity predicts, the floor C_0 is the $p \to 0$ limit of the picture, and the traced check compares two probabilities. The form has also never been tested as a form: we have no measurement of p independent of the rate it is meant to explain, so what has been confirmed is the mechanism and its consequences, not the factorization. It was wrong once for exactly this kind of reason — written $p(\rho)\,G$ until Table II of the main text showed the rate is not a function of ρ — and was repaired by weakening it. A model that cannot be falsified as a model belongs where a reader can weigh it, not where the argument appears to rest on it.

Figure S4 draws the model this section defends.

What it accounts for: Clustering follows by construction: being preempted is an *interval*, so the events inside it are corrupted together and a two-state process cannot help but cluster. The $60\times$ growth in tail weight against $5\times$ in core width is two different parameters moving — σ_c belongs to the running state, p to the scheduler. The scale family fails because a scale family has one parameter and here two move at very different rates.

A prediction that points the opposite way: The two structures disagree geometrically. A scale family changes σ, so its tail curves *rescale horizontally*; the main text changes p, so its curves *shift vertically and stay parallel* on a log scale. Taking the log ratio of two conditions' tail masses at several thresholds, the two-state model predicts a constant, independent of the threshold.

Restricting to tail estimates backed by at least 20 events — nine far-tail points resting on five to eight events are excluded, as they carry log-scale errors larger than the effect — the median spread of that log ratio across conditions is $\mathbf{0.29}$, a factor of 1.34, with a worst case of 0.91. Set against the $23\times$ failure of the scale-family collapse, the tail curves are close to parallel. The two conditions with the largest spreads sit at intermediate load, where the residual distribution is plausibly still shifting, so the honest statement is $\Pr[\text{inv}] = p(\rho)\, S(c;\rho)$ with p varying fast and S slowly: separability is a good first-order description rather than an exact law.

The two failures are not one failure seen twice: The introduction of the main text says the two act differently, and that sentence is doing more work than it looks: it is what stops a reader rebuilding the shared-number framing this project withdrew. Opposite movement would not establish it. Both failures answer to T_{true}, and a longer delivery makes one of them rarer and the other harmless, so a reader is entitled to ask whether they are one dependence seen from two sides. What settles it is not how fast either responds — any comparison of one response *factor* against the other *across the payload sweep* is the exponent of Equation S1 recomputed from fewer points, and a weaker copy of a result cannot corroborate it — but that only one of them can be escaped.

Their limits differ in kind. Retention saturates: $\min(1, T_{\text{true}}/\tau)$ reaches one and stops, and the failure switches off — of the 75 instrumented cells, the four whose payload is large enough report above the grid and lose nothing. The negative-span rate has no such exit. It is bounded above by 0.37 and below by a floor of 0.004 that real-time priority reaches but does not cross, which is why Section VIII-B keeps the sign check after every remedy it recommends. One failure can be lengthened out of existence and the other cannot, and no rescaling of a single dependence produces that.

What this does *not* show is that the failures are unrelated. They share an axis, and a benchmark on a sub-millisecond path meets both. They are not the same function of it, and the difference that establishes it is a difference of limits rather than of slopes.

What this is worth, and what it is not: It is mechanistic — occupancy names the moving part — and actionable: if the failure is occupancy, the mitigation is a timestamping thread the workload cannot preempt, not a better clock. But we found this by looking at the data; the separability test is exploratory (the main text), so we pre-registered a confirmatory version and ran it: the campaigns below, the decisive one being real-time priority at fixed utilization. Two states is the smallest model consistent with the data, not a kernel claim.

On the load axis the model is a tautology, and we say so before using it: Read as a function of ρ alone the model has no content: with p unconstrained, any monotone rate curve can be written $p(\rho)\, S$; the model's content is on the threshold axis, where it passed. Constraining p to a power law fits poorly ($R^2 = 0.65$ in log space). The form that restores the divided-out σ, $\Pr[\text{inv}] = p(\rho)\, S(T_{\text{true}}/\sigma(\rho))$, reaches $R^2 = 0.9905$ against 0.9811 (exponential) and 0.8863 ($(\rho/(1-\rho))^k$) at equal parameters — *and we do not report that lead as a result*: freezing σ at its mean improves the fit to $R^2 = 0.9982$, because σ and ρ move together on this ladder and no fit can credit one over the other. That is a design limit no further analysis of these data can lift.

The tracer check, and what it can and cannot rule out. Tracing a system to measure its scheduling behavior is exactly the kind of intervention that can create what it observes, so the traced cell has an untraced twin. Our first attempt at this campaign ran with a probe that failed to attach: the histogram was empty, but the negative-span rates are valid, and we kept that run as the control. It measured 0.2717 against the traced run's 0.2315 — the traced rate is 14.8% *lower*, two hours apart, same design.

We report that rather than a flattering comparison, because an earlier version of this section compared the traced cell against a different campaign's 88% configuration (0.2214) and called the 4.6% gap a clean bill of health. The honest reading is weaker. The ordinary 88% configuration has been measured five times across these campaigns and spans 0.221 to 0.305, a factor of 1.38; the traced value sits inside that spread, and so does its untraced twin. So we can say that tracing did not move the rate by more than the drift already present between campaigns, and we cannot say anything tighter than that. A tracing effect smaller than about 15% is not resolvable with this control.

What that limits is the *level*, which no claim here rests on. The comparison in the main text is between two quantities measured in the *same* run — a kernel histogram and a count of negative transports over the same events — so a common perturbation of that run moves both and leaves their agreement intact.

The gate fired on our own replication, and we report the configuration it withheld. The check is a rule fixed in advance: a traced configuration whose negative-span rate differs from its untraced comparator by more than 25% is not compared. The replication's 88% configuration measures 0.1956 against the same untraced control's 0.2717 — a drift of 28% — so its verdict is withheld by the same mechanism that admitted the original. Its numbers are in the main text because withholding a comparison is not a reason to hide a measurement, and because the ratio it would have contributed, 1.06, is the closest agreement in the table. Reporting only the configurations that passed would leave a reader with three ratios and no idea that one of them is inadmissible under our own rule.

The real-time configuration's zero is an artifact of the tracer, and the replication settles it. Every traced real-time configuration we have run — three of them, across two campaigns and two load levels — recorded *zero* negative spans in 2,985 events. The untraced twin of one of those configurations recorded 15. Under that rate the chance of a single traced configuration showing none is 3×10^{-7}, and of all three about 10^{-20}.

We first reported this as unresolved: either the configuration genuinely differed or tracing perturbed it, and one measurement could not separate the two. Three can. Attaching BPF to `sched_switch` suppresses the negative spans in the real-time configuration, and the effect is not subtle — it removes all of them. The direction is consistent with the ordinary configuration, where the two traced measurements are the lower two of the six we have at 88%, though there the drift between campaigns is wide enough that the same conclusion cannot be drawn.

This is the paper's own thesis arriving uninvited. We attached a tracer to measure why a measurement was failing, and the tracer changed the thing it came to observe — in the configuration where the quantity was smallest, which is exactly where the main text says a measurement is most exposed. It costs us nothing here, because no claim rests on the real-time configuration's level, and it is worth stating plainly rather than leaving in a footnote: a kernel tracer is not a free observer.

S10. Retention sits on the grid, and the spread rule says when

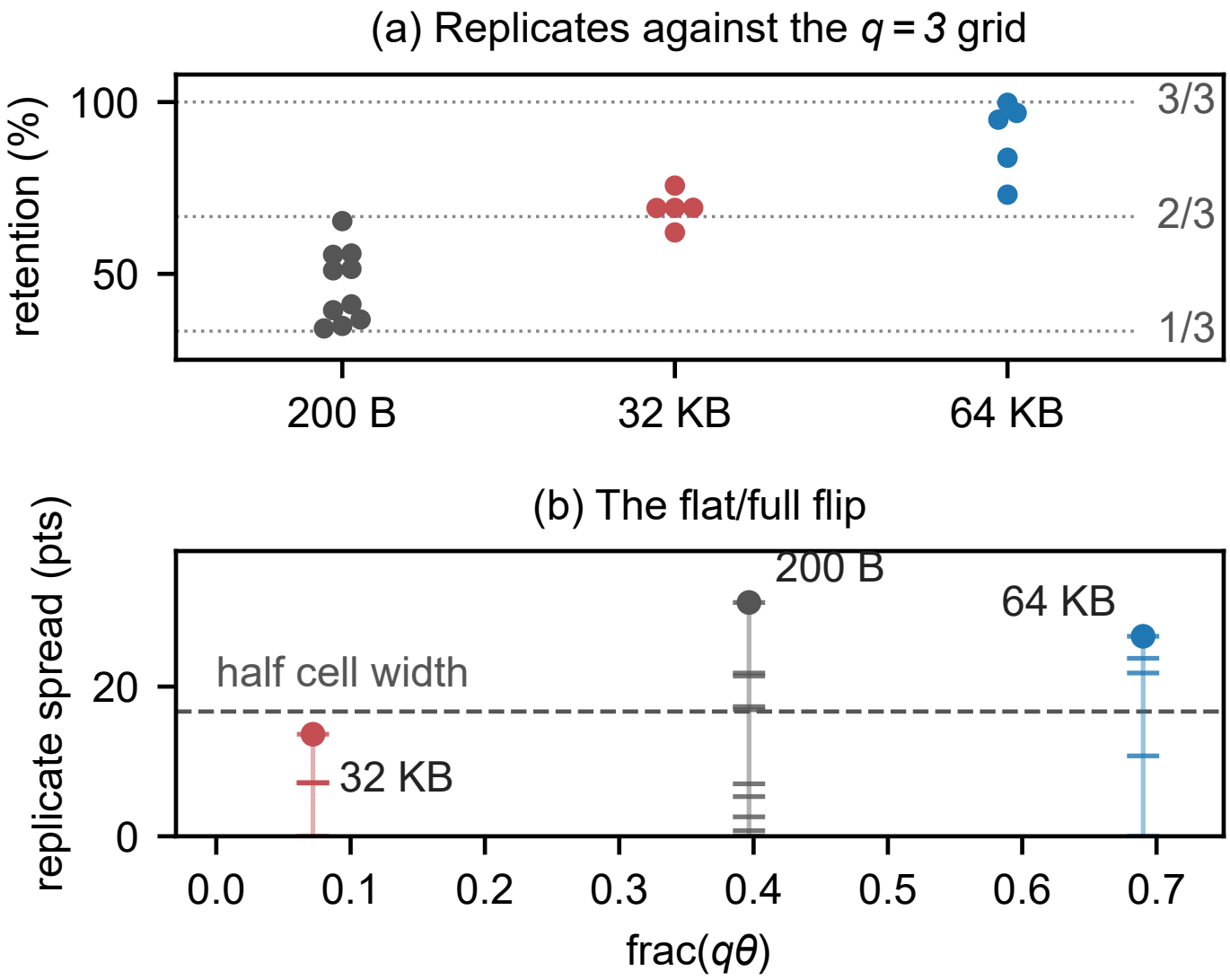

Fig. S5. **Campaign's confirmed prediction.** (a) Retention replicates at 300 msg/s ($q = 3$) by payload, against the thirds grid: 200 B straddles both branches, 32 KB pins near the 2/3 vertex, 64 KB re-opens to the upper cell. Horizontal position within an configuration separates replicates that would overprint and carries no variable. (b) Replicate spread against $\mathrm{frac}(q\theta)$: the flat–full boundary at half the cell width is crossed in both directions by the one manipulated variable. The spread is a range, so each stick shows the replicates it is made of — 32 KB clears the boundary with three of five pinned together, 64 KB is broad.

Figure S5 is the confirmation the main text's Section VI-B reports: the three payload configurations against their $q = 3$ grid, and the spread each one produces against the boundary the rule below puts at half a cell.

What the replicate spread does, and does not, predict: It is tempting to write the prediction as "spread $\approx 100/q$", and we did; that is wrong in a way that matters, because $100/q$ is the *cell width* — an upper bound, reached only when T_{true}/τ sits midway between two grid points and replicates realize both. On a grid point, one point takes nearly every run and the spread collapses. One statement covers both:

$$\text{spread} \longrightarrow \begin{cases} 100/q & \text{when } T_{\mathrm{true}}/\tau \text{ sits mid-cell,} \\ 0 & \text{when } T_{\mathrm{true}}/\tau \text{ sits on a grid point.} \end{cases} \tag{S3}$$

The measured continuous value is $T_{\mathrm{true}}/\tau = 0.495$, a half to within replicate noise. A half lies on every even grid ($1/2$, $2/4$, $4/8$) and never on an odd one, so at this operating point the odd denominators are predicted to show their full cell width and the even ones to show almost none. That is a prediction about specific configurations before they are run, not a description after.

Most configurations behave as their position predicts, and the two that do not are the reason this statistic was replaced: Table S15 states each rate's cell width, cell position, prediction and measured spread against Equation S3; the test needs no tolerance, since a spread above half the cell width separates full from flat. Of the 12 commensurate configurations, 10 match. The predictions are ten full against two flat, so both classes are realized and the agreement is with a prediction that varies, not with a constant. The table's last two columns are that comparison, one row at a time.

The two that miss are 700 and 1250 msg/s. Both are predicted full and measure flat, and we report them as failures of *this* statistic rather than of the model, for a reason that was stated before either was run and does not depend on how they came out. Spread is a range over replicates of a quantity with two-point support: an configuration realizes the full cell width only if its replicates land on *both* bracketing vertices, and five replicates that happen to fall on one of them produce a flat configuration however exactly right the grid is. At $q = 5$ and $q = 7$ the upper branch carries a minority of the probability, so this is the outcome to expect some of the time, and the estimator cannot tell it from a refutation.

That is precisely why the pre-specified replacement below does not use spread at all, and the replacement answers both configurations: 700 and 1250 msg/s each reject the continuum after Holm correction, at 0.009 and 0.003, and the table of that test records both as *grid*. An estimator that cannot separate "on the grid, one branch sampled" from "not on the grid" was the whole reason for building one that can, and the two misses are the clearest case in the corpus for having done it.

The denominator, not the rate that carries it: Every denominator above appears at one rate, so the pre-registered control is the 600 msg/s configuration: 1.667 ms $= 5/3$, the same $q = 3$ as 300 msg/s at *half* the interval, recorded in advance to land near 33%. It landed at 33.98, 34.29, 34.90, 36.17, 65.86 and 66.67 — four replicates on the $1/3$ branch, two on the $2/3$ branch, nothing between (one replicate's shutdown-hook totals arrived after our first read; the recovered 65.86 was on the upper branch). The loosest of them, 36.17, sits 2.83 points from its vertex, inside the 3-point rule stated below and the widest departure the configuration shows. 66.67% is $2/3$ to two decimals, matching the 65.35 that 300 msg/s produced on the same branch: two rates an octave apart land on the same grid because they share a denominator.

A quantity with two-point support breaks the replicate median, so the pre-specified replacement (`grid_membership_test.py`, committed with the artifact) tests each configuration's mean distance to its nearest grid vertex, normalized by cell width, against a continuum null of Normal($\theta_{\mathrm{local}}, \sigma$=1.2) replicates — both parameters measured, neither fitted to the configuration under test; the p-value is one-sided by Monte Carlo.

Of twelve commensurate configurations, ten are powered — the null and the grid predict separably — and 9 **of the** 10 **reject the continuum after Holm correction**, eight at the Monte-Carlo floor of 2.5×10^{-4}, including configurations whose binary flat/full classification had read as smeared. The tenth (900 msg/s) rejects at 0.044 raw and at 0.131 corrected, so it does not reject: an earlier version of this document and of the main-text table counted it as a rejection while the caption claimed the correction had been applied. That is now fixed in the only durable way, by generating the table from the artifact. The two unpowered configurations are the degenerate ones (400 and 800 msg/s, whose grids pass through θ), and the test reports them as undecidable rather than counting them for either side. Where every replicate lies within 3 points of a vertex the configuration is branch-classifiable and the upper-branch weight carries an exact Clopper–Pearson interval in the artifact. In plain terms: judged by a test that cannot be fooled by a value with no center, the grid is present in every configuration that could show it.

A quantity with two-point support breaks the replicate median: So a pre-specified grid-membership statistic (committed with the artifact) tests each configuration's distance to its nearest vertex against a measured continuum null: of twelve commensurate configurations ten are powered, 9 **of the** 10 **reject the continuum after Holm correction** — eight at the Monte-Carlo floor of 2.5×10^{-4}, the tenth unresolved — and the two degenerate configurations are reported as undecidable. Judged by a test that cannot be fooled by a value with no center, the grid is present in every configuration that could show it (method: supplementary material S10).

S11. Grid membership, configuration by configuration

Figure S6 makes the claim about the set; Table S7 is the same twelve configurations one row at a time, for a reader checking an individual rate. Both moved here from the main text in round 59, where the grid refinement became the explanation of the deletion rather than a contribution in its own right.

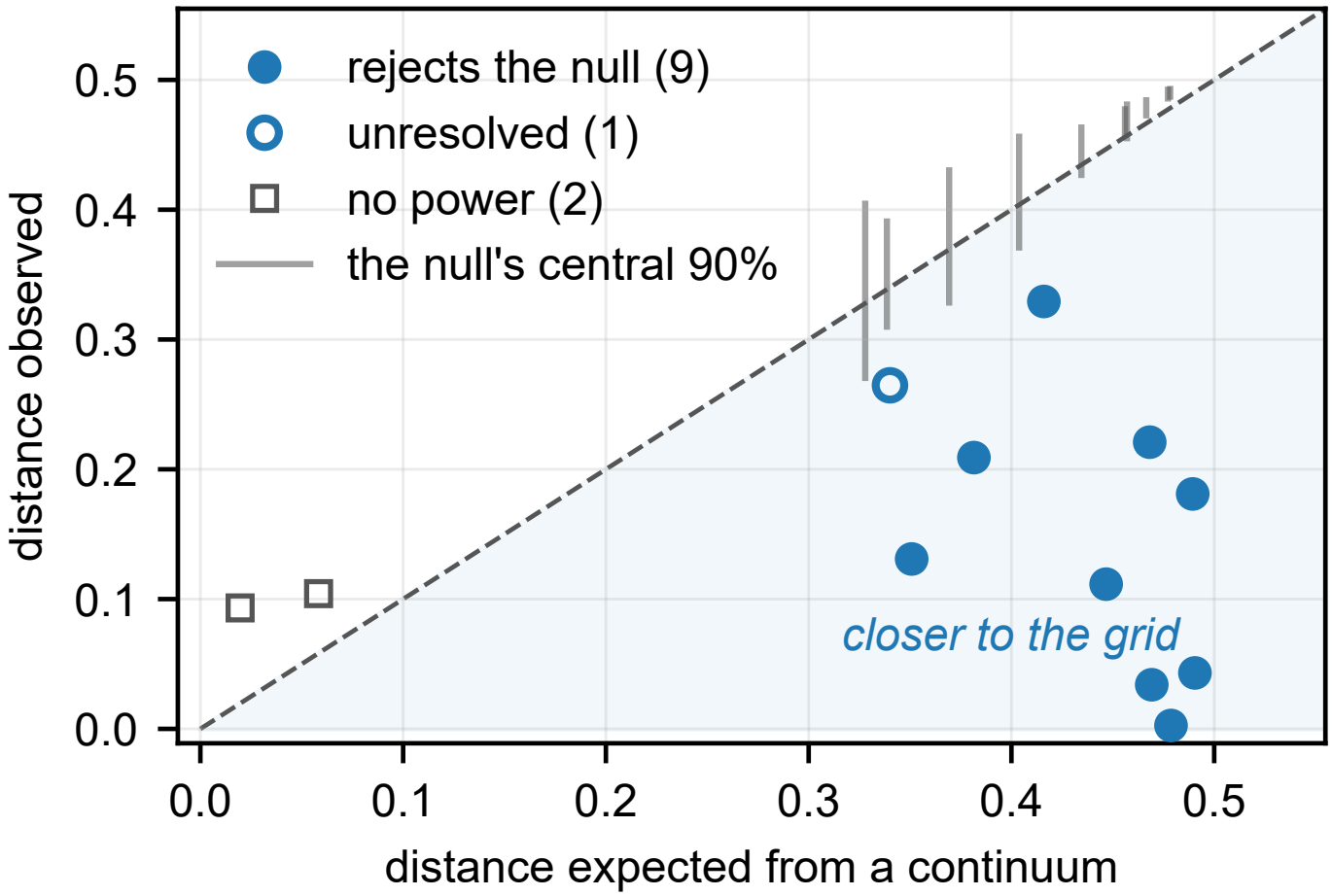

Fig. S6. **Grid membership, as a set.** Each configuration's observed mean distance to the nearest grid vertex against the distance a continuum would give, which is the diagonal. The gray bar beside a configuration is its own simulated null, *uncorrected*; the test is one-sided towards the grid, so a marker *below* its bar rejects that null. Filled circles also survive Holm correction. The open circle sits *below* its bar too, 0.2647 against a band opening at 0.2680, but at $p = 0.044$ raw and 0.131 corrected it is the one powered configuration that does not reject: correction is what the picture cannot draw. The two squares carry no bar because no test has power there: the two hypotheses coincide, and *the sign carries nothing*. Corrected p-values are in Table S7.

Of the ten powered configurations, nine reject the continuum null at $\alpha = 0.05$ after Holm correction within the family of 12. The tenth, 900 msg/s, rejects before correction ($p = 0.044$) and not after ($p = 0.131$); we report it as unresolved rather than as support. The remaining two configurations, 400 and 800 msg/s, have no power by construction: where T_{true}/τ sits on a vertex the two predictions coincide, so the distance is replicate noise and which side of the diagonal it falls on is not evidence either way. Table S7 is generated from `grid_membership.csv` at build time by the same helper that produces the figure, so the two cannot disagree.

S12. Every real-time-priority pair, the ladder they make, and the exposure curve

The main text quotes a range: real-time priority on the timestamping threads collapses the negative-span rate 7–80× at fixed utilization. That range is taken over 8 matched pairs from three campaigns, and until this revision only the two of Table S21 appeared anywhere — which left the two pairs that anchor the range, including the weakest result in the series, visible nowhere. Both are below.

Table S8 is generated from `docs/results/model/stamping_priority*.csv` at build time, so the range in the abstract and the rows here cannot disagree. E-A5 is the original manipulation, E-A5b widened the load ladder to 60 and 95%, and E-A7 replicated E-A5. Every pair has disjoint Wilson intervals, and 0 were withheld for a failed manipulation check — the design assumes real-time priority leaves ρ alone, so ρ is measured in both configurations and a discrepancy above 0.05 withholds the pair rather than reporting it hedged. Naming the threshold matters more than the count it produced: a withholding rule whose criterion is invisible cannot be checked, and this one never fired because it never came close. The widest disagreement any pair showed was 0.002, some 25 times inside the tolerance, so the zero above is a clean manipulation rather than a loose test.

Figure S7 shows what the range does not: which configuration moves. The real-time configuration sits between 0.0017 and 0.0144 across the whole load ladder, close to the floor the model names in advance; the ordinary configuration rises from

TABLE S7

GRID MEMBERSHIP AGAINST A CONTINUUM NULL. D IS THE MEAN NORMALIZED DISTANCE TO THE NEAREST GRID VERTEX; UNDER THE NULL ITS EXPECTATION IS THE VALUE IN THE *null* COLUMN. THAT COLUMN WAS, UNTIL THIS REVISION, THE DISTANCE OF θ FROM ITS NEAREST VERTEX WITH NO REPLICATE NOISE — THE DISTANCE OF THE EXPECTATION RATHER THAN THE EXPECTATION OF THE DISTANCE, WHICH ARE NOT THE SAME NUMBER AND DIFFERED BY UP TO 0.04 HERE. IT IS NOW THE MEAN OF THE SIMULATED NULL ITSELF, SO THE SENTENCE ABOVE IT IS TRUE. p_{Holm} IS A MONTE CARLO p-VALUE (4,000 DRAWS) AFTER HOLM CORRECTION WITHIN THE FAMILY OF ALL 12 CONFIGURATIONS; THE CORRECTED VALUE IS WHAT DECIDES THE VERDICT. ARMS MARKED *flat (coincident)* ARE THOSE THE MODEL PREDICTS FLAT, WHERE THE TWO HYPOTHESES MAKE THE SAME PREDICTION AND THE TEST HAS NO POWER. THE 700 AND 1250 MSG/S ROWS ARE THE TWO THAT THE SPREAD RULE OF TABLE S15 MISSES; THIS TEST RESOLVES BOTH, WHICH IS WHY IT REPLACED THAT RULE.

rate	p/q	n	D	null	p_{Holm}	verdict
1250/s	4/5	5	0.131	0.351	0.003	grid
1000/s	1/1	5	0.003	0.479	0.003	grid
900/s	10/9	5	0.265	0.340	0.131	not resolved
875/s	8/7	5	0.209	0.382	0.003	grid
800/s	5/4	5	0.104	0.058	1.000	flat (coincident)
700/s	10/7	5	0.329	0.416	0.009	grid
625/s	8/5	10	0.112	0.447	0.003	grid
600/s	5/3	6	0.034	0.469	0.003	grid
500/s	2/1	5	0.043	0.490	0.003	grid
400/s	5/2	5	0.093	0.020	1.000	flat (coincident)
300/s	10/3	10	0.221	0.468	0.003	grid
250/s	4/1	5	0.181	0.489	0.003	grid

TABLE S8

EVERY MATCHED PAIR. ORDINARY AND REAL-TIME NEGATIVE-SPAN RATES AT THE SAME ACHIEVED UTILIZATION, AND THE FACTOR BETWEEN THEM. BOTH CONFIGURATIONS' ACHIEVED ρ IS PRINTED, SO THE CLAIM THAT THE MANIPULATION MOVED OCCUPANCY AND NOT LOAD CAN BE CHECKED PAIR BY PAIR RATHER THAN TAKEN FROM A CAPTION: THE WIDEST DISAGREEMENT ANYWHERE IN THE TABLE IS 0.002, AGAINST THE 0.05 THAT WOULD HAVE WITHHELD THE PAIR. GENERATED AT BUILD TIME FROM THE THREE CAMPAIGN FILES.

campaign	load	ρ ordinary	ρ real-time	ordinary	real-time	factor
E-A5b	60%	0.6055	0.6035	0.0459	0.0017	27×
E-A5b	70%	0.7032	0.7025	0.1032	0.0144	7×
E-A5	75%	0.7531	0.7525	0.1320	0.0034	39×
E-A7	75%	0.7550	0.7537	0.1504	0.0044	35×
E-A5b	88%	0.8806	0.8809	0.2214	0.0034	66×
E-A5	88%	0.8809	0.8812	0.3049	0.0057	54×
E-A7	88%	0.8825	0.8825	0.2526	0.0054	47×
E-A5b	95%	0.9501	0.9501	0.2938	0.0037	80×

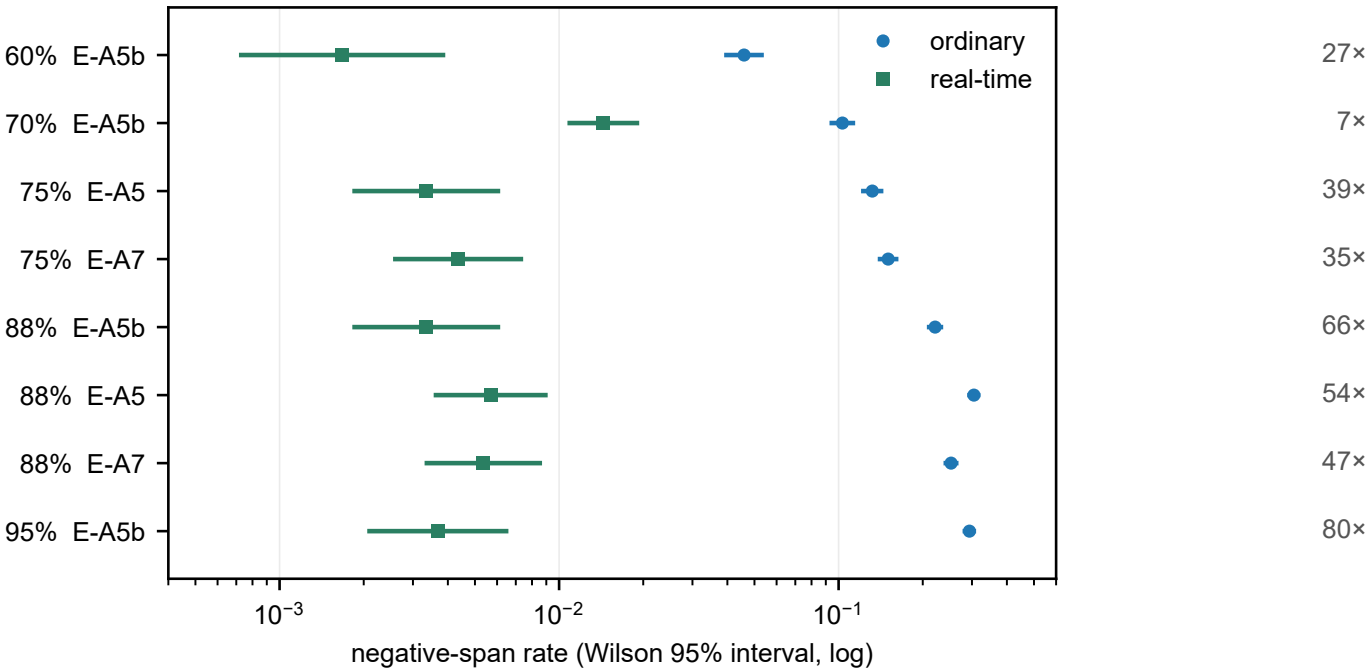

Fig. S7. **All 8 pairs, as a ladder.** Ordinary and real-time negative-span rates with Wilson 95% intervals, one row per pair, ordered by achieved utilization; the factor is printed at the right. Reading down, the real-time configuration barely moves while the ordinary configuration climbs with load, which is what makes the factor grow. The growth is a trend and not a law — 60% gives 27× and 70% gives 7× — and the range is reported as a range for that reason.

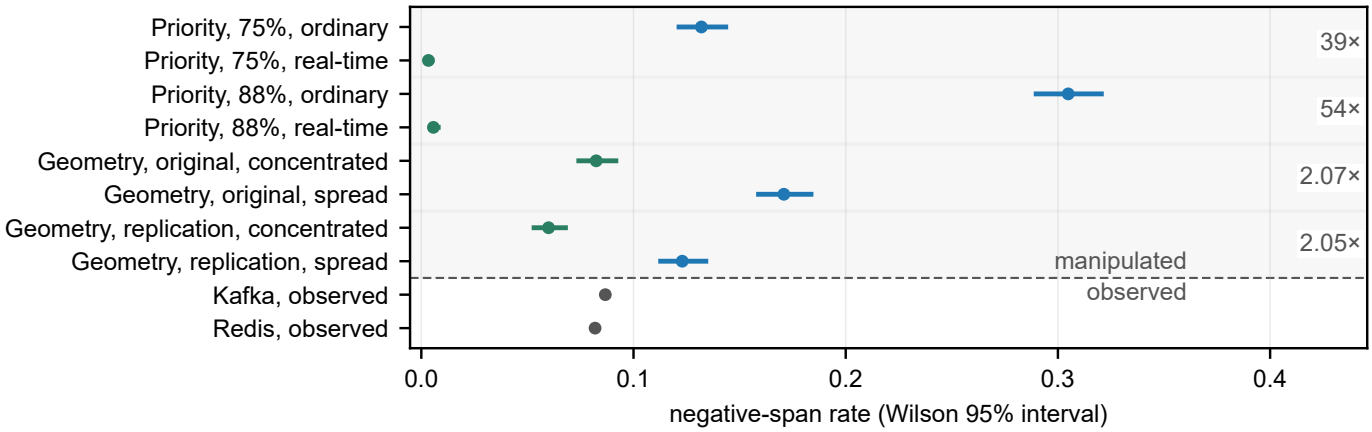

Fig. S8. **Every manipulation separates; the broker does not.** Wilson 95% intervals. Above the rule, the four matched pairs of Table II, utilization held fixed within each: what moves the rate is where the timestamping thread sits, not how loaded the machine is. Below it, observed rather than manipulated, the two brokers of Table I; their intervals are narrower than the plotted point.

TABLE S9

WHAT THE DISPLACEMENT COSTS AT LONGER PATHS, AND HOW WIDE THE COST IS. COMPUTED FROM THE MEASURED ACK LAGS, NOT MODELED. LEFT: THE ERROR ON A SINGLE REPORTED LATENCY, A/T_{true}, AT THIS CORPUS'S MEDIAN LAG OF 725 μS. MIDDLE: THE SAME ERROR AT THE TENTH AND NINETIETH PERCENTILES OF THAT LAG ACROSS OUR CONDITIONS, 500 AND 1900 μS. THE BAND IS THE COLUMN TO READ. A LAG IS NOT A CONSTANT, AND A READER LOCATING THEIR OWN PATH AGAINST THE POINT ESTIMATE ALONE WOULD PUT A 10 MS PATH AT 7% WHEN A TENTH OF OUR OWN CONDITIONS SIT AT 19%. RIGHT: THE GAP A COMPARISON WOULD REPORT BETWEEN TWO IDENTICAL SYSTEMS WHOSE CLIENTS DIFFER ONLY IN WHERE THE ACKNOWLEDGMENT IS TIMESTAMPED — A CALLBACK THREAD AGAINST AN INLINE TIMESTAMP. WHERE THE RIGHT-HAND COLUMN READS "INVERTS", THE APPARENT GAP HAS CHANGED SIGN; ON EVERY ROW BELOW THOSE NOTHING IS NEGATIVE, SO THE SIGN CHECK OF SECTION IV-E FIRES ON NONE OF IT.

True path T_{true}	Error on one measurement	p10–p90 across conditions	Apparent gap between two identical systems
0.25 ms	290%	200–760%	inverts
0.5 ms	145%	100–380%	inverts
1 ms	72%	50–190%	inverts
2 ms	36%	25–95%	67%
5 ms	14%	10–38%	22%
10 ms	7%	5–19%	11%
25 ms	3%	2–8%	4%
50 ms	1%	1–4%	2%
100 ms	1%	0–2%	1%

0.046 to 0.30. The collapse is therefore not a proportional reduction that happens to be larger under load — it is the ordinary configuration following utilization while the manipulated configuration does not, which is the two-state model of Equation S2 with p moved directly.

Two exhibits moved here from the main text when the manuscript was brought back inside twelve pages, and both are restatements rather than evidence, which is why they were the ones to go. Figure S8 draws the four matched pairs of the main text's mechanism table and the two brokers of its span table: no pair of configurations overlaps, which is checkable in those tables and is faster to see here. Table S9 tabulates the exposure curve whose headline points the main text quotes — the relative error A/T_{true} against path length, and beside it the gap a comparison would report between two identical systems whose clients timestamp in different places — and Figure S9 draws the same lags as a curve, which is the form a reader locating their own path wants. Neither carries a number that is not already stated where it is argued.

The rows added below one millisecond are the regime broker benchmarks actually report in (Section S33), and they are where an acknowledgment-referenced harness stops measuring the path at all. A send-referenced one is unaffected: it reports the delivery itself, which is why the framework audited in S33.3 needs no correction from this table.

Which estimator produced the two headline ratios, stated because it decides them: The main text says the reported span is 24% of the delivery it stands for, understating it 4.2×. Both are the *median over conditions of* $\mathrm{median}\, S/\mathrm{median}\, D$ — a median of per-condition ratios, computed in `emit_paper_numbers.py` and gated there. The estimator is not a detail. Working from the medians this supplement prints, which is the natural recomputation, a reader who differences them condition by condition gets 3.5×, and one who divides the pooled medians gets 2.6×. Three defensible readings, a factor of 1.6 between the extremes, and only one of them is the number in the text. This project has been caught by the same distinction once before, on the independence overshoot of Section VIII-B, where median-of-ratios and ratio-of-medians differ by 12.4× against 21.3×; there it is stated in the sentence that quotes it, and now it is stated here too.

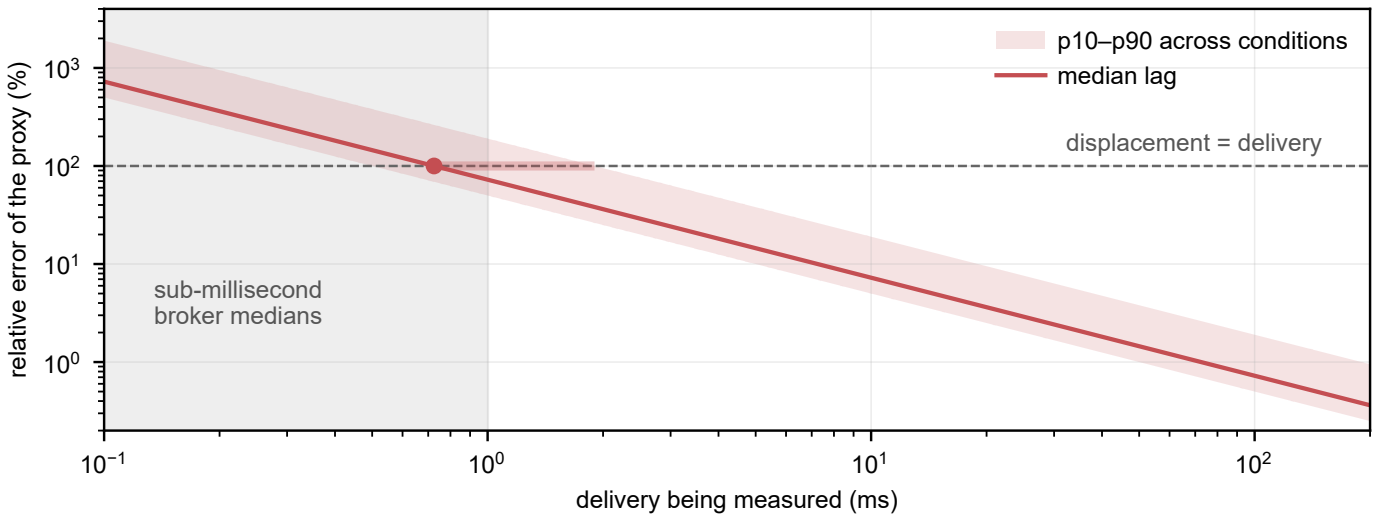

Fig. S9. **Where a path sits on the exposure curve.** Table S9 as a curve: the relative error of the acknowledgment-referenced proxy, A/T_{true}, against the delivery being measured. The line is this corpus's median acknowledgment lag and the band its tenth to ninetieth percentile across conditions, so a reader can bracket their own path rather than take the middle. The bar on the dashed line is the crossover, where the displacement equals the delivery it is a correction to: 0.72 ms at the median lag and 1.90 ms at the ninetieth percentile, marked as an interval because it is one. To its left the correction is larger than the quantity corrected. The shaded column is where published broker medians sit (Section S33.4), which is the reason the curve matters.

S13. What the brokers actually do

The main text states two conclusions and points here for the evidence: that the broker comparison is not a purchasing argument, and that one configuration choice, invisible on the standard testbed, decides the outcome. This is the campaign behind both. It sits in the supplement because the paper's contribution is two measurement failures, and what the brokers do is a smaller result than either — smaller, in the end, than the comparison that motivated the work was expected to be.

S13.1. Equivalence on gated artifacts

Over the runs the sign check leaves standing, the brokers sit within a millisecond of each other on the transport proxy: a Hodges–Lehmann shift of 0.408 ms (90% interval 0.389–0.419), with TOST rejecting both one-sided nulls against a 1 ms margin at $p < 0.001$, holding between 0.408–0.417 ms across 3 concurrency levels.

That span is not a causal chain, so the equivalence is stated on the quantity that is: TTI is equivalent over the same levels against a wider margin (Section S2). Equivalence is claimed rather than assumed from a failure to reject, which is the reason for the TOST rather than a two-sample test.

S13.2. The condition that reverses the ordering

One-way delay injected identically at both leaves Kafka tracking it almost additively, while Redis collapses: a median transport of 4.6 s at 5 ms of injected delay, 31.3 s at 20 ms and 102.5 s at 50 ms, over five feeds per configuration. That is 900 to $2{,}050$ times the delay applied.

The cause is the consumption pattern rather than the broker. A per-message acknowledgment turns each event into a sequential round trip, so injected delay multiplies once per event, where a prefetching consumer serves from buffer and pays it once per batch.

S13.3. The falsifier, registered in advance

The prediction was recorded before the run: if the acknowledgment pattern is the cause, then batching the acknowledgments must remove most of the effect without touching the broker, the network or the injected delay. Batching them two hundred at a time cuts the median from $4{,}138$ ms to 103 ms, a factor of 40, replicated four times.

S13.4. Why the standard testbed cannot see it

Both settings are invisible on a co-located testbed: the round trip they multiply is a few hundred microseconds there, so the difference between paying it once per event and once per batch is below the noise. The choice that decides the outcome under any realistic separation is therefore not visible in the environment most comparisons are run in — which is the same shape of failure as the two the paper is about, a tool whose configuration determines the result while nothing in the output records it.

S14. The classical statistics of a discarded zero, and what the guard deletes

The main text argues that deleting the samples which compute to zero conditions the reported distribution on being positive, and that the retained fraction is therefore part of the measurement. Two classical sources sit close enough to that argument to be named, and both need care in the naming, so we give them the space here rather than compress them into a clause.

The baseline the admission condition invalidates: The GUM gives the standard model for a quantized reading. Clause F.2.2.1, "The resolution of a digital indication", states that if the resolution is δx then the stimulus "can lie with equal probability anywhere in the interval $X - \delta x/2$ to $X + \delta x/2$" and is therefore described by "a rectangular probability distribution … of width δx with variance $u^2 = (\delta x)^2/12$" [S5]. That symmetric model is the departure point for this paper: it holds for *retained* readings, and the admission condition of Section VI of the main text removes one side of the interval, so neither the rectangular shape nor the $(\delta x)^2/12$ variance survives the filter. Quantization is not the problem; the asymmetry the admission condition introduces on top of it is. We cite the clause rather than the formula because the same $(\delta x)^2/12$ appears in F.2.2.2 for hysteresis, and the form alone does not identify the mechanism.

The oldest statement that discarding zeros is not free: Pratt [S6] showed that discarding zero differences before ranking violates a natural monotonicity requirement: a sample not judged significantly positive can become significantly positive when *every* observation is decreased. He calls this an anomaly and gives a procedure that avoids it — rank all observations including the zeros, drop only the zeros' ranks, and refer the statistic to its null distribution conditional on the number of zeros.

Two qualifications keep the analogy honest. First, Pratt *conditions on* the zero count; he does not treat it as evidence of a shift, and he explicitly declines to claim any optimality for the procedure. Our claim about the retention rate is the weaker and more practical one — that the count must be published so a reader can see what the distribution is conditioned on — and it does not need Pratt's stronger machinery. Second, the reading that makes the parallel exact is not Pratt's own framing but one he reports from Meier in a remarks section: that a zero arises because "with only our crude measurement, we do not know" the sign a finer measurement would have resolved. That is precisely the sub-resolution sample of Section VI of the main text, and it is a 1959 statement of it; we attribute it where it belongs rather than to Pratt's main development.

A second standard already owns the lemma: The arithmetic core of the grid — a rate whose ratio to the sampling tick is J/M in lowest terms visits exactly M distinct phases — is standardized in ADC metrology. IEEE Std 1241-2010's sine-wave histogram test requires the record to hold a number of input cycles "relatively prime" to the number of samples (§3.4.1, Eq. (28)); the same standard's sine-fit clause states the frequency choice as $f_i = (J/M)\, f_s$ and names the practice *coherent sampling* (§1.4.1, Eq. (18)) [S7]. Both clauses exist for the same reason we choose incommensurate send rates. (The standard's notation is J/M where the main text writes p/q.) As with the counter literature, we claim the consequence under deletion, not the lemma.

What the admission condition deletes here, measured: The three arguments above are about what deletion does in principle. Figure S10 is what it does to this corpus, in three panels. Panel (a) is the control the main text states in prose: over the same $738{,}730$ joined events on the same clock, the acknowledgment-referenced span falls below zero $62{,}264$ times and the send-referenced span never does, so the negatives are a property of which timestamp serves as the origin and not of the delivery being timed. Panel (b) puts a millisecond timestamp in front of that population. The difference of two truncated timestamps lands on 0 or on 1 according only to where the tick boundaries fall, and a guard that admits a sample only when the difference is strictly positive removes $338{,}242$ of $738{,}730$ events — 45.8%, 95% interval 45.4–46.2 resampling runs rather than events — of which $303{,}468$ computed to exactly zero and $34{,}774$ to less. None of it is counted in what such a harness reports. Panel (c) applies the five dispositions the registry of Section S25 found in shipping software to that one measured population: two of them shrink the sample the statistic is computed from and publish no count of what left; two hold the sample count fixed by reporting a value that was never measured; one keeps the data.

One qualification, because the panel invites the stronger reading. Panel (b) is that rule applied to *our* spans, not a record of the external benchmark deleting our events: it says what a millisecond admission condition would remove from this population. The per-cell retention the main text quotes is the other measurement — what that harness's own instrumented runs did report — and the two answer different questions.

S14.1. The two floored clocks behind the $-1000\,\mu$s signature

Section VI-C states the signature and what it rules out. This is why the two drivers differ, which is a property of the harness rather than of either broker.

The Redis driver takes its publish timestamp from `entry.getID().getTime()` in `RedisBenchmarkConsumer.java`:72 — the stream-entry identifier the *Redis server* assigns on `XADD` [S8]. The Kafka driver uses a producer-set timestamp. So the Redis end-to-end span subtracts a consumer-side wall clock from a server-side one, across two hosts, with both floored to the millisecond; the Kafka span floors two timestamps whose difference is taken inside one JVM.

Two consequences follow, and together they are what makes the sign channel legible.

First, the arithmetic admits exactly one negative value. Two clocks floored to the same timestamp resolution, offset by less than one tick, can disagree by one tick and by no other amount, so the only negative a floored cross-host difference can produce is $-1000\,\mu$s. The corpus contains $41{,}403$ negatives and every one of them is that value.

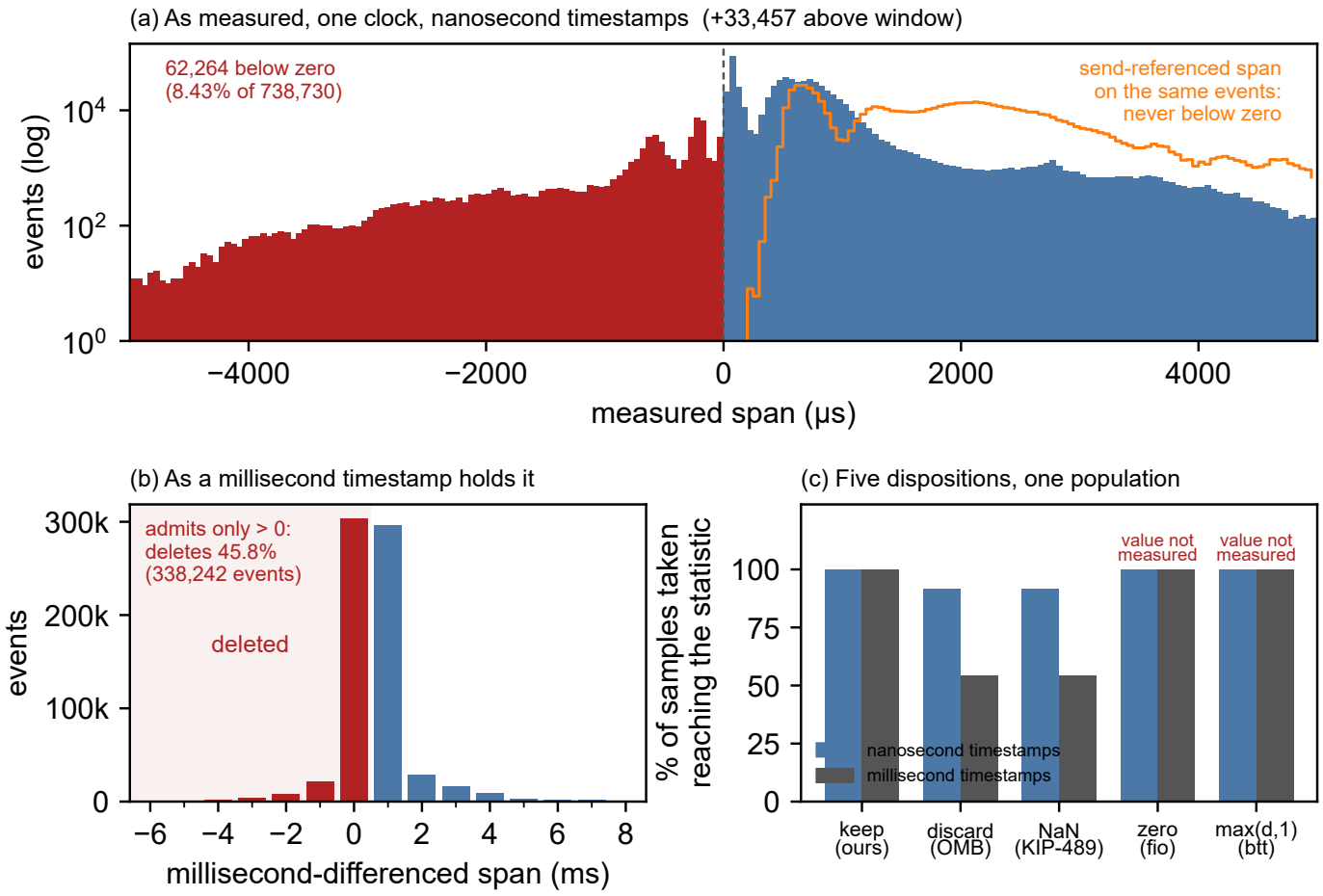

Fig. S10. **What the benchmark deletes, beside what it then reports.** (a) At nanosecond resolution the acknowledgment-referenced span goes below zero 62,264 times in 738,730; the send-referenced span, same events and same clock, never does. (b) A millisecond timestamp cannot see (a): it differences two truncated timestamps, so a sub-millisecond delivery lands on 0 or 1 by tick alignment alone, and an admission condition then deletes 45.8% of the population uncounted. (c) The five dispositions found in shipping software (Section S25), applied to that same population. Every count is emitted from the committed corpus by `scripts/span_histogram.py`.

Second, the Kafka corpus's 10,913,263 discarded samples contain no negative at all, because a difference taken inside one process has no second clock to disagree with. The same admission condition therefore hides two different failures — a resolution failure under Kafka and a clock failure under Redis — and an unsigned discard counter could not tell them apart.

It also means the two drivers do not share a measurand: Kafka's span is client-to-client on one clock, Redis's server-to-client across two. A comparison of the two numbers as though they measured the same interval is a further error the harness invites and does not flag.

S14.2. The permutation null, constructed

Section VI-B names the test and reports its verdicts. This is how the null is built, which matters because the null is the whole content of the claim: a continuum hypothesis has to be turned into a distribution before anything can reject it.

For each configuration, let D be the mean distance from its replicates to the nearest vertex of the q-grid, normalized by the cell half-width. Under the continuum null, retention is a continuous function of rate, so a replicate's position within its cell carries no information and the positions are exchangeable and uniform on the cell. The null distribution of D is therefore obtained by permuting replicate positions within the configuration: 10^4 permutations, which fixes the smallest attainable p at 2.5×10^{-4} and is why no configuration reports a p below that value however far it sits from the diagonal.

Two properties of this construction are worth stating because they bound what the test can say. It is exact under the null rather than asymptotic, which matters at $n = 5$. And it has no power at all where the grid and continuum predictions coincide, which is not a weakness to be apologized for but the reason the two configurations at those rates are reported as untestable rather than as evidence — a test that separated them everywhere would be measuring something other than what it claims.

S15. WHICH CLOCKSOURCE, BOUNDED FROM A MEASUREMENT

Section VIII-D states the verdict in one sentence: a read-cost bound admits only two clocksources, and the corpus closes the channel a second time on headroom. Both halves are derived here — the increment the probe saw, what it excludes, and why the send-referenced span's floor settles the question independently of which clocksource was in use.

A reviewer asked which clocksource the cloud instances used, because a virtualized guest need not be given a counter that is coherent across its vCPUs, and if ours were not, that would be a rival to the account in Section V. We did not record the field, and the instances have been reclaimed, so it cannot be read back. Our first answer was that we could not say. That answer was wrong, and the correction is worth setting out because the method is the same one the rest of the paper uses: a physical constraint deciding what the record-keeping did not.

The platform probe recorded the smallest non-zero increment observable from `clock_gettime(CLOCK_MONOTONIC)` on the machine that produced the reported runs: 90 ns. An increment is observed by reading the clock twice, so that number is an upper bound on the cost of a single read — and read cost separates the four candidate clocksources by more than an order of magnitude.

acpi_pm, excluded by arithmetic: The ACPI power-management timer counts at `PMTMR_TICKS_PER_SEC` = 3,579,545 Hz (`include/linux/acpi_pmtmr.h`), a period of 279 ns. A clock whose tick is 279 ns cannot report an increment of 90 ns, whatever it costs to read. This exclusion needs no timing figures at all.

hpet, excluded by cost: HPET is memory-mapped I/O to an off-die chipset timer. The x86 maintainers, in the commit removing it from the vDSO, describe reads as "exceedingly slow (on the order of several microseconds)"; published measurements of a `clock_gettime` call on HPET sit near 1.2 μs. We are careful about the ground here: HPET's own tick, near 70 ns at 14.318 MHz, is *finer* than the observed increment, so granularity does not exclude it and we do not pretend it does. The exclusion rests on read cost alone.

What remains, and why it answers the question: That leaves `kvm-clock` or `tsc`, measured at 33–60 ns and 24–29 ns respectively, both comfortably inside the bound. Each carries the guarantee the reviewer asked about, by a different route.

For `kvm-clock` the route is the vDSO. Without vDSO acceleration the same clock costs about 156 ns per call, which the bound excludes; and `arch/x86/kernel/kvmclock.c` sets the vDSO mode only under a condition:

```
if (!(flags & PVCLOCK_TSC_STABLE_BIT))
    return 0;
kvm_clock.vdso_clock_mode = VDSO_CLOCKMODE_PVCLOCK;
```

`PVCLOCK_TSC_STABLE_BIT` is the flag the KVM guest ABI documents as "time measures taken across multiple cpus are guaranteed to be monotonic" [S9]. A `kvm-clock` fast enough to fit the bound is therefore one whose hypervisor asserts cross-vCPU monotonicity.

For `tsc` the route is selection. `kvm-clock` registers at rating 400 and `tsc` at 300, so on a KVM guest `tsc` is chosen only where `kvmclock_init()` demotes `kvm-clock` to 299 — which it does exactly when `CONSTANT_TSC` and `NONSTOP_TSC` are present and `check_tsc_unstable()` is false. Should warp appear later, `tsc_sync.c` reports "Measured %Ld cycles TSC warp between CPUs, turning off TSC clock" and the source is re-rated away.

What this does not establish: It identifies the set of clocksources consistent with the measurement, not which one was in use, and it leans on published read costs for hardware we no longer hold. Those costs vary by roughly a factor of two between independent measurements; the margin here is an order of magnitude, which is why the conclusion survives the spread. It is an inference from a measurement rather than a recorded fact, and Section IV-A says so. The derivation is `scripts/clocksource_bound.py`, and its tests pin each exclusion to its own ground so that an exclusion firing for the wrong reason cannot reach the right answer by luck.

S16. WHY THE ACKNOWLEDGMENT TIMESTAMP IS LATE, AND WHY THE BROKER IS NOT THE REASON

In plain terms: the span that inverts is the one whose reference timestamp is read only after a thread has been woken, and the corpus shows twice over that this has nothing to do with which broker is running. Section V states the mechanism; this section gives the source lines, the control that separates the candidate causes, the rivals that were considered and dropped, and the parts we could not test.

S16.1. Where each timestamp is actually taken

Producer and consumer are separate processes on one host, and both read CLOCK_REALTIME. Five timestamps enter the spans this paper reports, and they are not taken in comparable places. Three of them are producer-side.

- **The send timestamp** is read on the calling thread immediately *before* the client is invoked. In `scripts/redis_producer.py` the worker executes `send_ns = now_ns()` and only then calls `r.xadd(...)`.
- **The Redis acknowledgment timestamp** is read on that same thread the moment `r.xadd(...)` returns. `xadd` is synchronous: the thread writes the request and blocks in a socket read until the reply arrives. No second thread is involved anywhere.

- **The Kafka acknowledgment timestamp** is read, by default, in the client's I/O thread when the delivery callback fires. `scripts/kafka_producer.py` also offers `--ack-stamp inline`, which timestamps on the calling thread once the send future resolves, matching what `xadd` does; it requires `--max-inflight 1`.

The two clients therefore differ in whether a second thread stands between the reply and the clock read. They do not differ in something more basic: in both, the acknowledgment timestamp is read only after a thread that was not running has been made runnable and then scheduled.

The consumer's two timestamps, and the asymmetry between the clients: The other two are consumer-side, and until this revision neither this section nor any other said where they are taken. That was a gap in an audit whose whole subject is timestamp placement: every span the paper reports ends at one of them.

- **The receive timestamp** t_{recv} is read when the consumer holds the record. In `scripts/kafka_consumer.py` the client is constructed with a `value_deserializer`, so the payload is parsed inside `poll()` and the parse precedes the timestamp. In `scripts/redis_consumer.py` the timestamp is read first and `json.loads` runs after it.
- **The output timestamp** t_{out} is read once the record is in hand. It is the next statement under Kafka and is separated from t_{recv} by the payload parse under Redis.

So the consumer's own handling span, $t_{\mathrm{out}} - t_{\mathrm{recv}}$, is 281 ns under Kafka and 19,480 ns under Redis — medians over run medians across 5,913 runs, a factor of 69. It never goes negative on any of the 738,730 events, as a chain cannot.

Three consequences, and we state the direction of each because a disclosure without one is an adjective. *First*, the payload parse falls inside $D = t_{\mathrm{recv}} - t_{\mathrm{send}}$ for Kafka and outside it for Redis, so the transport proxy and D carry one client's parse and not the other's; against Redis's median delivery of $800\,\mu$s that is 2.4%. *Second*, a longer D produces fewer negative spans, and the broker carrying the extra parse is the one with the *higher* reported rate, so correcting the asymmetry would widen the gap in Table I rather than close it. *Third*, TTI contains both parses and is unaffected, which is one more reason the equivalence in S13 is also stated on TTI.

We could not fix this in the pipeline where it belongs, because it would require re-running a campaign whose machines are released. What the pipeline now does instead is measure it: `recount_spans.py` carries the span as its fifth column, and a gate fails if the two consumers ever timestamp in places that differ again without the number moving with them.

S16.2. Why one span cannot invert and the other can

The send timestamp is taken before the message leaves the process, so $t_{\mathrm{send}} \rightarrow$ emit $\rightarrow$ deliver $\rightarrow t_{\mathrm{recv}}$ is a chain and its ordering holds by construction, however late the timestamping thread was: a late thread makes the span longer, never shorter. That is not an empirical claim about our scheduler, it is a property of the order in which the two operations are written, and it is why the send-referenced span is negative on 0 of 738,730 events under either broker.

The acknowledgment timestamp has no such guarantee. The broker's append precedes both the producer's receipt of the acknowledgment and the consumer's receipt of the record, but those are two downstream branches with no happens-before between them (Supplement S1.3). Both branch timestamps are read after a wakeup, so the error on their difference is two-sided, and the difference goes negative whenever the acknowledgment-side delay exceeds the true interval plus the receive-side delay. The exposure lies in which operand can be late in the shortening direction, and that is fixed by how the span is constructed, not by what is being measured.

S16.3. The second thread is not the mechanism

The harness was built expecting the thread hand-off to be the cause; the comment at `kafka_producer.py` says that Redis "timestamps inline on the calling thread the moment XADD returns, and never pays that wait". The corpus does not bear that out, and the disagreement is informative.

Two committed results bear on it. First, the timestamping campaign (`ec3`) ran Kafka under both settings at matched load, and Table S26 is the outcome: moving from `callback` to `inline` lowers Kafka's median while Redis does not move, shrinking the between-system gap by about a quarter, and a thirty-run replication reproduces it. The hand-off is therefore real, it is worth some tens of microseconds of median inflation, and a comparison that leaves it in place compares two instruments as well as two brokers.

Second, and against the expectation, removing the second thread does not remove the negative spans. Redis, which never has one, inverts on 8.19% of its 370,836 events against Kafka's 8.67% of 367,894; by runs affected Redis is the worse of the two (88% of runs against 65%). A thread returning from a blocking socket read must also be made runnable and then scheduled before it can call the clock, and that wait is charged to the measurement exactly as a callback dispatch would be. What the two clients share is the wakeup; the hand-off is an increment on top of it, not the thing itself.

What we could not test: The clean form of this control would be the negative-span rate for Kafka under `--ack-stamp inline`, compared with the same runs under `callback`. The timestamping campaign retained only its median summary (`docs/results/model/ec3_stamping.csv`); the per-event records that would carry sign-separated counts were not kept, and we do not reconstruct them here. The cross-broker contrast above is the evidence we have, and it is weaker than the within-broker one would have been.

S16.4. Two rivals: the scheduler itself, and the hypervisor

Neither the operating system nor virtualization is the explanation, and it is worth saying why in each case.

The scheduler is not malfunctioning. A base slice with a wakeup-preemption check is a throughput and fairness trade-off, and the arithmetic of Section V-E is the kernel maintainers' own. A general-purpose scheduler that guaranteed sub-hundred-microsecond wakeup latency to every thread would be a worse general-purpose scheduler, which is why the escapes from it are opt-in: real-time priority, a preemption-free build, or a pinned busy-polling core. The kernel also already publishes the repair — `SO_TIMESTAMPING` returns the time the packet arrived rather than the time a thread got round to asking — and none of the instruments we audited uses it.

Virtualization is not necessary either, and our own numbers point the other way. The workstation testbed runs the brokers in containers with no hypervisor beneath them and the audit rejects 62.4% of its assessed runs, against 51.9% on the cloud instances. The effect is larger where there is no hypervisor at all. Two further observations disagree with a steal-time account: real-time priority is a guest-side control and collapses the rate 39–$54\times$ (these two matched pairs; all 8 are in S12, and Section V-B of the main text gives 7–$80\times$ across the load sweep), which stolen vCPU time would not yield to; and the 2–4 ms mode sits on the guest kernel's own base-slice arithmetic rather than on any quantity the hypervisor exposes. Steal time may still contribute on the cloud instances, where the mechanism campaigns were run; what the workstation shows is that it is not required.

A third rival, inside the interpreter: The two rivals above are the ones a reader expects. A third is specific to how this harness is built, and the manuscript owes it a paragraph because the mechanism was restated around a thread reaching a CPU.

The timestamping threads are CPython (Section IV), and both producers are multithreaded: `redis_producer.py` dispatches sends through a thread pool and runs a correction worker beside it. CPython releases the interpreter lock around blocking I/O and takes it back on return, and `now_ns()` is a Python call. So the sequence is not "reply arrives, thread scheduled, clock read" but "reply arrives, thread scheduled, *lock reacquired*, clock read". The extra wait is bounded by the interpreter's switch interval, which our campaigns left at the default and none of them recorded; the platform probe now records it, and `interpreter_serialisation()` is part of every future run's provenance.

Three things follow, and we state them because two of them are uncomfortable. First, the traced estimator cannot see this term: `runqlat` measures `sched_wakeup` to `sched_switch`, and lock acquisition happens after the switch, in userspace. Second, the priority manipulation does not cleanly separate it, because a thread raised to `SCHED_FIFO` also reaches the acquisition queue sooner. Third, magnitude does not separate them either: the default switch interval and the base slice of Section V-E are both of millisecond order, so neither mechanism is excluded by the size of the delay it would produce. Note what the first point costs us here — the main text's stall spectrum cannot adjudicate between them, because it is built from the tracepoints that miss one of the two entirely.

What settles it is the direction of the disagreement rather than any of those. If a second serialization sat in series with the run-queue wait, an estimator built from the run-queue wait alone would fall systematically *short* of the observed negative-span rate. Over 3 configurations it does not: the ratios are $0.78, 1.06, 1.32$, straddling unity, with the largest above it. A term big enough to matter would have to hide inside a spread that already brackets the answer. The interpreter lock is therefore present in principle, bounded in practice, and not the mechanism — and the send-referenced span is immune to it entirely, since a timestamp taken before the action bounds its event whatever the interpreter is doing.

S16.5. Why averaging does not repair either mode

For the deletion failure the natural objection is that quantization error falls both sides of the truth, so averaging should recover it. That is exactly right, and it is why the deletion matters: the readings that straddle are the estimator, and HP AN 162-1 makes the point in 1970. But the correspondence stops one step further on, and the paper's concession to that note should not be read as covering the case here. A counter measures a *constant* interval, and which side of a tick a reading falls on is decided by phase, which is independent of the measurand. Our retention is $\min(1, T_{\mathrm{true}}/\tau)$, a function *of the measurand*: the faster the delivery, the likelier it is deleted. The surviving population is therefore selected on the quantity being estimated, which a fixed interval cannot be, and averaging over the survivors does not converge on the truth however many are taken. It also cannot produce a percentile at all, and a latency benchmark reports percentiles.

The consequence is worth stating in its own right. For a sub-resolution path every survivor reads exactly one tick, so the reported median is pinned at τ while the truth is T_{true}, and the overstatement is τ/T_{true}, the reciprocal of the retention. A tool of this design penalizes the paths that are fastest, and by the largest factor exactly where it deletes the most.

For the scheduling failure averaging is worse than unhelpful. The error is not symmetric noise but a one-sided lateness on one operand, so it does not cancel; and nothing in the path carries what would be needed to remove it afterwards. The reply does not report when it arrived, the thread cannot know how long it was not running, and the only clock reading in existence is the late one. This is not a bias that can be subtracted but information destroyed at the moment of observation, which is why finer clocks, better synchronization and larger samples all leave it untouched.

What does work follows from that. Timestamp before the action rather than on resumption, so the reading bounds its event. Where the event is not yours to timestamp, attach the timestamp to it — a kernel or hardware receive timestamp, or a broker-assigned one — which is the probe-placement argument in Section VIII-B. Keep a sub-resolution sample as a left-censored

observation, since "$T_{\text{true}} < \tau$" is a fact about the measurand and deleting it discards the censoring indicator along with the value. And treat a negative as a gate rather than a datum: it is not a censored observation but the reference timestamp failing, so it belongs in a sign-separated count and in a run-level rejection, never averaged into a distribution.

S16.6. The redesign that does not help

A reader who accepts the mechanism usually proposes the same fix: have the consumer acknowledge back to the producer, so that one clock times a round trip and the two-clock problem disappears. It is the right instinct and it does remove skew, but it does not remove this failure, for two separate reasons.

The first is that it measures the wrong quantity. A producer-side round trip is not the staleness of a record at the consumer, and staleness at the consumer is what a streaming pipeline is judged on. Substituting the one for the other answers an easier question than the one asked.

The second is that the stall does not care how many clocks are involved. Every negative in this corpus was produced on a single clock, and the configuration in which both processes read that one clock is the configuration the audit rejects most, at 62.4%. A round trip still ends with a thread being woken and scheduled before it can read the clock, so it inherits the same late reference timestamp the acknowledgment span has. Collapsing two clocks into one removes a term the corpus already shows is not the cause.

S16.7. Where the repairs belong

No layer here misbehaves, which is why the fix is not obvious from inside any one of them. The kernel schedules as designed, the clients return as soon as they can, the brokers acknowledge correctly. The benchmark is the only party that assumes something none of them promised: that observing an event tells you when the event happened.

- **Operating system:** nothing is owed. The escape hatches and `SO_TIMESTAMPING` already exist.
- **Kafka:** `record.timestamp()` is a millisecond wire field, so anything built on it inherits a millisecond resolution and must carry its own finer timestamp in the payload to escape. Documenting that, or exposing a finer causally attached timestamp, is the ask.
- **Redis:** the `XADD` entry identifier embeds a server-clock, millisecond-floored time. One documented sentence saying it is not comparable with a client clock would have prevented the driver defect of Section VIII-A outright.
- **Benchmark authors:** timestamp before the action; count discards sign-separated; publish retention, timestamp resolution and synchronization state; never difference a server-assigned timestamp against a client clock without saying so; dither the send instant; and record the achieved rate.

S16.8. The scheduler constants, derived

The main text states the base slice and moves on; this is where it comes from. The constants are *derived* from the published kernel package and the campaign's own data rather than measured, which is weaker evidence than a measurement and is reported as such.

The core count is recovered from the experiment rather than assumed: each geometry condition loads k cores and records the achieved utilization, so k/ρ recovers 8 six times over with a spread of 0.05. The kernel's own rule then gives the base slice, $4 \times 0.75\,\text{ms} = 3$ ms — the maintainers' arithmetic, "0.75 msec * (1 + ilog(ncpus)) ... 3.00 for 8 cpus and above" [S10]. It is specific to this kernel: the constant became 0.70 ms in v6.15, after our 6.8. The build sets `CONFIG_HZ`=1000, a 1 ms tick.

With run-to-parity a woken thread that loses the wakeup-preemption check waits for the incumbent to exhaust that slice, and the expiry is noticed at the tick. That tick is also why the mode is a band and not a spike: the same posting notes a task "can only wait until the next tick to consider that it has reached its deadline, and will run 1ms longer".

No scheduler tunable is written anywhere in the archived campaign, which is why the derived value is the one that applied. It is strong evidence rather than proof.

S17. THE 1970 COUNTER NOTE, LINE BY LINE, AND THE GRID GEOMETRY IT EXPLAINS

Figure S11 draws the identity this section traces: one delivery at four phases of a tick, three of them rounding to a difference of zero and deleted, and beneath it the phase diagram that decides how many of a run's sends survive.

Section III-C concedes the arithmetic of the deletion failure to Hewlett-Packard's Application Note 162-1 [S11] and states what remains ours. This is the correspondence in full, so the concession can be checked rather than taken on trust.

- **The retention identity.** For an interval of $Q + F$ clock periods, AN 162-1 gives $\Pr[R = Q{+}1] = F$: the fraction of readings landing on the upper of the two straddling counts is the fractional part itself. That is the retention law of Section VI.
- **The class of a repetition rate.** Its *class* is the denominator of the ratio of repetition rate to clock rate in lowest terms, under the same co-prime condition that fixes our q, and it determines how many distinct readings the counter can produce.

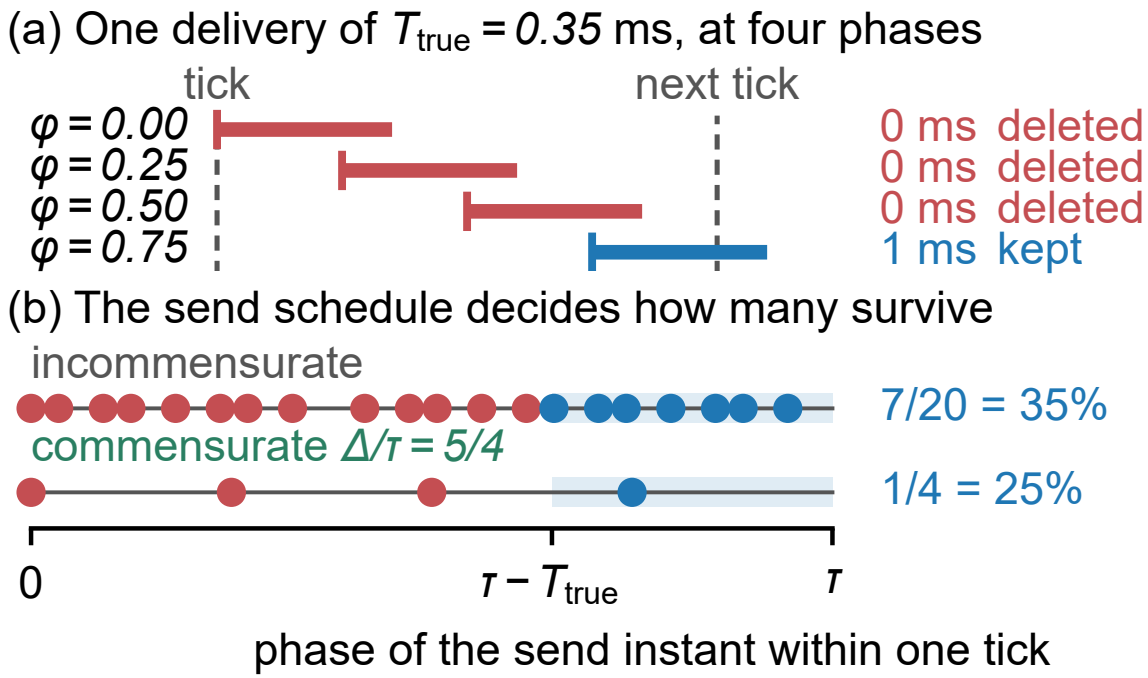

Fig. S11. **Why a sample disappears, and why the survivors are not a sample.** (a) One delivery of $T_{\mathrm{true}} = 0.35$ ms at four phases of one tick. Three finish inside it, so both timestamps round alike, the difference is exactly zero, and the admission condition removes them; the fourth crosses the boundary and is recorded as one whole tick — which is why the corpus prints 1.0 and 2.0 ms and nothing between. (b) A delivery survives from phase $\tau - T_{\mathrm{true}}$ onward — the shaded band — so uniform phases retain T_{true}/τ (Equation 4); a commensurate producer visits only q and retains a count out of q, one of the values Equation S3 allows. T_{true} here is illustrative and off-vertex by choice: at 0.5 ms the two answers coincide.

- **The ceiling on averaging.** The gain in resolution available from averaging N readings is bounded by a factor $1/q$ and not by $1/\sqrt{N}$, which is the quantization ceiling of Section VI-B. AN 162-1 names $q = 1$ as the corner where averaging cannot help at all — the case a paced producer on a round interval walks straight into.
- **The symptom.** It tells the operator what to look for: a counter reading that appears to hang on a round number while the input is known to vary.
- **The cure.** Phase jitter on the repetition rate, so the samples stop landing on the same points of the clock's cycle. We arrived at it independently and recommend it in Section VIII-B.

The one thing the note does not contain is the failure this paper reports, because its instrument keeps both readings. Retention is the estimator there; deleting the population that carries the fractional part is what converts a solved metrology problem back into an open one.

When dither works, which is a question with an answer: Our remedy is to dither the send instant, and a reader from signal processing will ask under what conditions dither actually recovers the underlying quantity. It is a fair question and the literature has answered it. For quantized Gaussian measurements subtractive dither helps when the ratio of the noise's standard deviation to the quantization interval is roughly *below* one third; above that it costs more than it buys [S12]. The condition is one-sided. Their second threshold does have a lower bound, but it decides which *estimator* wins inside that regime rather than whether dither helps at all, and we do not lean on it. We name the result rather than apply it, because our dither is not that dither. Theirs adds noise to the *value* being quantized; ours randomizes the *phase* of the sampling instant against the tick, which is RFC 3432's construction [S13] and is what breaks the commensurability of Section VI-B. The two coincide only in the uniform-phase limit that Equation 4 already assumes. The neighboring result is worth the reader's attention anyway: it is the closest thing to a proof that the recommendation has a domain of validity rather than being folklore.

Part III. Apparatus, Data and Tables

This part is reference material, meant to be looked things up in rather than read through. Section S18 maps every experiment to the claim it supports, and Section S19 states the threats to the scheduling attribution with what rules each one out. Sections S20 to S23 describe the workload, every setting that can move a latency number, the harness and the transfer procedure. Sections S24 and S25 give the campaign ledger's schema and the source evidence behind the count of tools that dispose of a sample silently. Section S26 collects every table.

Campaign	Manipulated	Held fixed	What it settles
E1 concurrency	feeds *N*: 1, 9, 10, 12	rate, host, plan set	the original question: does broker choice matter?
E-B delay sweep	injected delay 0-50 ms	load, feeds	H1 effect size (direction only; confounded)
E-A / E-A3 load sweep	background load 0-12	rate, feeds, plan	H2 utilization, H10 mixture, F_Δ recovery
E-A2 process count	processes at fixed rate	aggregate event rate	H4 oversubscription
E-C3 / E-C4 timestamping	ack timestamp: callback vs inline	load, in-flight, backend	H3 asymmetry (replicated)
window sweep	observation window 60-600 s	rate, host, match	start-up cost vs per-event constant
transport (two runs)	feeds, powered	verified real-time rate	broker transport, equivalence + shift
E-A5/A5b/A7 timestamping priority	SCHED_FIFO on the timestamping threads	utilization, to 0.003	scheduling, not utilization (8 pairs, 7-80×)
E-A6 / E-A6b load geometry	cores free vs all duty-cycled	achieved utilization	utilization is not the variable (2.07×, 2.05×, at rho 0.7531)
E-A9 run-queue trace	nothing: observation only	load, priority setting	P(stall > T_{true}) predicts the rate, unfitted
E-A10 transport sweep	payload size; T_{true} 77×	load, hosts, code path	other side of the inequality; exponent 0.34, not settled
E-A8 co-location	broker on the driver	utilization, to 0.003	nothing: transport did not move, so it is withheld
OMB external	instrumented discard counter	our broker, under load	the exposure is not ours alone

Every claim in the discussion traces to one row. No row both generates and tests the same hypothesis, and a row that settled nothing says so.

Fig. S12. Experiment map. Each campaign manipulates one variable, holds the others fixed, and settles one claim. The delay sweep is marked confounded because injecting delay also inflates variance (the main text), so we use it for direction only.

S18. The experiment map, the claim-evidence table, and the audit threshold figure

Figure S12 is the experiment map; Table S10 is the claim-evidence table; Figure S13 shows the threshold's consequences.

S18.1. What the audit gate selects, bounded

Three artifacts bound the selection the audit gate introduces, each recomputable by a committed script. Table S11 applies the audit gate to the powered transport campaigns and bounds the selection the gate introduces (`powered_gate_sensitivity.py`); the condition-level threshold sweep behind Section IV-E of the main text is `first_result_threshold_sweep.csv` (no first-result cell fully usable at any threshold up to 20%; best cell 23/30); the traced survival slopes that used to back the main text's effective-exponent statement are `traced_tail_slope.csv` (windowed log–log index 0.332 over 0.25–2 ms, 4.128 over 4–32 ms). That last artifact is retained for the record but is *superseded*: the slopes in it are least-squares fits through nested survival points with no interval, and S8 explains why they do not estimate what we claimed they estimated. The estimators that replace them are in `scripts/tail_index_traced.py`.

TABLE S10
WHAT EACH CLAIM RESTS ON. E1 IS THE ONLY CORPUS WHERE RETENTION IS A LIVE CONCERN; THE CAMPAIGNS AFTER IT RETAIN EVERY RUN, SO THE BREAKDOWN-POINT ARGUMENT OF THE MAIN TEXT IS NEEDED FOR ONE ROW ONLY.

Claim (section)	Campaign	Runs per cell
Audit rejection rates (main text)	all corpora	2,266 runs total
H1 effect size (main text)	co-located vs network	15 per delay level
H2 utilization shape (main text)	E-A, E-A3	25 per load level
H4 process count (main text)	E-A2	15 per level
H3 asymmetry (main text)	E-C3, replicated E-C4	10, then 15 per configuration
Mixture structure (main text)	E-A3, nine levels	25 per level
Scheduling, not utilization (main text)	E-A5, E-A5b, E-A7	25 per configuration
Load geometry (main text)	E-A6, both geometries	25 per configuration
Traced run-queue tail (main text)	E-A9	25 per configuration
T_{true} sweep (main text)	E-A10, four payloads	25 per level
Broker transport (main text)	powered, replicated	15, then 8 per cell
Start-up withdrawal (main text)	window sweep, twice	3 per window
Historical: E1 (main text)	E1	8–35, *52.8–100% retained*

TABLE S11
AUDIT GATE APPLIED TO THE POWERED TRANSPORT CAMPAIGNS. "FLIP V^*" IS THE SMALLEST VALUE AT WHICH IMPUTING EVERY CONDEMNED REDIS RUN AT V^* WOULD FLIP THE HODGES–LEHMANN SHIFT'S SIGN; THE CONDEMNED RUNS' OBSERVED MEDIANS CENTER ON 0.10–0.12 MS.

Campaign	N	Redis ret. (%)	HL all	HL gated	Δ	Flip V^*
powered	1	53.3	0.4095	0.4084	-0.0011	—
powered	9	43.7	0.4181	0.4156	-0.0025	0.60
powered	12	37.0	0.4197	0.4170	-0.0027	0.60
replication	1	25.0	0.3969	0.3807	-0.0162	0.55
replication	9	45.8	0.4120	0.4137	$+0.0017$	0.60
replication	12	39.8	0.4132	0.4157	$+0.0025$	0.55

S19. THREATS TO THE SCHEDULING ATTRIBUTION, AND WHAT RULES THEM OUT

The mode count is checked against the binning, because it has to be: A count of local maxima read off a histogram belongs jointly to the counts and to the bin edges, and this paper argues that instruments impose structure which is then reported as a property of the system. Figure V-E reports local maxima of a `bpftrace` $\log_2$ histogram, and those bucket edges came from the tool exactly as the millisecond quantum of Section VI came from the benchmark. The objection applies to us.

The usual answer is unavailable and will stay unavailable: `runqlat.txt` holds the tool's summary and **the per-event stall durations were not retained**, so the raw data cannot be rebinned now or later. What the archived histogram does support is coarsening. Merging adjacent buckets doubles the bin width, and the merge can be phased two ways — pairing from the first bucket, or from the second — which puts the edges in different places. A mode that survives both phasings is not sitting inside one bucket by luck.

It survives. 3 maxima on the tool's own 16 buckets, 3 again under both phasings at $2\times$ the bin width, and fewer at $4\times$. The claim therefore holds to within a factor of 2 in bin width and is resolution-limited beyond it — a weaker statement than we could have made carelessly, and a stronger one than a disclaimer. `scripts/stall_mode_robustness.py` recomputes it, and the suite fails if the committed record and the trace disagree.

Threats to validity (full)

Construct validity: the check may not measure what we claim: A negative span is proof that *something* is wrong, but our attribution of it to scheduler delay in reaching the clock is an inference. Two rival explanations are available. Clock skew is ruled out directly for Testbed A, where both processes read the same operating-system clock; it is not ruled out on Testbed B, where producer and consumer may sit on different hosts, and there we rely on the co-located conditions failing at rates comparable to the distributed ones.

Coarse kernel timestamp granularity would also produce negative spans, and that rival makes a different prediction we can test. Quantization acts on each event independently, so its negative spans would be scattered at random through a run; a descheduling event, by contrast, makes a *consecutive run* of events wake late together. A Wald–Wolfowitz runs test on the sequence of span signs separates the two: independence gives $z \approx 0$, clustering $z \ll 0$. Across conditions the negative spans are strongly clustered — median $z = -6.9$ at the co-located floor, -6.8 idle, -5.1 saturated — and the clustering is present even with no background load, so it is intrinsic to the timestamping path rather than induced by contention. This is the signature of a shared scheduling delay, not of independent quantization, and the main text additionally reports the measured

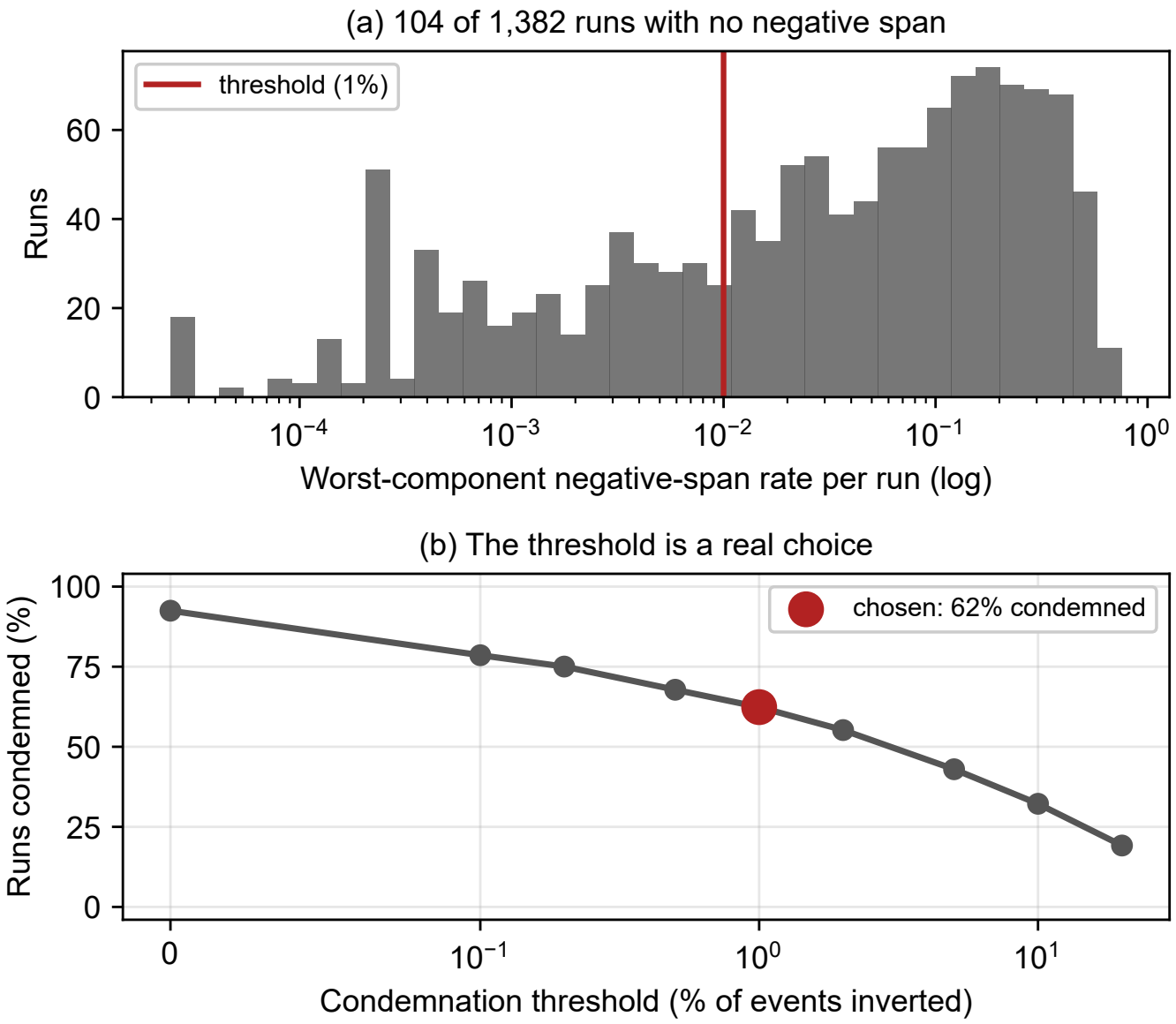

Fig. S13. Audit on Testbed A. (a) Per-run negative-span rates: 104 runs carry no negative span at all and the rest spread over three decades, so there is no natural threshold. That is the count at which the rate is exactly zero, not the number the audit passes: the one-per-cent rule condemns 62.4% of the 1,382. (b) Share of runs rejected against the threshold. The choice matters for the run count and not for any conclusion.

timer granularity on both platforms. We regard the Testbed A attribution as well supported and the Testbed B attribution as probable rather than established.

Internal validity: the audit selects: The check rejects unequally between configurations, so the surviving corpus is not a random sample of the measured one. This is the most serious internal threat, and the main text addresses it with a worst-case bound rather than an argument. That bound holds only while retention exceeds one half, and our tightest cell sits 2.8 points above that line; a slightly worse campaign would have left the equivalence claim undefendable.

Internal validity: the surviving comparison is small: After rejection, per-cell samples run from 8 to 35. The $N{=}1$ cell cannot support inference and we treat it as descriptive; conclusions rest on the levels with 19 or more runs.

External validity: one workload, two systems, two platforms: We demonstrate the failure on a sparse bursty feed, two brokers and two operating systems. The model of the main text predicts it wherever cross-process spans are measured near the harness's own jitter under load, but we have not shown that. Sharma et al. [S14] report the same symptom in an unrelated domain, which is suggestive but is not our evidence.

Construct validity: a percentile is not automatically a property of the system: The 103 ms producer offset reproduced across both testbeds, four concurrency levels and 164 runs, and we took that breadth as evidence it was a property of the system. It was a property of the window (the main text). Reproducibility across runs distinguishes a deterministic effect from

a noisy one; it does not distinguish a system effect from a harness effect, and we treated it as though it did. Every percentile in this paper is now reported with the events per run behind it.

Construct validity: the injected-delay sweep is not a clean effect-size manipulation: H1's quantitative slope (the main text) is estimated from a `tc netem` delay sweep, on the assumption that injecting d ms of delay simply raises T_{true} by d. It does not: on our bursty feed, a per-delivery delay builds a queue, so the measured transport *variance* climbs from 10 ms^2 at zero delay to $9{,}200$ ms^2 at 50 ms — a clean offset would leave it unchanged. The delay sweep therefore confounds T_{true} with backlog-driven jitter, and we do not lean on its fitted slope. H1's robust evidence is the comparison it does not confound: the co-located configuration, where $T_{\mathrm{true}} \approx 0.5$ ms, fails wholesale, while the network configuration, where T_{true} is tens of seconds, passes $15/15$ on the same hardware (the main text). Testing the fine-grained slope cleanly needs a manipulation that lengthens T_{true} without perturbing the scheduler, which netem is not. The payload sweep of the main text is that manipulation by another route: padding adds serialization and transfer time without contending for the run queue, and it moves T_{true} by $77\times$ against netem's confounded range. It is not a perfect constant offset either — serialization pushes utilization slightly *up*, which works against the observed direction rather than for it — but it is clean enough to carry the claim netem could not, which is why the T_{true} result rests on it and not on this sweep.

The mechanism is established; the quantity is not predicted: The clearest limit on this work is what it still cannot do. We can say what causes a negative span, manipulate it on both sides, and predict the rate to within a third in the three configurations where we traced the scheduler directly. We cannot write down the rate a benchmark will suffer at a given load on a given machine. Two attempts failed and are reported above: the queueing form fits worse than the mean once the sweep reaches the range where forms diverge, and our own registered bracket missed a $1.44\times$ growth by predicting 2.45–$3.07\times$. Both failed for the same reason — both were parameterized in ρ, and ρ is not the variable. The delivery factor $G(T_{\mathrm{true}})$ of Equation S2 is the nearest thing we have to a quantitative law, and it carries no load term at all: it predicts how the rate scales with T_{true}, not what the rate is. So the practical guidance this paper supports is a check to run, not a number to look up. A reader wanting to know whether their own measurement is exposed must measure it, which is precisely why we argue the check should be routine rather than the model be trusted.

Conclusion validity: multiplicity and pre-specification: Holm families are declared over the design actually executed rather than chosen afterwards, and equivalence margins were fixed before testing. The threshold in the check was fixed before the final campaign but *after* earlier campaigns had run, which is weaker than pre-registration; the sensitivity analysis of the main text exists partly to compensate.

Limitations (full)

The surviving evidence is narrower than the design. The audit removed the throughput sweep, the connection sweep above ten, the cluster configuration, the durability quantification and all of Testbed A. We cannot characterize behavior at the concurrency a multi-tenant platform would see.

E1's replay rate is inferred, not documented. The main text recovers it from the data and the recovery is, we think, correct, but it is an inference from a 52.34 ms median in one cell rather than a setting anyone recorded. A reader who does not accept the inference should read the main text as establishing that both configurations met the same rate, which is all the between-system comparison needs.

The E1 reconciliation is an explanation, not a re-run of E1. Supplementary material S5 shows that restricting the powered runs to their opening seven events reproduces E1's figures in both shift and absolute level, which is why we no longer treat the two campaigns as contradictory. But it demonstrates that the window is *sufficient* to produce the disagreement, not that it is the only difference between the campaigns: E1 ran on a corpus whose replay rate we infer rather than read, and we cannot rule out that some other difference contributes as well. What we can say is that no additional cause needs to be invoked.

The mechanism is established for our client, not for Kafka in general. The blocking first `produce()` is visible in the loop trace of every run (the main text), so for this client the attribution is direct rather than inferred. Whether a different client, or a pre-created topic, or a broker configured against auto-creation would pay the same cost, we did not test. The practitioner advice in the main text is written to be safe either way.

The occupancy result is a mechanism, not a load model. The main text shows by manipulation that the negative-span rate follows timestamping-thread occupancy rather than utilization, and that is the strongest causal claim in this paper. It is not a formula. We can say the rate falls fifty-fold when the timestamping thread stops being preempted; we cannot say what the rate will be at a given load, and both attempts to write one — the queueing form and our own bounded bracket — failed against the sweep. A practitioner should read this as "keep the timestamping thread runnable and measure what you get", not as a curve to substitute into a budget.

The external evidence is single-host, and the cross-host case is unmeasured. The 223-run campaign of the main text ran in embedded mode, so both timestamps came from one JVM on one host. That is sufficient for the resolution failure, which needs no second clock — the timestamp resolution is a property of the timestamp, not of the network — but it means the cross-host channel is established by source audit and not by observation. We attempted a distributed run eight times, at three background-load levels including none at all, and the benchmark's own coordinator-to-worker protocol failed on every one;

the load-starvation hypothesis we formed after the first failures is refuted by the unloaded attempts failing identically. That difficulty is worth recording rather than hiding, and it cuts both ways: a failure mode this hard to instrument is one a practitioner is unlikely to find by accident, and a distributed mode that does not complete a run in repeated attempts by readers of its source is itself a fact about the state of the tooling.

We are also explicit that the clock bound on this testbed does *not* rule the channel out. `chrony` reports inter-host offsets near 0.1 ms, which is comfortably sub-tick — but the bound it places on its own error, root dispersion plus half the root delay, is 4 to 7 ms per host across the 4 hosts captured, and the two worst of those bounds sum to 12 ms against a 1 ms timestamp. An operator checking whether cross-host subtraction is safe reads the offset line and concludes it cannot happen; the figure that bounds the disagreement is two orders of magnitude larger and three lines further down the same output. The channel is therefore open on this testbed, and we could not measure it.

H3 is measured at one operating point. The symmetric-timestamping comparison (the main text) was run at one in-flight request and one load level, because inline timestamping requires the single in-flight setting. The direction and the one-sidedness of the effect are what the model predicts, but we do not characterize how the 0.07 ms offset scales with load or pipelining.

The five-feed acknowledgment null is unexplained, as set out in the main text.

One workload. The arrival-rate result should generalize to any sparse bursty feed, but we demonstrate it on one.

S20. The workload is a live football feed, described as far as the results require

The workload is a live football event feed, and we describe it only as far as the results require. Full characterization of 3,315 matches from the StatsBomb open dataset [S15], spanning 52 competition-seasons from 2003 to 2023, is available in the artifact; event data of this kind is collected by trained human operators, whose accuracy has been studied [S16], [S17].

Three properties of the workload drive every experimental choice (Table S12). We name, for each, the later result that depends on it, because a setting section that cannot say what it is for should not be in the paper. A reader who wants only the measurement contribution can note that the workload supplies three things a synthetic generator would not have supplied — a sparse arrival process, a burst at a known instant, and an empirically bounded concurrency — and skip to the main text.

It is sparse. A match produces a median of $3{,}507$ events over roughly $8{,}412$ seconds, a mean rate of $\mathbf{0.415}$ events per second. This is four orders of magnitude below the rate at which either broker begins to strain, and it is the property that makes the main text's result possible.

It is bursty, and one burst has a known instant. Over a sliding ten-second window the same feeds peak at 2.70 events per second, a peak-to-mean ratio of 6.45; half of all inter-arrival gaps are one second or shorter. The feed is idle most of the time and busy in short bursts. One of those bursts is special: a kickoff is recorded as a pass, a ball receipt and a carry inside the same second, so *every* match in our corpus opens with at least five events sharing $t_{\mathrm{sim}}{=}0$. That coincidence — the densest burst arriving when the producer is coldest — is what turns a one-off client start-up cost into the median of a short run, and it is the mechanism behind the withdrawal in the main text. A synthetic Poisson generator has no kickoff, and would not have produced it.

Its concurrency is bounded and knowable. Match records carry kick-off times, so the number of simultaneous feeds is empirical rather than swept. Across 2,346 kick-off instants the largest carries **12** simultaneous matches, and at most **21** are in play at once. Domestic leagues schedule the final matchday simultaneously by rule, so a 20-team league plays ten concurrent matches. We therefore benchmark at $N \in \{1, 9, 10, 12\}$: the median slot, the upper quartile, the structural league bound, and the observed maximum. This is what keeps the concurrency axis of the main text from being an arbitrary sweep — the levels are measured facts about the domain, not round numbers. Two limits apply. The dataset is a sample, so observed simultaneity is a lower bound; and the bound is per league, so a platform serving many competitions faces their sum.

Taken together these also explain why the broker question turned out to be the least interesting thing we could have asked, and we would rather say so here than let a reader discover it in the main text. At 0.415 events per second and at most a dozen concurrent feeds, this workload sits four orders of magnitude below where either broker strains. The domain that made the failure visible is the same domain in which the original question has a boring answer.

S21. Every setting that can move a latency number

Table S13 states every setting that can move a latency number, with the reason it holds its value.

Mohammad [S18] finds that published Kafka comparisons are frequently not comparable because client configuration and acknowledgment semantics differ silently between the systems being compared. We therefore state every setting that can move a latency number, with the reason it holds the value it does (Table S13). Two of these were not chosen but *learned*: the pipelining setting and the acknowledgment-batching setting each cost us a result before we understood them, and both are the subject of the main text.

The general principle is that the two clients must be equalized on *semantics*, not on parameter names. Kafka and Redis expose different knobs, and matching them syntactically is what produces the asymmetries this paper is about. Concretely, Kafka's producer buffers and pipelines by default while our Redis producer dispatches to a worker pool; leaving `max-inflight` at 1 makes the Kafka client wait for a broker round trip per event while the Redis client does not, which is not a broker difference but a harness difference.

TABLE S12
WORKLOAD, FROM 3,315 MATCHES. MEDIANS ACROSS MATCHES; PEAK RATE IS THE MAXIMUM OVER ANY SLIDING TEN-SECOND WINDOW.

Property	Value
Events per match	3,507
Match duration	8,412 s
Mean arrival rate	0.415 ev/s
Peak rate (10 s window)	2.70 ev/s
Peak-to-mean ratio	6.45
Median inter-arrival gap	1.0 s
Distinct kick-off instants	2,346
Max simultaneous kick-offs	12
Max matches in play at once	21

TABLE S13
EVERY CONFIGURATION THAT CAN MOVE A LATENCY NUMBER, AND WHY IT HOLDS THAT VALUE. SETTINGS MARKED *learned* WERE DISCOVERED BY A DEFECT THEY CAUSED, NOT CHOSEN IN ADVANCE.

Setting	Value	Why
Kafka producer		
`acks`	`all`	Durability semantics matched to Redis's synchronous `XADD`; weaker acknowledgment would compare different guarantees.
`linger.ms`	0	No artificial batching delay; the workload is sparse, so any linger would dominate the quantity measured.
`max-inflight`	64	*Learned.* At the default 1 the client blocks on a round trip per event, making the comparison one of harnesses rather than brokers (the main text).
`compression`	none	Football events are small JSON; compression would add CPU to the path being timed for no representative benefit.
`ack-stamp`	`callback`	Default, and deliberately asymmetric with Redis; the main text measures what the asymmetry costs by also running `inline`.
Redis producer		
send workers	pool	Non-blocking dispatch so the emission loop keeps schedule; send and acknowledgment are timestamped in the worker, preserving a valid transport span.
Consumers		
Redis `ack-batch`	200	*Learned.* Acknowledging per message adds one network round trip per message, which at 20 ms of delay is the difference between 103 ms and 4,138 ms (the main text).
Redis `count`	200	Read batch; matched to the acknowledgment batch so neither dominates.
Kafka `poll-timeout`	1000 ms	Client already amortizes fetches; no per-record round trip exists to remove.
Replay		
speed-up	derived	*Learned.* Plans carry a baked-in $120\times$ compression, so the flag is not the rate; it is derived from the plan and verified against wall time before every campaign (the main text).
window	60–600 s	Swept deliberately: the number of events a run contains is itself a variable, and the main text is the reason.

S22. THIS HARNESS HAS DISCARDED LATENCY DATA BEFORE

Two silent losses that were bugs (moved from the main text): The admission condition is not the first time this harness has discarded latency data with nothing in its output to show it. A serialization defect found in 2023 truncated the histogram on long runs and lost most of the samples [S19]; a second defect — distinct, though the two interact in the repair's own account — reset each client's histogram on read during a retry, and was found only during "a statistical analysis on some results that looked strange" [S20]. Both were repaired. They are recorded here because they establish the contrast the main text draws: those were bugs, and the admission condition is not one – it was added deliberately, for a stated and locally sound reason, and it is still there.

The second failure mode is measured in the OpenMessaging Benchmark rather than in our harness, so the instrumentation is a patch to somebody else's code and its design needs stating.

What the patch does, and what it deliberately does not: At the named commit, a sample is admitted to the histogram only by `if (endToEndLatencyMicros > 0)`. We add counters in the existing `else` branch and change nothing else: not the latency computation, not the admission condition's condition, not any reported statistic. The benchmark's own output

is therefore exactly what it would have been unpatched, which is the property that lets us set its published summary beside the retention behind it. The diff is in the artifact.

The counters must distinguish sign, and an earlier version did not: Both a causality violation and a sub-millisecond delivery fail a `> 0` test. A single unsigned total cannot tell them apart, and we initially reported one, drawing a conclusion the sign-separated data later refuted (the main text). The counters are now four — zero, negative, most-negative, kept — and the distinction between them carries the finding.

Counts must be exact, which required two corrections: The counters are declared `static`: an embedded run constructs several `WorkerStats` instances, and per-instance counters silently split the totals between them. Totals are printed once by a JVM shutdown hook rather than read from periodic progress lines, because those fire every $10{,}000$ discards and reading the last one quantizes the answer to a multiple of $10{,}000$ — three runs with genuinely different totals all reported $50{,}000$ that way. Every cell records which source its counts came from, and cells whose counts are quantized are excluded from every analysis by rule rather than by inspection.

A validity gate that refuses to report a number: A benchmark that fails to run discards nothing, and a zero from such a run looks exactly like a measured zero. Our first distributed attempt wrote `discarded=0` after the worker died on a classpath fault, and that vacuous zero reached a draft of this paper as a genuine negative result. The campaign now validates before writing: the run must have produced publish-rate output and no failure lines, and if it did not, the row is written `valid=0` with the reason and no count. The gate has since refused eight distributed attempts (the main text), and its refusals are in the artifact alongside its acceptances.

Denominators: OMB warms up for one minute by default, resets its histograms at that boundary, and reports percentiles over the test phase alone; our counters are never reset and would otherwise span both. We make the setting explicit and report a matched sweep with warmup disabled. It changes the sample count per cell from $\approx 120{,}000$ to $\approx 90{,}000$ and leaves the retention fraction unchanged, so the figures computed with warmup included stand.

Campaign design: Retention at a fixed configuration proved not to have a stable central value (the main text), which dictates the design: every claim is made over whole distributions of replicates rather than over a summary of them, and the axes are swept one at a time with everything else held fixed — background load, message size, producer rate, and the warmup setting — because the variable that turned out to matter was one we did not initially think to hold fixed.

Every run is indexed, including the failures: Each cell is one row in a ledger built by reading the cell's own log rather than a receipt written afterwards, recording configuration, counts, whether the counts are exact, and the size and SHA-256 of the log they came from. This is not bookkeeping for its own sake: it recovered a valid cell whose script died between finishing the benchmark and writing its result row — a cell that is the median of its load level — and it exposed a fabricated negative sample that our own indexer had produced by mapping an older result schema positionally. A campaign that silently drops its failures is a selection of the runs that worked, which is the failure this paper is about, one level up.

The full record, stated once: The external campaign totals 223 instrumented runs in twelve named sub-campaigns: four load sweeps, a resolution sweep, a payload sweep at an incommensurate rate, a bimodality series ($n{=}18$ at one configuration), two rate-phase sweeps, the rate-independence control, a 63-cell comprehensive campaign (configurations to $n \geq 5$, three new denominators, a payload$\times$grid manipulation, a duration sweep, and a two-rate probe of the continuous value), and a distributed-mode series that never completed a run and is reported as exactly that. Six predictions for the comprehensive campaign were registered with falsifiers before it launched, and each interim reading was committed to the version-controlled record before the block that followed; three registered predictions failed and are reported beside the one that passed. From 22:02 UTC on the final night a once-per-minute log of the driver's clock discipline (residual frequency, skew, offset) runs alongside every cell, so each run joins to the clock state during it.

Verification of the manuscript itself: Every quantity this paper states from the campaign is recomputed from the committed ledger by an automated suite ($2{,}000$-plus checks, branch coverage of the analysis code at or above 95%), including every row of Tables the main text–the main text and every checkable sentence of the main text; run counts and discard totals are generated into the text from the ledger rather than typed, after an earlier draft was found quoting a count stale in nine places at once. The rendered PDF is checked separately from the source, which has caught defects the compiler passes. The suite is mutation-tested: corrupting a table value, or a value and its derived classification together, fails a check.

A validity gate refuses to report a number from a run it condemned: It refuses one from a run that did not demonstrably produce output — a benchmark that fails to run discards nothing, its zero looking exactly like a measured zero; the gate has refused eight distributed attempts, refusals archived beside its acceptances. Warmup is disabled and reported with a matched sweep (retention unchanged; samples $\approx 120{,}000 \rightarrow \approx 90{,}000$), axes are swept one at a time, and every run, failures included, is one ledger row recording configuration, sign-separated counts, and the log's size and SHA-256. Two counting defects we found and fixed, and what the ledger's indexing exposed, are supplementary material S22.

S22.1. Distributed mode never completed a run in eleven attempts

The cross-host case is the one in which OMB's subtraction spans two clocks, and therefore the one in which its admission condition could discard a genuine causality violation. **OMB's own distributed mode cannot report a measurement of**

it. Across eleven attempts — eight at three background-load levels including none at all, which refutes our own hypothesis that CPU starvation was the cause, and three further attempts run with full diagnostics for this revision — the benchmark's distributed mode never completed a run, failing each time inside its own coordinator-to-worker protocol rather than in our instrumentation or configuration. The validity gate wrote `valid=0` with a reason on every attempt; per-attempt versions, logs and failure signatures are archived in the artifact and summarized in supplementary material S24, and a draft upstream report accompanies them.

The exposure of OMB-in-distributed-mode is therefore established by source audit — the timestamp is written on the producer's host and read on the consumer's, and the admission condition drops whatever the difference produces — and *not* by observing that binary. What *is* now observed is the pattern itself across two hosts: the harness of the main text runs the identical timestamp-subtract-guard logic over two disciplined clocks across 12 runs, and measures 0 negatives in $905{,}040$ samples, an intact grid, and a deleted fraction coupled to clock-offset drift. The two statements are complementary and we keep them distinct: the pattern's cross-host behavior is measured; OMB's own distributed binary remains unmeasurable by us.

The library's own documentation does not describe the behavior the fix was written against: The 2018 rationale rests on `HdrHistogram` rejecting negative values [S21], and it does. What its documentation says is another matter. The `Recorder.recordValue` entry documents the exception it may throw "if value is exceeds `highestTrackableValue`" [S22] — the upper bound, and only that — and the project's README describes the configurable range and the significant digits without mentioning negative values at all. The rejection surfaces as an `ArrayIndexOutOfBoundsException` from the counts-array index rather than as a documented contract. That makes the maintainer's local fix more defensible rather than less: it was written against behavior the library documents for the opposite bound. It also makes the wider case, that a disposal rule has to be stated by whoever depends on it, because the dependency will not state it for you.

It has accepted a measurement-validity fix before, which is why the remedy is not a hopeful one: In March 2022 a contributor reported that the load generator did not record publish delay — the gap between an event's scheduled send instant and its actual one — arguing that "whatever prevent the load generation to work as expected should be recorded and weighted correctly [...] otherwise inability to keep-up with the expected load will pass unnoticed in the percentiles ocean" [S23]. The report was accepted and the delay is now recorded. Two things follow. The maintainers act on a measurement-validity argument when one is put to them, so the rules of Section VIII-B are addressed to a project that has taken this kind of correction before rather than to an indifferent one. And the admission condition came through that revision untouched: a review explicitly about what the harness fails to record left in place a filter that deletes what it did record. The two defects look nothing alike from inside the code, which is the argument of this paper in one repository's history.

The histograms it discards after reading them: In March 2026 a second contributor asked for the HDR histograms to be serialized into the result JSON, observing that the harness "already maintains HDR Histograms internally throughout each run and discards them after extracting the summary scalars", and that summary scalars alone cannot say whether a measured difference is significant "or within the noise floor of the measurement" [S24]. We record it because it is this paper's argument arriving from the other side, and because of what it would and would not fix. A serialized histogram supports every test the request names — Mann–Whitney, Kolmogorov–Smirnov, bootstrapped percentile intervals — computed over a population whose size is still not stated, because the admission condition runs before the histogram is filled. The export is worth having. It carries the deletion into a new container unless the retained fraction travels with it.

What we do not claim: That any published result obtained with this benchmark is wrong. We have shown that the deletion happens, that it is large, that it is irreproducible, and that it is unreported. Whether it has affected a specific published measurement depends on that measurement's path latency and producer rate, and we have audited none. That asymmetry is the reason we recommend publishing a retention rate as routine practice: its cost is one number, and the alternative is a benchmark that cannot report its own failure.

S23. Where the check transfers, and what it costs to run

The check is worth reporting only if it transfers, so we state it as a procedure rather than as a description of what we happened to do. It applies to any latency computed as a difference of timestamps taken in different processes, threads or hosts — which covers most end-to-end measurements in this literature.

Eight of the steps are the reporting rules the main text closes on, and they are stated there rather than twice: name the span you subtract, publish the rejection rate, publish the retained fraction beside the latency, know where your path sits on the exposure curve, count your events before quoting a percentile, and record the achieved rate rather than the flag meant to produce it (Section VIII-B). What follows is the four this document adds, each a check the main text has no room for.

1) **Read your own reported percentiles for quantization. This costs nothing and needs no patch.** If every percentile a benchmark prints is a whole multiple of its clock's resolution, the measurement has no resolution below that timestamp resolution and the distribution is an artifact of the grid. In the benchmark we audited, all 32 percentile values across eight runs were whole milliseconds, and three runs reported $\mathrm{p50} = \mathrm{p95} = \mathrm{p99} = \max = 1.0$ — a distribution with one value in it, printed to three decimal places, in output anybody could have read. We found it only after instrumenting the source, which was unnecessary.

2) **Put your send interval over the timestamp resolution, reduce the fraction, and look at the denominator.** A fixed interval that visits few phases within the timestamp resolution puts nearly every sample in a run at one of them, which makes the surviving fraction bimodal and irreproducible rather than merely small (in the main text). The diagnostic is arithmetic on two numbers you already know, but it is the reduced denominator q that matters, not whether the interval looks round: 500 msg/s gives $q = 1$ and spread 99.52 points, and 300 msg/s gives $q = 3$ and spread 31.23, while 889 msg/s — which sits 0.00014 away from $9/8$ — gives a denominator in the hundreds and is stable to 1.7. A small q is the hazard; proximity to a simple fraction is not. The remedy is to dither the send instant, which works without having to get the arithmetic right.
3) **Before attributing an offset to every event, vary the run length and count.** Lengthen the observation window by a known factor and count how many events exceed the offset, rather than re-computing the average. A per-run cost holds that count fixed while the run grows; a per-event one grows it in proportion. The two are indistinguishable in any percentile, which is why we report the count. In our sweep the count stayed at four while events per run grew several times over (supplementary material S3).
4) **Where retention differs between configurations, bound the selection it introduces.** Unequal rejection is itself a bias, and the main text gives the worst-case imputation we use.

The cost is one pass over data the harness already produces. In our implementation the rule is under two hundred lines and runs in seconds over $2{,}266$ runs, which is why we argue it belongs in routine practice rather than in a dedicated study.

When it matters most: Equation S2 of the main text tells a reader where to be careful without repeating our experiments: risk rises as the measured quantity approaches the harness's own scheduling jitter, and as utilization approaches saturation. A benchmark measuring milliseconds on a loaded machine is in the dangerous region; one measuring seconds, or running well below saturation, is not. That is a rule of thumb, and the main text says what would be needed to make it quantitative.

The check transfers to any latency computed across execution contexts: It reaches most end-to-end measurements in this literature. The load-bearing rules: (1) identify every span whose endpoints are timestamped by different execution contexts, and check those; (4) reject by a rule stated before looking at results, and report its sensitivity; (5) report the retention rate beside the results, counting rejection causes separately (negative = measurement failure; computes-to-zero = resolution failure); (8) put the send interval over the timestamp resolution, reduce the fraction, and read the denominator — small q is the hazard, dithering removes it; (10) record the achieved rate, per run, verified against elapsed wall time, next to the results — not the flag meant to produce it; (12) where retention differs between configurations, bound the selection. Rules (2), (3), (6), (7), (9) and (11) are stated, each with the incident that taught it, in supplementary material S23. The cost is one pass over data the harness already produces; our implementation is under two hundred lines and runs in seconds over $2{,}266$ runs, which is why we argue it belongs in routine practice rather than in a dedicated study.

S24. The campaign ledger, and what distributed mode did instead

One row per cell of every instrumented-benchmark campaign, in the ledger `external_campaigns_index.csv`: the swept axis and its level, a validity flag whose zero rows are kept with a reason rather than deleted, the source of each count, the instrumented guard's three sign-separated counters, and the size and SHA-256 of the log the row was built from, so any row can be re-derived from its source. The column-by-column dictionary is `docs/campaign_ledger_schema.md` in the artifact, beside the file it describes; the Python-harness campaigns use the same cell naming and publish `docs/results/external/harness_results.csv`.

S24.1. OpenMessaging distributed mode, failure diagnostics

Eleven attempts in total never produced a measurement. The three diagnosed attempts of the referee campaign (chain 18, block E) fail identically at OMB commit `5b1fa70`: `java.util.concurrent.CompletionException` wrapping `java.lang.IllegalArgumentException` in the coordinator–worker path, before any workload starts. Per attempt, the version, full coordinator and worker logs, and the extracted signature are archived in `docs/results/external/dist_diag/`; a draft upstream report is at `docs/omb_distributed_issue.md` (not filed — filing is the author's action).

The cross-host case is the one whose subtraction spans two clocks: There OMB's subtraction spans two of them, and its own distributed mode never completed a run in our hands — eleven attempts, all failing identically inside its coordinator-to-worker protocol, including with no background load at all, which refutes our own load-starvation hypothesis. Its cross-host exposure therefore rests on source audit, the pattern itself measured across two clocks in the main text (per-attempt record and draft upstream report: supplementary material S24). We do not claim any published result obtained with this benchmark is wrong: exposure depends on a deployment's path latency and producer rate, and we audited none — why we recommend publishing a retention rate as routine practice, at the cost of one number.

S24.2. The statistical inventory

Section IV-F keeps the choices a reader might contest — Wilson over Wald, Holm within a named family, and why a tail index on interval-censored buckets is not a slope. This is the rest of the inventory, so that every interval in either document has a named method behind it.

Every ratio between two proportions carries a two-proportion z with pooled variance, and we state whether the intervals are disjoint. Location differences use Hodges–Lehmann shifts with distribution-free intervals and rank-sum tests; equivalence claims use TOST with the margin stated and the interval printed; Holm correction is applied within a named family, and the corrected value is the one the tables display; percentile bootstrap intervals cover statistics with no closed form. The grid-membership inference uses a permutation null, described where it is used. Regression intervals are Student-t at $n-2$ degrees of freedom. Tail indices are estimated on the traced histogram by grouped maximum likelihood with a profile-likelihood interval and by an exceedance ratio with a Wilson interval, because the buckets are interval-censored and a slope through survival points is not an estimator; the grouped estimator on log-spaced bins is Virkar and Clauset's, which is the Hill estimator for binned data, and we use their bootstrap for goodness of fit [S25].

All interval arithmetic is recomputed from the committed artifacts at build time by the same code that emits the numbers into the text, so no figure in either document is transcribed.

S24.3. The metric map

This section says which interval each reported quantity actually covers. The drawing is the main text's Figure 1, which names the four bracketing timestamps; what belongs here is the mapping from those timestamps onto the metric names, because a reader comparing our numbers against another harness's needs to know what was subtracted from what.

Scheduling lag is $t_{\mathrm{send}} - t_{\mathrm{sched}}$. Both timestamps are taken inside the producer process, so it is a causal chain and cannot invert. *Acknowledgment lag*, $A = t_{\mathrm{ack}} - t_{\mathrm{send}}$, is also producer-internal and also cannot. *Broker transport* is the transport proxy $S = t_{\mathrm{recv}} - t_{\mathrm{ack}}$: it subtracts a producer-process timestamp from a consumer-process one, which is exactly why it can come out negative when either process is delayed, and it is the span this paper is about. *End-to-end TTI* is $t_{\mathrm{recv}} - t_{\mathrm{sched}}$, the sum of the three (Equation 2), and it inverts only if the whole delivery does. A fifth timestamp, t_{out}, is consumer-internal and is not drawn in that figure; S16.1 places it.

S25. The evidence behind the count of tools that dispose silently

The main text says how many independent tools dispose of an inverted or sub-resolution sample without counting the disposal. This section is the evidence behind that count. Each row of `docs/results/external/harness_registry.csv` carries one source line, the file and symbol it came from, the upstream URL and the date it was read. The class labels are not stored in the file: they are recomputed on every build, and on every test run, by the same classifier that read the OpenMessaging Benchmark a month before the other tools were audited. A reader who disagrees with our reading of a guard can therefore check the line against the pattern that matched it rather than against our prose. Table S14 is the fold of those rows up to one verdict per *span*, which is the grain the finding lives at: one tool can guard one span and leave another alone, and the OpenMessaging Benchmark does.

TABLE S14

Audited harnesses, one row per span rather than per tool. *Filter* admits only positive samples; *substitute* replaces an inverted interval with a constant; *refused by own library* means the recording library returns a refusal the caller never checks; *counts* means the disposal is exposed in the output. The OpenMessaging Benchmark appears twice, because it treats its two spans differently and that difference is the finding of Section VI-A. Sources, in row order: [S8], [S8], [S26], [S26]–[S34]. This table and this list are generated from `harness_registry.csv` at build time.

Tool	Span measured	Clock	What happens to the value	Counts
Apache Kafka (bundled)	end-to-end	nanosecond	truncate to the quantum	no
Apache Kafka (bundled)	publish	millisecond	truncate to the quantum	no
Apache Pulsar perf	end-to-end	millisecond	filter	no
NATS CLI	end-to-end	nanosecond	—	no
OpenMessaging Bench.	end-to-end	millisecond	filter, truncate to the quantum	no
OpenMessaging Bench.	publish	nanosecond	—	no
RabbitMQ PerfTest	end-to-end	nanosecond	—	no
Rezolus	scheduler wakeup	nanosecond	—	yes
`blktrace/btt`	block stage	nanosecond	substitute a constant	no
`emqtt-bench`	end-to-end	millisecond	—	no
`fio`	I/O completion	nanosecond	substitute a constant	no
wrk2	end-to-end	microsecond	refused by own library	no

The most interesting row is the one carrying no defect at all. The OpenMessaging Benchmark's publish span is taken inside one process, timestamped with a nanosecond clock and recorded unconditionally; its end-to-end span crosses processes, is timestamped in milliseconds, and is filtered. The coarse clock and the admission condition both landed on the span that needed care — the asymmetry Section VI-A draws its conclusion from, and the reason this table folds by span.

What the round-56 additions changed, including a category the survey did not have: Four harnesses were added after a co-author asked what the audit had *not* looked at, and they changed the shape of the finding rather than adding to a tally.

Apache Kafka's own bundled tools were the gap that mattered, because they ship inside every distribution and are therefore installed wherever the broker is. `EndToEndLatency` is immune to the scheduling failure by construction — one thread, one monotonic clock, a synchronous send, so no negative span is possible — and it loses the measurement anyway:

```
System.out.println(i + "\t" + elapsed / 1000.0 / 1000.0);
latencies[i] = elapsed / 1000 / 1000;
```

The printed line is floating point and correct; the `long[]` the percentiles are read from is filled by integer division. Every sample is kept, nothing is filtered, and on a sub-millisecond path `p50`, `p99` and `p999` all report zero. The companion `ProducerPerformance` takes its send-to-acknowledgment difference on `System.currentTimeMillis()` and casts it to `int`.

This is what forced a fifth class into the classifier. Until then its categories were three ways a sample can *vanish*, and a survey whose only question is "what happens to a negative" cannot see a tool that keeps every positive and records it as zero. `quantized_retention` matches a division or cast that discards the unit the value was measured in, and applying it retroactively labels the OpenMessaging end-to-end line too — `MILLISECONDS.toMicros(now - publishTimestamp)`, where the microseconds are decoration on a millisecond subtraction. The deletion failure is therefore not a claim about a filter. It is a claim about a timestamp resolution, and deletion is one of the ways it collects.

The other two additions are counterexamples, and they are in the table for the reason Rezolus is. RabbitMQ PerfTest differences a timestamp carried in the message against a local clock and passes the result to `performanceMetrics.received(diff_time)` with no sign test at all; the NATS CLI appends `time.Duration(time.Now().UnixNano()-sendTime)` unconditionally. Both would let a negative through into their metrics, which is what RFC 7679 asks of a measurement, and both are exposed to skew rather than to quantization. *The field does not uniformly delete*, and the paper must not be read as saying so.

Two tools were looked at and found not to make the measurement at all. SPECjms2007, the only messaging benchmark from a standards body, reports throughput (`SPECjms2007@Vertical` and `@Horizontal`); it defines Delivery Time as "the elapsed time measured between sending a specific message and that message being received" and never reports it. Redpanda's `rpk` ships no latency benchmark, its published comparisons being OpenMessaging runs. Neither is evidence for or against the failure, and both are recorded so that a reader asking "did you check X" gets an answer rather than a silence.

Two further details are worth stating plainly. First, `fio`'s guard appears in three separate timing functions, each returning zero for a negative interval, and the comment above it — "time warp bug on some kernels?" — is a question rather than a diagnosis. The behavior is defensive and reasonable in isolation; what is missing is any record that it fired. Second, `btt`'s `tdelta()` returns one nanosecond whenever the stage timestamps are out of order, with no comment, no counter and no mention in the manual page. A substitution of this kind is harder to detect than a filter, because the sample count is unchanged: any retention rate computed downstream reads 100%.

Fourth, `emqtt-bench` is in the table as a correction of our own reading rather than as a finding. Its `> 0` test guards a counter, not the histogram, so nothing is deleted from the sample; we had first read it as a filter. A guard is only a deletion if what it guards is the sample, and that distinction is now what the classifier encodes.

Third, `wrk2`'s disposal was mis-evidenced in an earlier draft of this registry, and the correction is worth spelling out because the wrong version survived one round of review. `wrk.c` detects the negative interval, prints a block that begins "We are about to crash and die", and records the value anyway with the boolean return unchecked. We first attributed the resulting drop to upstream HdrHistogram_c's value guard (`if (value < 0 ...) return false`) [S35]. `wrk2` does not build against upstream: it vendors its own `hdr_histogram.c`, which has no value guard at all — it bounds-checks the *derived* counts index (lines 219 and 234) and returns an unchecked `false`, and its index computation carries `assert()`s that are live in the shipped build (`-O2`, no `-DNDEBUG`), so a negative may abort the run before the refusal is reached. Whether the assertion trips or the bounds check refuses, the sample is never recorded and nothing counts the loss — and the panic block's own prediction turns out to be the accurate part. The registry row now cites the vendored file, and the classifier's refusal pattern matches both spellings of the same disposal.

Two patterns had to be added to the classifier before it could see these tools, and both additions are recorded here because they bound what the survey can claim. The original patterns were written against JVM sources and could not match Erlang's guard, which is a short-circuit rather than an `if`, nor Erlang's clock read, which carries its unit as an argument. The survey therefore reports what the classifier can currently recognize, which is a lower bound on the field rather than a census of it. The complementary lineage is behavioral: Mittal evaluates six open-loop load generators for rate fidelity and measurement skew without reading their source [S36], and the two methods find disjoint defect classes, which is the argument for doing both.

The pattern has since reached the audited broker itself. In November 2025, Kafka's coordinator metrics were found throwing on negative durations when a host clock stepped backwards, and the merged fix clamps the negative to zero with no counter and no log line [S37] — silent substitution, the class this table marks *substitute*, adopted in the very system whose benchmark opened this audit. The main text names the case and its citation in one sentence and sends the reader here for the rest, because

TABLE S15

REPLICATE SPREAD AGAINST THE PHASE DENOMINATOR q (MOVED FROM SECTION VI-B), AT FIXED PAYLOAD, LOAD, DURATION AND HOST. *Pos.* IS THE DISTANCE FROM THE MEASURED CONTINUOUS VALUE TO THE NEAREST GRID POINT, AGAINST THE CELL HALF-WIDTH: 0 SITS ON A GRID POINT, 1 MIDWAY BETWEEN TWO. POSITION IS SCALE-FREE, SO EVERY CALL IS UNCHANGED WHETHER THE CONTINUOUS VALUE IS ESTIMATED POOLED OR PER RATE. *Predicts* IS *full* WHERE THE POSITION EXCEEDS A HALF; *shows* IS *full* WHERE THE MEASURED SPREAD EXCEEDS HALF THE CELL WIDTH. BOTH ARE RECOMPUTED FROM THE GEOMETRY AT BUILD TIME, SO THE LAST TWO COLUMNS CAN BE READ AGAINST EACH OTHER: 10 OF THE 12 COMMENSURATE ROWS AGREE, AND THE TWO THAT DO NOT ARE 700 AND 1250 MSG/S, DAGGERED AND SET IN BOLD HERE AND DISCUSSED IN SUPPLEMENTARY MATERIAL S10. BOTH ARE RESOLVED IN THE MODEL'S FAVOR BY THE REPLACEMENT STATISTIC OF TABLE S7, WHICH DOES NOT USE SPREAD. INCOMMENSURATE CONFIGURATIONS HAVE NO CELL AND THEREFORE NO CLASS: THEIR *predicts* ENTRY IS THE NOISE FLOOR RATHER THAN A PREDICTION, AND THEIR MEASURED SPREADS — ABOUT THREE POINTS — ARE THAT FLOOR, NOT A DISAGREEMENT. ROWS ARE ORDERED BY q, WHICH IS WHAT DECIDES THE PREDICTION. EVERY INCOMMENSURATE RATIO SHOWN IS ALREADY IN LOWEST TERMS.

rate	Δ/τ	q	n	width	pos.	predicts	spread	shows
1000/s	1/1	1	5	100.0	0.98	full	99.4	full
500/s	2/1	1	5	100.0	0.98	full	99.5	full
250/s	4/1	1	5	100.0	0.98	full	98.7	full
400/s	5/2	2	5	50.0	0.04	flat	17.6	flat
600/s	5/3	3	6	33.3	0.95	full	32.7	full
300/s	10/3	3	10	33.3	0.95	full	31.2	full
800/s	5/4	4	5	25.0	0.07	flat	7.2	flat
1250/s†	4/5	5	5	20.0	0.91	full	7.6	**flat**
625/s	8/5	5	10	20.0	0.91	full	19.3	full
875/s	8/7	7	5	14.3	0.87	full	10.7	full
700/s†	10/7	7	5	14.3	0.87	full	6.5	**flat**
900/s	10/9	9	5	11.1	0.84	full	6.9	full
incommensurate — phase set effectively continuous								
333/s	1000/333	>64	5	—	—	≈ 0	4.5	—
383/s	1000/383	>64	5	—	—	≈ 0	2.8	—
457/s	1000/457	>64	5	—	—	≈ 0	2.1	—
611/s	1000/611	>64	5	—	—	≈ 0	3.1	—
889/s	1000/889	>64	5	—	—	≈ 0	1.7	—
1053/s	1000/1053	>64	4	—	—	≈ 0	3.0	—
1219/s	1000/1219	>64	4	—	—	≈ 0	4.5	—

TABLE S16

RETENTION AGAINST THE PRODUCER'S SEND INTERVAL, WITH MESSAGE SIZE, BACKGROUND LOAD, DURATION AND HOST HELD FIXED. ONLY THE COMMENSURATE RATE IS UNSTABLE, AND THE TWO INCOMMENSURATE RATES AGREE WITH EACH OTHER DESPITE INTERVALS DIFFERING BY 19%. PREDICTIONS WERE RECORDED BEFORE THE RUNS.

Rate	Interval	Multiple of tick?	Retention per replicate	Spread
500/s	2.000 ms	**exact**	0.47, 1.51, 1.55, 18.02, 99.99	99.5
457/s	2.188 ms	no	48.77, 49.50, 49.69, 50.51, 50.87	2.1
383/s	2.611 ms	no	50.28, 51.09, 51.45, 51.77, 53.13	2.8

its mechanism is genuine wall-clock non-monotonicity, and setting it out beside the scheduling failure would invite exactly the clock-versus-stall conflation the paper separates.

S26. TABLES

Table S15 is the per-configuration ledger behind Section VI-B's grid claim: one row per send rate, with the arithmetic denominator q, the replicate count, and the spread those replicates actually showed. The main text keeps the inference drawn from it and the set view of the same configurations; this is the record they are drawn from, and every cell in it is recomputed from the committed campaign index by the test suite rather than proofread. Table S16 keeps the individual replicate retentions for three configurations, which the ledger reduces to a single spread: it is the same evidence at finer grain, and it is where the 0.47-to-99.99 range behind the $q = 1$ corner can actually be seen.

Two further tables hold evidence the main text spends a line on each. Table S17 sweeps payload at an incommensurate rate, where samples stay dephased and implied latency can be read off directly. Table S18 is the load sweep behind the queueing form we withdrew; it is kept because a fit we abandoned should stay inspectable by anyone who wants to disagree with the abandonment. A third answers the question Section VIII-A raises about unequal audit loss: Table S19 bounds the selection effect that loss could have introduced into the broker equivalence, by imputing every rejected Redis run at the largest value its own cell allows. Its margin is the one that equivalence is stated against.

S26.1. The mechanism campaigns' tables

Table S20 is the mixture across nine load levels; Table S21 the occupancy manipulation; Table S22 the load-geometry comparison with its replication; Table S23 the payload sweep; and Table S24 the kernel-trace comparison, including the

TABLE S17
RETENTION AGAINST PAYLOAD AT 457 MSG/S, INCOMMENSURATE WITH THE MILLISECOND GRID SO SAMPLES STAY DEPHASED. IMPLIED T_{true} INVERTS THE MAIN TEXT. THE FINAL COLUMN IS THE INCREMENT IN IMPLIED LATENCY PER BYTE OF ADDED PAYLOAD, RELATIVE TO THE 200 B BASELINE; THE TWO SMALLEST STEPS ARE NOISE-DOMINATED AND MARKED.

Payload	Retention (3 reps)	Median	Implied T_{true}	ΔT/byte
200 B	51.08, 52.03, 52.88	52.03%	0.520 ms	baseline
1 KB	50.11, 50.32, 51.47	50.32%	0.503 ms	$-2.600^{\dagger}$
2 KB	52.13, 54.62, 54.77	54.62%	0.546 ms	$1.752^{\dagger}$
4 KB	51.86, 54.17, 56.24	54.17%	0.542 ms	$0.686^{\dagger}$
8 KB	54.92, 55.22, 57.89	55.22%	0.552 ms	$0.499^{\dagger}$
32 KB	68.01, 68.44, 69.40	68.44%	0.684 ms	**0.630**
64 KB	84.21, 85.36, 86.82	85.36%	0.854 ms	**0.638**

† added serialization under 0.07 ms, comparable to the 0.02–0.04 ms replicate noise, so the ratio is scatter over a small denominator. The two unmarked levels have a signal-to-noise ratio near 20 and carry the result.

TABLE S18
H2: NEGATIVE-SPAN RATE AGAINST ACHIEVED CPU UTILIZATION, NO CORE PINNING, BACKGROUND LOAD SWEPT FROM IDLE TO OVERSUBSCRIPTION. THE WITHDRAWN QUEUEING FORM FITS AT $R^2 = 0.945$ AGAINST 0.640 FOR A STRAIGHT LINE.

Utilization ρ	Negative span rate
0.003	0.007
0.253	0.009
0.504	0.022
0.628	0.047
0.753	0.132
0.878	0.207
1.000	0.208
1.000	0.221
1.000	0.264

withheld configuration.

S26.2. The audit, timestamping-mode and injected-delay tables

Table S25 is the consistency audit of every run; Table S26 the two timestamping modes; Table S27 the injected one-way delay.

Magnitudes of this size cannot be a per-event cost: The account we test is a round-trip-bound consumption pattern: our Redis consumer acknowledges each entry with `XACK` and waits — the idiomatic tutorial pattern — so a read of two hundred entries is followed by two hundred sequential round trips, free at 0.22 ms per trip and hopeless at 20 ms, where service rate falls below arrival rate and delay is set by backlog [S38]; Kafka's consumer serves from a prefetched buffer and commits on a timer, no per-record round trip to expose.

The falsification criterion was stated in advance: if amortizing the acknowledgments does not help, the account is wrong. At one feed under true real-time replay, batching them in groups of two hundred takes median TTI from 4,138 ms to 103 ms — a factor of **40.2**, replicated four times — and read-loop instrumentation corrects the shape: each `XREADGROUP` returns a full batch (median 106 messages in ∼32 ms), so the constraint is entirely in the acknowledgment path.

PART IV. THE LITERATURE, AND WHAT IT ALREADY REQUIRES

This part places the result. Sections S27 and S28 locate our irreproducibility in the variability literature, and the deletion failure against a resolution literature that knew the problem without reaching this. Sections S29 to S32 cover the second clock, where other runtimes write the timestamp, the rules neighboring communities already impose, and what standardized benchmarks require about resolution. Section S33 states where our reading parts from concurrent work.

S27. WHERE OUR IRREPRODUCIBILITY SITS IN THE VARIABILITY LITERATURE

Run-to-run instability has a supporting literature, and it is on our side: The irreproducibility we measure — three-replicate medians reproducing at one load level in five, and the distribution *family* itself moving between passes of an identical configuration (the main text) — sits in a line of work showing that benchmark variability is structural, not noise. Mytkowicz et al. [S39] showed that factors as innocuous as link order and environment-variable size flip published conclusions; Georges et al. [S40] that JVM results without confidence intervals routinely mislead; Barrett et al. [S41] that virtual machines frequently never reach the steady state their benchmarks assume, so the distribution being sampled is itself non-stationary — the closest

TABLE S19
BOUNDING THE UNEQUAL-RETENTION SELECTION EFFECT. "WORST CASE" IMPUTES EVERY REJECTED REDIS RUN AT THE LARGEST VALUE OBSERVED IN THAT CELL. MARGIN 1 MS.

N	Redis retained	Shift (ms)	Worst case	Equivalent
1	8/8	$+0.081$	$+0.081$	yes
9	20/27	$+0.021$	-0.051	yes
10	17/30	$+0.116$	-0.485	yes
12	19/36	$+0.053$	-0.227	yes

TABLE S20
MIXTURE, MEASURED ACROSS NINE LOAD LEVELS (25 RUNS EACH, PRE-REGISTERED). THE CORE WIDTH $\sigma_{\mathrm{core}} = \mathrm{IQR}/1.349$ GROWS $5\times$ FROM IDLE TO THE KNEE; THE NEGATIVE-SPAN RATE — THE TAIL — GROWS $60\times$. NEGATIVE SPANS ARE TEMPORALLY CLUSTERED AT EVERY LOAD (RUNS-TEST z). THE THREE $\rho = 1$ ROWS ARE DISTINCT OVERSUBSCRIPTION LEVELS THAT THE UTILIZATION COORDINATE CANNOT SEPARATE.

Utilization ρ	Core σ_{core} (ms)	Negative span rate	Clustering z
0.003	0.126	0.004	-6.8
0.253	0.138	0.004	-6.8
0.504	0.180	0.006	-6.8
0.628	0.181	0.024	-6.9
0.753	0.238	0.076	-5.7
0.878	0.631	0.224	-4.9
1.000	1.224	0.371	-4.3
1.000	1.293	0.317	-4.7
1.000	1.710	0.261	-4.4

published analogue of our regime mobility. Maricq et al. [S42] found hundreds of trials necessary for stable comparisons on real hardware, and Kalibera and Jones [S43] supply the replication framework. Against this stands common practice rather than any contrary finding: the surveyed norm of a handful of repetitions [S44], [S45] presumes a stable distribution that our phase-locked configurations demonstrably lack. We add the mechanism: at these configurations the instability is not variance around a value but a two-point support whose weights move, so no replicate count converges (the main text).

Result by result: For each headline result, the nearest support and the nearest constraint we can find:

- *Deletion law and grid structure* (in the main text). Supported mathematically by equidistribution and dither theory [S46]–[S48], and practiced in profiling since randomized sampling clocks were introduced to break exactly this kind of synchronization [S49]; no empirical study reports a benchmark's retained fraction, so the nearest contradiction is implicit — every published OMB-derived latency figure [S8] assumes retention near 100%, an assumption our 0.36%–100% range refutes for sub-tick paths.
- *Summary insensitivity* (the main text). Supported in spirit by Ousterhout's injunction to measure beneath the summary [S50] and by reporting standards that require sample counts [S45]; contradicted by no study, but silently contradicted by every report of a latency percentile without the count behind it.
- *Irreproducibility and regime mobility* (the main text, the main text). Supported by [S39]–[S42]; constrained by the same literature's remedy — more replicates — which our two-point supports show cannot converge here.
- *Scheduling-stall negative spans* (in the main text). Supported by the tail-latency and scheduler literature [S51]–[S53] and by the concurrent negative-span reports [S14]; qualified by Sharma et al.'s skew-threshold reading, which our zero-skew negative spans show is unsafe under load.
- *The sign audit* (the main text). Ten million discarded samples, not one negative. Nothing in the literature contradicts a count; what it refuted was our own prior claim, which is the point of publishing it.

Li et al. [S51] decompose the tail latency of three Linux servers into hardware, operating-system and application-level sources, and report the finding that matters most for us: measured tails are *substantially worse* than a queueing model alone predicts, with the excess traced to interference from background processes, request re-ordering under constrained concurrency, interrupt routing, power management and NUMA effects. A single-parameter queueing description is therefore a leading-order account, not a complete one — which is precisely what the main text measures directly on the harness rather than on the system under test. Their background-interference mechanism also predicts the temporal signature we observe: a core blocked by another task delays not one event but every event arriving in that interval, so the resulting failures should appear in consecutive bursts. The main text tests that prediction and finds exactly this clustering.

Our contribution here is therefore not the observation that scheduling delay is heavy-tailed, which is established, but that this shape propagates into the *measurement* of an unrelated quantity, and that load moves the tail's weight far faster than the core's width (the main text). The practical consequence is specific: a benchmark can sit at a utilization where its measurements look tight — the core is narrow — while the tail that actually corrupts them is already growing.

TABLE S21

OCCUPANCY, NOT UTILIZATION. TIMESTAMPING PROCESSES AT ORDINARY PRIORITY VERSUS `SCHED_FIFO`, WITH THE BACKGROUND LOAD UNCHANGED. UTILIZATION IS MEASURED IN BOTH CONFIGURATIONS, NOT ASSUMED: IT MATCHES TO 0.001, SO THE MANIPULATION MOVED OCCUPANCY ALONE. THE NEGATIVE-SPAN RATE FALLS BY A FACTOR OF 39–54 TO THE RUNNING-STATE FLOOR. UTILIZATION-ONLY MODELS PREDICT NO CHANGE.

	Utilization ρ		Negative span rate		
Load	ordinary	real-time	ordinary	real-time	factor
75%	0.7531	0.7525	0.1320	0.0034	39×
88%	0.8809	0.8812	0.3049	0.0057	54×

TABLE S22

SAME UTILIZATION, DIFFERENT RATE. BOTH LOAD GEOMETRIES IN ONE CAMPAIGN. CONCENTRATED LOAD LEAVES $C - k$ CORES FREE; SPREAD LOAD DUTY-CYCLES ALL OF THEM AND LEAVES NONE. AT $k = 6$ THE TWO CONFIGURATIONS SIT AT THE SAME ρ TO FOUR DECIMALS — IN *both* CAMPAIGNS — AND THE RATES DIFFER TWOFOLD IN BOTH. EVERY CELL IS 2 985 MATCHED EVENTS OVER 25 RUNS; z IS A TWO-PROPORTION TEST BETWEEN THE CONFIGURATIONS OF THAT ROW. THE $k = 7$ ROW IS WHERE THE TWO CAMPAIGNS DISAGREE: THE ORIGINAL CANNOT SEPARATE THE GEOMETRIES THERE, THE REPLICATION CAN, AND WE WITHDRAW THE STRONGER READING (TEXT).

	E-A6 (original)			E-A6b (replication)			
k/C	conc.	spread	z	conc.	spread	z	ratios
5/8	0.0201	0.0379	4.09	0.0111	0.0509	8.89	1.88×, 4.61×
6/8	0.0824	0.1709	10.27	0.0600	0.1229	8.44	2.07×, 2.05×
7/8	0.2281	0.2415	1.22	0.1973	0.2342	3.46	1.06×, 1.19×

Achieved ρ, $k = 6$: 0.7531 in all four configurations. $k = 5$: 0.6284/0.6250 and 0.6284/0.6259. $k = 7$: 0.8778/0.8809 and 0.8778/0.8822.

The presumption that a ready thread runs is not ours: It is documented: Lozi et al. [S53] document Linux scheduler bugs under which cores stay idle *for seconds* while runnable threads queue elsewhere, invisible to conventional testing because throughput stays healthy. Under exact work conservation a timestamping thread would never wait while a core was free, and the geometry contrast of the table in supplementary material S26 would have to be flat; it is not, and it converges only where no core is free to migrate to.

Li et al. [S51] decompose Linux tail latency into hardware, OS and application sources, report measured tails *substantially worse* than a queueing model alone predicts — precisely the excess our harness measures — and their background-interference mechanism predicts the temporal signature we observe, consecutive corrupted events, tested and found in the main text (full engagement: supplementary material S27).

S28. THE RESOLUTION LITERATURE KNEW THE PROBLEM; THE DELETION FAILURE IS NEW

The problem is older than every tool in the audit: Danzig and Melvin were building microsecond timers for workstations in 1990, and said why in their title: the available clock was too coarse for the interval being timed [S54]. Nothing in this paper's The deletion failure is a new observation about clocks. What is new is that the tools which inherited the problem stopped counting what it costs them, and that the cost is computable in advance from two numbers an operator already knows.

No defence of the admission condition has been found, and we looked for one: The audit of Section VII finds the practice without ever finding a stated reason for it, which is a weaker position than finding a reason and rebutting it, so we record the search. Benchmark-framework papers that do treat validity systematically stop short of this particular threat: ComBench, a framework for communication benchmarking, devotes a threats section to validity but treats latency measurement only in passing [S55], and the same is true of the harness documentation surveyed above. We therefore claim only that we could not find a defence, not that none exists.

Coordinated omission is the closest neighbor, and is the mirror image: In Tene's *coordinated omission* [S56] the worst samples are never taken, so percentiles are optimistic by orders of magnitude; here the samples *are* taken, the latency *is* computed, and a downstream admission condition rejects the result — one loses the slow tail, the other the fast bulk. Both end with a confident summary over a biased subset, and a correction for one does nothing about the other.

Not hypothetical: the benchmark we instrument records into an `HdrHistogram` [S57], which ships explicit coordinated-omission machinery and none for this failure — its sound rejection of negative values is precisely what motivates the admission condition whose `else` branch silently deletes the sub-tick samples: a protection against one measurement artifact created the conditions for another.

The mathematics is a century old; the interaction is unreported: Weyl's equidistribution theorem [S46] and the three-distance theorem [S47] govern the irrational and rational regimes of Δ/τ; Schuchman's condition [S48], generalized by Wannamaker et al. [S58], is the remedy that the main text states as dithering. The timing literature treats a coarse tick as staleness, not deletion. What we claim as new, narrowly: not quantization — elementary, and Cloudprofiler [S59] exists because milliseconds are known to be too coarse — but the *interaction* of quantized timestamp, few-phase producer and admission

TABLE S23

SLOWER DELIVERY IS A MORE RELIABLE MEASUREMENT, TWICE. PAYLOAD PADDING LENGTHENS THE TRUE TRANSPORT WITHOUT TOUCHING THE SCHEDULER. UTILIZATION IS HELD TO A 0.012 SPREAD; THE SERIALIZATION COST PUSHES IT SLIGHTLY UP, AGAINST THE PREDICTED DIRECTION. BOTH QUANTITIES MOVE MONOTONICALLY, IN OPPOSITE DIRECTIONS, IN BOTH CAMPAIGNS: TRANSPORT SPANS 76.9× AND 76.8×, THE RATE FALLS 4.09× AND 4.26×. EACH CELL IS 2 985 EVENTS.

	E-A10		E-A10b		
Padding (B)	Transport (ms)	Negative span	Transport (ms)	Negative span	ρ span
0	0.701	0.2603	0.761	0.2338	0.881–0.882
4 096	2.381	0.1906	2.474	0.1652	0.882–0.883
65 536	14.638	0.0888	16.059	0.0784	0.885–0.885
262 144	53.898	0.0637	58.482	0.0549	0.893–0.895

TABLE S24

KERNEL TRACING PREDICTS THE NEGATIVE-SPAN RATE, UNFITTED, IN EVERY CONFIGURATION THAT HAS ONE. $\Pr[\text{STALL} > T_{\text{true}}]$ FROM SCHED_WAKEUP/SCHED_SWITCH, AGAINST THE NEGATIVE-SPAN RATE MEASURED FROM APPLICATION TIMESTAMPS IN THE *same* CELL; THE TWO SHARE NO PROBE AND NO ESTIMATOR. EVERY CELL IS 2 985 EVENTS. THE THREE ORDINARY CONFIGURATIONS AGREE WITH THEIR TRACED TAILS TO WITHIN ABOUT A THIRD, WITH NO CONSISTENT SIGN. EVERY REAL-TIME CONFIGURATION READS EXACTLY ZERO WHERE THE UNTRACED CONTROL OF THE E-A9 PAIR READS 15/2985; THAT IS A TRACING ARTIFACT, NOT A FLOOR (TEXT). THE E-A9B 88% CONFIGURATION IS *withheld* BY THE TRACER CHECK — 28% DRIFT AGAINST A 25% RULE FIXED IN ADVANCE — AND IS SHOWN BECAUSE A WITHHELD COMPARISON IS STILL A MEASUREMENT.

Campaign	Arm	Load	ρ	Traced events	$\Pr[\text{stall} > 0.5\,\text{ms}]$ / rate	Ratio
E-A9	ordinary	88%	0.8822	551,956	0.181 / 0.231	0.78
E-A9	real-time	88%	0.8819	570,591	0.018 / 0.000	—
E-A9b	ordinary	75%	0.7543	709,819	0.168 / 0.128	1.32
E-A9b	real-time	75%	0.7531	641,308	0.016 / 0.000	—
E-A9b	ordinary	88%	0.8825	687,986	0.207 / 0.196	*1.06*†
E-A9b	real-time	88%	0.8822	649,788	0.016 / 0.000	—
untraced control (E-A9's own first attempt, probe never attached)						
—	ordinary	88%	0.8812	—	— / 0.272	—
—	real-time	88%	0.8809	—	— / 0.005	—

† withheld by the tracer check; shown, not used.

condition, producing a retention fraction bimodal, irreproducible between identical runs, invisible in the reported summary, its damage set by the denominator of the interval-to-resolution ratio in lowest terms rather than by "round" rates.

Run-to-run instability has a supporting literature, and it is on our side: Mytkowicz et al. [S39], Georges et al. [S40], Barrett et al. [S41] and Maricq et al. [S42] establish that benchmark variability is structural, that factors as innocuous as link order flip published conclusions, and that hundreds of trials can be needed — against the surveyed norm of a handful of repetitions [S44], [S45]. Our phase-locked configurations supply the mechanism those studies lack: instability that is not variance around a value but a two-point support whose weights move, so no replicate count converges (the main text).

S28.1. The payload sweep's underpowered rows, and the comprehensive campaign's duration and plateau paragraphs (moved from the main text)

The sweep discriminates only where the manipulation exceeds the noise, and we say which levels those are. Replicate noise in the implied latency is 0.02–0.04 ms; at 32 and 64 KB the added serialization (0.26, 0.52 ms) exceeds it near twentyfold and the per-byte estimates agree, while below 8 KB it is comparable to the noise and the ratios scatter (-2.600, 1.752, 0.686, 0.499) — rows reported rather than dropped, because omitting them would make the table look more decisive than the experiment was. The law rests on the two largest levels; our first attempt sat entirely inside the underpowered regime and settled nothing, so the sweep was extended rather than reported.

Duration does nothing, which kills the natural alternative: A cumulative drift walk would scale with run length — registered: intermediates rise with duration. Measured over one, three and ten minutes at 500 msg/s: intermediate counts 1, 1, 0, branch values pinned to the vertices throughout (0.43, 1.11, 100.00 at ten minutes), probability at most $(1/3)^3 \approx 0.04$ under the walk's own reading. The walk is falsified; what stands is per-send jitter, applied at each send, not accumulated.

The continuous value plateaus, and one miss stands: Incommensurate configurations at 1053 and 1219 msg/s measured $\theta = 47.2\%$ and 48.2% — the downward trend below 889 msg/s does not continue, refuting the linear extrapolation one registered prediction relied on — and the 1250 msg/s configuration, predicted mid-cell and full, instead pinned on the 2/5 vertex (40.66–42.00, one at 48.26): probability ≈ 0.08 under the branch law, recorded as a miss — yet that configuration sits six to seven points below the continuous value beside it, which no continuous account produces: the grid is present even where the branch-probability law under-delivers. The payload sweep succeeded where co-location failed because padding adds serialization without contending for the scheduler.

TABLE S25
CONSISTENCY AUDIT OF EVERY RUN BEHIND A REPORTED RESULT. EVERY COUNT IS DERIVED AT BUILD TIME FROM `INTEGRITY_WINDOWS/CLOCK_INTEGRITY_BY_CONDITION.CSV` AND `INTEGRITY_BY_CONDITION.CSV`, THE AUDIT'S OWN OUTPUTS. A CONDITION IS USABLE ONLY IF ALL OF ITS RUNS SURVIVE. THIS CORPUS IS NOT THE SPAN RECOUNT OF THE MAIN TEXT'S TABLE I, WHICH COVERS EVERY CLOUD RUN THAT PRODUCED MATCHED EVENTS RATHER THAN EVERY RUN BEHIND A RESULT.

Corpus	Runs	Rejected	Conditions	Usable
A (single machine)	1,382	862 (62.4%)	76	8
B (four machines)	884	459 (51.9%)	40	13
Total	2,266	1,321 (58.3%)	116	21

TABLE S26
H3: MEDIAN BROKER TRANSPORT UNDER THE TWO TIMESTAMPING MODES, BOTH CONFIGURATIONS AT ONE IN-FLIGHT REQUEST UNDER MATCHED LOAD. `CALLBACK` IS THE DEFAULT ASYMMETRIC TIMESTAMPING MODE; `INLINE` TIMESTAMPS KAFKA ON THE CALLING THREAD AS REDIS ALREADY DOES. THE BETWEEN-SYSTEM GAP SHRINKS, AND THE SHRINKAGE IS ENTIRELY ON KAFKA'S SIDE.

Timestamp	Kafka (ms)	Redis (ms)	Difference (ms)
`callback` (asymmetric)	0.392	0.106	+0.286
`inline` (symmetric)	0.322	0.107	+0.215

S29. TWO NEIGHBORING LITERATURES, AND WHAT THEY DO ABOUT THE SECOND CLOCK

Two items belong here rather than in the main text, which sits at the journal's twelve-page limit and at its cap of forty-five references. Both are relevant to Section VIII-C, and neither changes a result.

S29.1. "One-way latency" that is half a round trip

The most common way to avoid the second clock is to use only the first, by measuring a round trip and halving it. Three widely used benchmarks do this, and we checked each against its own source and documentation rather than against custom:

- **OSU Micro-Benchmarks** [S60]. `osu_latency.c` computes `(t_total * 1e6) / (2.0 * iterations)`, and the documentation reports "average one-way latency". The halving is visible only in the source.
- **NetPIPE** [S61]. The documentation is explicit — "[l]atencies are calculated by dividing the round trip time in half for small messages" — and the source comment asserts that the halved value "is now the 1-directional transmission time".
- **Intel MPI Benchmarks** [S62]. The user guide states "[r]eported timings — time $= \Delta t/2$" for `PingPong`, against `PingPing`, which reports the unhalved time. The guide does not call the result a one-way latency.

The three differ in what they disclose — one claims one-way without saying it halves, one says both, one says it halves without claiming one-way — but none of them states the assumption the halving requires, which is that the two directions are equal. That assumption is a strong one on a cloud path, and it is the assumption our own transport proxy declines to make (Section II-A). We raise it not as a defect in those tools, whose paths are typically symmetric by construction, but because halving is the standard alternative to the subtraction this paper is about, and its cost is unstated wherever we looked.

S29.2. Drift-aware synchronization in MPI benchmarking

Hunold and Carpen-Amarie give the most careful treatment of clocks in the benchmarking literature we found [S63]. They criticize earlier MPI benchmarks for correcting the clock offset only, when node clocks also drift, and adopt a drift-aware synchronization that fits a linear model per process; the duration of a collective is then the difference between the maximum finishing time and the minimum starting time over all processes, taken against that corrected global clock.

The contrast with our setting is the useful part. They remove the second clock by building a better one, and then subtract global timestamps as though a single clock had produced them. That is the right move for their measurement, and it is unavailable for ours in two ways. Their correction is fitted offline over a whole run, whereas a benchmark reporting a latency per message has no such luxury; and a fitted global clock still leaves a residual offset on any individual interval, for which they give no error model, because they never consider the case where such a difference comes out negative. The word does not appear in their manuscript. Our reading is that this is not an oversight but a consequence of scale: at the millisecond durations of a collective over hundreds of processes, a residual of a few microseconds cannot invert the sign, and the question does not arise. It arises here because the interval is smaller than the timestamp resolution.

S30. WHERE THE TIMESTAMP IS WRITTEN, IN OTHER RUNTIMES

Where it is written instead, and by whom: The mechanism of Section V needs a timestamp taken by a thread that can be preempted, which is why Section VIII-D makes no claim about designs that pin a core or stamp in hardware. A third design is now in production and is worth naming because it is the closest live work on host latency: *netstacklat* stamps at kernel

TABLE S27
INJECTED ONE-WAY BROKER DELAY, APPLIED IDENTICALLY TO BOTH SYSTEMS, FIVE FEEDS. MEDIANS. THE ZERO-DELAY ROW IS REJECTED IN BOTH CONFIGURATIONS AND MARKS THE ABSENCE OF A USABLE BASELINE.

Delay	Kafka TTI	Kafka transp.	Redis TTI	Redis transp.
0 ms		*rejected*		
5 ms	12.4 ms	5.6 ms	4,651 ms	4,645 ms
20 ms	77.8 ms	20.8 ms	31,401 ms	31,268 ms
50 ms	336.6 ms	44.9 ms	103,143 ms	102,460 ms

hook points and reads those timestamps from eBPF, measuring how long a packet waits in the Linux network stack before an application reads it [S64]. Nothing it subtracts is taken by a user-space thread waiting for a core, so the failure this paper reports cannot reach it — corroboration for the scope rather than a counterexample to the mechanism.

Two of its habits are the rules of Section VIII-B, arrived at independently and in a different community. It states the resolution error its histogram imposes rather than leaving a reader to infer it from the bucket edges, and prices the finer alternative it did not take. And it bounds its own observer effect, where this paper measures its probe cost once (Section V). A monitoring tool that publishes both without being asked is the better evidence that the rules cost little, and it is not ours.

And a proof that placement is the thing that decides: Where the timestamp is written is a design choice in this section and an empirical finding in Section VIII-B — our own two clients differ by a payload parse, worth 19.5 µs. A 2026 result from concurrency theory says it is neither: it is a limit. Rodríguez, Castañeda and Piña formalize the *Causal Observability Problem* — assign timestamps to the boundary events of a concurrent execution so they reflect real-time order — and prove that no solution at the observable boundary is both complete, never missing a genuine precedence, and sound, never reporting a spurious one [S65]. Which of the two a monitor gets is decided by where the instrumentation sits relative to the operation boundary: inside yields completeness, outside yields soundness. They state that the dichotomy holds "independently of the underlying timestamp mechanism".

The connection is structural rather than evidential, and it is worth saying which. Their setting is shared-memory concurrent objects and their instrument is a monitor object; they report neither failure mode studied here, and nothing in their proof is about resolution, preemption or brokers. What it supplies is the reason Section VIII-C gives for ordering the remedies as it does. A better clock does not help because the question was never the clock, and here that is a theorem in another setting rather than an observation in ours.

Section V argues that an acknowledgment timestamp is late because the thread that writes it must first be scheduled. That is not peculiar to a JVM benchmark, and three widely used runtimes document the same structure in their own terms. None of them frames it as a hazard, which is the point: the property is public, and the consequence for a cross-process subtraction is not drawn anywhere we could find.

The timestamp is taken when the application next asks for it: `librdkafka`, the client under the benchmark's Kafka driver, serves its delivery-report callback from `rd_kafka_poll()` and the other queue-serving calls [S66]. The callback therefore runs on the application's own thread, at whatever moment that thread next polls, rather than when the broker's acknowledgment arrives. Our δ_{ack} is exactly the interval between those two events.

The definition itself names the reap, not the completion: `fio` defines completion latency, for asynchronous ioengines, as running to "the time . . . when the I/O's completion was reaped by fio" [S67]. The definition is honest and the scope matters: for synchronous I/O the same documentation says only "to when it was completed". For the asynchronous case — the one used for latency work — a reaping timestamp is written into the definition of the metric.

Vendors document the device-side timestamp; one draws the consequence: AMD states, under `hipEventElapsedTime()`, that events "record their timestamp when they reach the head of the specified stream" and that "the time that the event recorded [sic] may be significantly after the host calls `hipEventRecord()`" [S68]. NVIDIA documents the same mechanism — "[t]he device will record a timestamp for the event when it reaches that event in the stream" [S69] — without drawing AMD's conclusion. We note the asymmetry rather than flatten it: both vendors describe a timestamp written at a moment the host does not choose, and only one says out loud how far from the host call it can be.

S31. NEIGHBORING RULES, AND WHAT THEY ALREADY REQUIRE

Two of this paper's recommendations have partial precedents, and the honest thing is to say where. Neither displaces the contribution, because neither is applied where a streaming benchmark reports a latency; but a referee is entitled to see them named.

Counting what was not measured: In the same summary block as its latency percentiles, `sockperf` prints counts of the messages it sent, received and skipped, together with a line counting dropped, duplicated and out-of-order messages, derived from sequence numbers [S70]. The counters are error counters rather than a retention rate, but the reader of that summary can see the denominator the percentiles were computed over, which is what Section VIII-B of the main text asks for.

The OpenTelemetry exponential-histogram format goes one step further and reserves a field for it. `zero_count` holds observations at or below a zero threshold, described in the specification as including "values that have been rounded to zero", and the total `count` is defined to include that bucket [S71]. We do not claim it as a general sub-resolution indicator, because it is not one: it captures unrepresentable mass only at the bottom of the scale. What it establishes is narrower and still useful — that a mainstream telemetry format already treats rounded-to-zero observations as data to be carried beside the total rather than dropped, which is precisely what the admission condition of Section VI of the main text does not do.

A prior sighting in a benchmark, blamed on clocks: Klenik and Pataricza report negative derived durations in a Hyperledger Fabric benchmark, call them a serious violation of event causality, and trace them to a peer clock drifting under a lightweight synchronization service [S72]. Their case is genuinely two-clock — the durations oscillate smoothly around zero, a drift signature, the same channel as our Redis $-1000\,\mu$s finding and not the scheduling failure — and their check is post-hoc data validation with no per-run gate and no campaign rate. It is the nearest prior sighting of the observable inside a benchmark, and the attribution went where the field's default sends it: to the clock.

The strongest precedent for our own recommendation, in network measurement: Our rule says to keep the unusable sample and label it rather than delete it, and one system already does exactly that. TimeWeaver infers one-way delays from passively observed NTP traffic, and where earlier methods discarded a host outright for missing timestamps or negative latencies, it keeps those measurements and assigns them to lower accuracy tiers [S73]. That is a fourth disposition beyond the three Section VI of the main text finds in benchmark harnesses — not deletion, not substitution, not passing the value on unmarked, but retention with a quality label — and it is the one we would have asked for. It does not compete with this paper: it is network measurement rather than a benchmark harness, it derives no retention arithmetic, and it reads no harness source. We record it because a reader entitled to ask whether anyone has done the sensible thing deserves the answer, which is yes, in a neighboring field.

It is a precedent for our *first* rule as well as our third, and we had missed that. Making the case for tiers, the paper states what the filter it replaces cost: of the $8{,}804$ clients the prior method considered, about $3{,}631$ — two in five — "were rejected due to missing timestamps, negative latency values, and other reasons". That is a published rejection rate, which is what Section VIII-B of the main text asks a benchmark to report, arrived at in 2018 and for the same reason: a filter's cost is not visible in what survives it. Two qualifications, so the precedent is not claimed wider than it is. The denominator is clients rather than samples, and the rate is TimeWeaver's characterization of the prior method rather than of its own tiering, which it reports instead as counts per tier.

S32. What standardized benchmarks require about resolution

Section VIII-B of the main text asks that a harness state its timestamp resolution and count what it discards. Standardized benchmarking already contains rules of that form, and one of them is almost exactly the rule we propose. Setting them beside the flagship standardized messaging benchmark is the sharpest way to show the gap, so we do that here rather than assert it.

A rule of the right form, in embedded benchmarking: EEMBC's CoreMark-PRO run rules require that "[e]ach workload must run for at least 1000 times the minimum timer resolution" [S74]. That is the form of rule this paper argues for: a floor on the measured interval expressed *relative to the timestamp resolution*, not in absolute time. The distinction is easy to lose, and worth keeping — plain CoreMark sets an absolute ten-second floor instead, with no reference to resolution at all, and the two rules are not interchangeable.

A rule about unusable samples, in power measurement: SPEC's Power and Performance Methodology, clause 8.2.2, says the benchmark "should report the number of samples that may have a return value 'Unknown"' and the number whose uncertainty exceeds the maximum allowed [S75]. Two qualifications matter and we state them rather than round them off: the modal verb is *should*, in a methodology document rather than run rules, and the samples in question are unusable readings rather than samples excluded after the fact. Within those limits it is the same instinct — publish the count of what the timestamp could not measure.

Rules about the sign itself, in network measurement: Three standards already require some part of what the main text asks, and they are the closest precedents we found for treating an impossible value as data rather than as debris. OWAMP requires a packet timestamped in the future to be recorded unaltered within its timeout window [S76]; ETSI requires removed measurement cycles to be reported [S77]; and RFC 7679 treats negative singletons as usable distribution data [S78]. None of the three is ours and none is applied where a streaming benchmark reports a latency, which is the gap the main text names rather than fills for the first time.

And none of it, in standardized messaging: SPECjms2007 is the flagship standardized benchmark for message-oriented middleware. Its run rules record delivery time as a per-destination histogram whose default granularity is 100 ms and whose default upper bound is 10 s [S79], [S80]. Searching the run rules, design document and user's guide for a resolution clause, a minimum measurable interval, or a counter of discarded latency samples returns nothing: the only timing-quality requirement anywhere in them is clause 3.2's bound of 100 ms on *inter-node clock agreement*, which is a skew limit and not a resolution limit. The reported `deliveryTime` element carries an average, a maximum, a granularity and a ninetieth percentile, and no field for samples outside the histogram's range.

We are careful about one thing here. The user's guide does document sample discarding elsewhere — `trimInitialSamples` and `trimFinalSamples`, "this many samples are thrown away" — but those apply to throughput

histories rather than to delivery-time histograms, and they are configured constants rather than reported counts. It would be wrong to say the benchmark discards nothing; what it does not do is count the latency samples its own resolution makes unusable.

Two tools reached the same precondition independently, and both disclose rather than delete. `zrk`, a rewrite of `wrk2` begun in July 2026, states that at "the sub-millisecond latencies a fast service actually produces" a millisecond artifact "is frequently larger than the number being measured", and records the overshoot into the sample rather than discarding it [S81]. `wrk2` itself concedes in its own documentation that "measured latencies are only accurate to a $+/-$ ~1 msec granularity, due to OS sleep time behavior" [S82]. Neither counts a discard, because neither discards: the timestamp resolution is disclosed and the sample is kept. That is the contrast Section VI turns on.

The comparison is therefore not that standards bodies have failed to think about this. One of them states our rule in almost our words, for a different domain. The rule simply has not reached the place where brokers are compared on sub-millisecond paths.

The property those rules refer to is itself a measurement: A floor expressed relative to the timestamp resolution presumes the timestamp's resolution is known, and on a general-purpose platform it frequently is not: the granularity a runtime advertises, the granularity it achieves, and the cost of the call that reads it are three different numbers. TimerMeter quantifies exactly those properties for the timers a runtime actually exposes [S83], which is the measurement a harness must already have made before a rule of CoreMark-PRO's form can be applied to it. We name it because the rule of Section VIII-B has the same precondition: a harness cannot state its timestamp resolution honestly without having measured it, and the corpus here is a case where the advertised figure would have been the wrong one to state.

S33. Where our reading parts from the concurrent report

Section III-B states the disagreement in three sentences. This is the whole of it, because a dispute with concurrent work should be checkable rather than asserted.

Sharma et al. [S14] count negative timing spans in distributed inference systems and propose a `causality_health` check. Injecting skew, they observe no violations up to 3 ms and clear ones by 5 ms, and read that as a skew threshold. Their measurements we do not dispute; the reading we do, on one sentence: "queueing alone cannot produce negative timing spans".

Under the queue they model — backlog at a service stage — the sentence is true, and Section V refutes that account of our own corpus as well. It is false as written because two different queues are in play and only one of them is theirs. A message waits in a service backlog; the thread that timestamps the clock waits in the kernel's *run queue* for a CPU. The second wait is what displaces a timestamp, and it is unaffected by whether any backlog exists: our rates are highest at high utilization with no backlog at all.

The consequence is practical. If backlog cannot invert a span, then eliminating backlog leaves skew as the only explanation, and a millisecond threshold on skew becomes a sufficient check. On a loaded machine it is not: we see negative-span rates near 23% at 88% utilization on a single host — where the inter-host offset is not merely small but undefined, there being one clock — and move that rate fiftyfold with a scheduling parameter, touching no clock.

Their own formulation is nearer ours than the dichotomy allows. They note that the minimum separation "exhibits variability due to scheduling jitter", with violation probability rising as ϵ exceeds its quantiles. That is the negative-span law of Section II-A with skew written where we write the stall — the same law, attributed to the other term. What a threshold in milliseconds of skew cannot do is transfer to a machine where the stall, not the skew, is the term that grows.

S33.1. The survey behind the two related-work sentences

Section III-B and Section III-C state what this paper takes from each of these and what it adds. This is the survey behind those sentences.

S33.2. Production tracing reached the conclusion without the mechanism

The observable is familiar outside benchmarking. Distributed tracing systems ship clock-skew adjusters, which "modif[y] start time and log timestamps for spans that appear to be 'off' with respect to the parent span" [S84]. An issue titled *Clock Skew Adjuster considered harmful* objected that the result is "wrong data and really hard to diagnose" [S85], and Jaeger has defaulted the adjustment off since 2020.

A 2026 preprint reaches Villain's conclusion again on current hardware, pairing an edge inference runtime's own timing against a logic-analyzer reference and finding that deployment interference corrupts the timing infrastructure as well as the timing it reports [S86]. It reports no negative spans, no quantized deletion and no law relating either to the interval measured; what it confirms is the premise, fourteen years after Villain and in another domain.

That is our conclusion, reached by practitioners, from the symptom alone. What the argument lacked was a mechanism and a magnitude: if the displacement is a late timestamp rather than a skewed clock, then an adjuster is not correcting error but moving evidence, and how much evidence it moves is a measurable quantity nobody had measured. Section V measures it.

S33.3. A framework that does not have the problem

Section VIII-B claims the rules above are practical rather than aspirational, and the strongest evidence for that is not ours. In March 2026 Paul et al. published mq-bench, a framework for exactly the comparisons this paper is about, and evaluated eight brokers with it [S87]. Its measurement design makes five of the choices we recommend, and argues for none of them:

- **Native resolution.** "Each message includes a 24-byte binary header containing a 64-bit send timestamp in UNIX nanoseconds." There is no millisecond resolution, so the deletion law of Section VI has nothing to bind on.
- **A span that is a causal chain.** It computes $t_{\mathrm{recv}} - t_{\mathrm{send}}$, which is the span the main text's span table shows never inverts, rather than the acknowledgment-referenced proxy that does.
- **One clock.** "Clock synchronization between publisher and subscriber processes is ensured by running both on the same workload generator machine."
- **No guard.** We looked for a positivity filter, a clamp or a discard rule and found none.
- **A published loss fraction.** Its reliability experiment injects TCP resets and reports what did not arrive: "all brokers exhibit approximately 6.3–6.6% message loss" at QoS 0, or 1,150–1,200 of 18,200 messages, against 0% at QoS 1 and 2.

mq-bench then reports medians as low as 0.21 ms — squarely inside the range where both failures reported here bite — and needs no remedy, because it never acquired either defect.

The last of those bullets is the one we did not expect to find. Our rule asks for the retained fraction to be published beside the latency, and here it is, in a framework that never argues for the practice: the loss is induced rather than spontaneous, but the reporting is the point. Their first two experiments run at QoS 0, which is at-most-once and has no redelivery, so the headline latencies are drawn from a configuration that can silently drop — and the third experiment is what tells a reader how much. That is the shape the rule asks for, arrived at independently.

One thing it does not do is say why its span is safe. A reader cannot tell from the paper whether the send-referenced difference was chosen because it is a causal chain or because it was the convenient one to instrument, and that is exactly what the rule "name the span you are subtracting" asks an author to make explicit. The choice is right either way; the record of why is what would let the next framework inherit it.

The one choice it does explain is the one our own rule warns about. Publisher and subscriber are co-located, and the stated reason is that co-location "ensures" clock synchronization — which it does, and which is the single rival Section VIII-B says co-location removes. It carries them no further. Had the same framework referenced its span to an acknowledgment instead, one host would have left the scheduling failure exactly where it is, and the reason given for the safe configuration would not have been a reason for the measurement being safe. That is the distinction the rule is for, drawn here on a framework that does not need it.

S33.4. How large the literature is that misses this

A 2025 preprint synthesizes 42 peer-reviewed Kafka studies published between 2015 and 2025 and reports evaluation rigor, configuration disclosure and reproducibility inconsistent across them [S18]. It finds neither of the failures reported here. The field's own taxonomy has no slot for them either: a 2024 survey reviews 27 benchmark efforts across five dimensions — one of which is *tracked metrics* — and over 21 pages says *benchmark* 169 times and *latency* 33, while *timestamp*, *clock* and *resolution* occur never [S88]. It closes on the aspects it judges overlooked, and the instrument is not among them. The size matters for the same reason the audit of Section VII does: the instruments are few and shared, so a defect in one is a defect in a decade of comparisons rather than in one project.

The adjacent literatures are alive to benchmarking pitfalls without reaching this one. Fruth et al. catalogue the perils of database benchmarking [S89], Zhang et al. attribute tail latency to its sources through precise load testing [S90], and the scheduling-latency literature has compared measurement approaches across kernels for over a decade [S91]. Each takes the tool's own reading as given. That is not a criticism of any of them — it is the assumption this paper is about, and its being shared is the reason the omission survives.

Three 2026 data points, two of them inside the regime: The main text concedes that most published comparisons report a median above the timestamp resolution, where the deletion law does not bind. That concession stands, and it is worth recording that it is not universal. The denominator behind that record is small and is stated here rather than in the main text: of the three 2026 broker comparisons we placed against the resolution, two report figures at or below it. The third is an enterprise-scale Apache Pulsar benchmark whose 3.88 ms median sits well above the tick, so the arithmetic of Section VI never reaches it [S92] — a case for the concession rather than against it, and the record was assembled only from exceptions until it was looked for. It earns its row twice over: across four pages it names that median seven times, and the words *timestamp*, *clock*, *resolution* and *discard* do not occur in it. Each is one row of `docs/results/external/literature_regime.csv`, and the count in this sentence is read from that file rather than typed. A 2026 practitioner comparison of NATS, Redis and Kafka gives Kafka "sub-10ms latency at p99" and gives the other two no percentile at all: Redis has "sub-millisecond response times" and NATS "sub-millisecond latency", with "microsecond-level message delivery" [S2]. That is the sharper exhibit, and not because the numbers are small. A ranking that names a percentile for one system and not for the other two cannot be read at all in this regime: sub-millisecond *somewhere* in the distribution is consistent with a median on the timestamp resolution and with

a median well above it, and those two have retained fractions that differ by orders of magnitude. The comparison reports no retained fraction, no rejection count, no timestamp resolution and no send rate, so nothing in it distinguishes the two cases. It is the clearest current instance of a published ranking produced inside the regime this paper is about.

A second 2026 comparison closes that gap from the other side. It ranks Kafka, Redis Streams and NATS on end-to-end p99 and gives Redis Streams 0.8 ms — a percentile, named, and below the tick — against 3.2 and 12.5 ms [S3]. The measurement is across a three-node cluster, so two clocks, and the method is one line about the node shape: no tool, no timestamp source, no synchronization statement, no resolution, no discard count. Unlike the ranking above, this one *can* be placed against the timestamp resolution, and it lands under it. We draw no conclusion about the number, which may be perfectly good; the point is that nothing published beside it would let a reader tell, and on a millisecond-resolution cross-node timestamp most of that distribution would compute to zero or to a negative an admission condition would drop.

Against that, the rules of Section VIII-B are being adopted independently, and by a benchmark rather than by a metrics agent. A 2026 voice-agent latency methodology records both endpoints *on one clock* — a dual-channel recording, chosen because "a platform measures from where it stands, and the caller is not standing there" — publishes its discard counts per platform and per reason so that "the denominator is auditable", and declines to claim differences below its detector's resolution [S93]. That is the one-clock rule, the retention rule and the resolution rule, arrived at in another domain by people who had not read this paper. We cite it as corroboration that the rules cost little, not as evidence for any claim of ours.

The load-generator half has an older precedent, and a closer one. Section IV-A says the achieved rate is recorded per run because the self-validation SPEC and TPC ask for applies to the measurement harness as well as to the system under test. Klenik and Kocsis did exactly that in 2021, porting TPC-C to Hyperledger Fabric [S94]: they instrument their own workers for what they call the scheduling precision reserve — the gap between when a transaction is picked and when it must be submitted — publish it against terminal count, and report that it "can drop below zero in the case of hundreds of terminals". What they then do with that negative is the part worth copying. They do not clamp it and they do not drop it; they weigh it against the timescale of the thing being measured, observing that the violation "never exceeds half a second" where TPC-C's constraints "are in the order of magnitude of seconds", and conclude it is tolerable *here*. That is this paper's own question — is the timestamp resolution smaller than the delivery? — asked about a load generator, answered in the open, and four years earlier. They publish their invalidated fraction as well.

The sharpest exhibit for the gap is a tutorial on exactly this subject: If the omission were confined to benchmarks under deadline it would be a smaller claim. It is not. A 2026 tutorial on statistical evaluation for computer-systems papers devotes a section to *latency-specific pitfalls*, opens it with "latency data is the most common metric in systems papers and the most consistently mis-reported", and names three traps: the mean as a summary, coordinated omission, and warmup effects [S95]. The harness's own clock is not among them. Over 31 pages the words *timestamp*, *resolution*, *quantization* and *retention* do not occur; *clock* occurs once, about non-determinism under seeding. Its pre-submission checklist asks for "median + tail percentiles + CDF; warmup discarded" and for no denominator, no resolution and no sign check.

The industrial half of that exhibit is a vendor's own fairness guide. A May 2026 Kafka benchmarking methodology, written expressly to say how to run a comparison that is fair, covers workload profile, hardware normalization, the durability contract and failure scenarios, and says nothing about discarded samples, negative values, timestamp resolution or the send rate's relation to the clock tick [S96]. Its entire treatment of the subject is the phrase "with outlier handling described" — the unfalsifiable gesture the rules of Section VIII-B exist to replace with a number. A tutorial and a vendor guide, four months apart, omitting the same thing is a stronger statement about the field than either alone.

A third document closes the set, and it is the one that should have caught this. In July 2026 a broker-monitoring vendor published a methodology guide whose central section is *The Seven Performance Benchmarking Errors* [S97] — not advice on running a benchmark, but an enumeration of the ways one goes wrong. The seven are co-locating the load generator with the broker, skipping JVM warmup, measuring only throughput, not running long enough, unrepresentative message sizes, triggering producer flow control, and not matching production configuration. Every one is about experimental design; none reaches the measurement tool. The words *clock*, *timestamp*, *resolution* and *negative* do not occur on the page. We note without irony that its first error is co-locating the load generator, while Section VIII-A finds that a co-located testbed *hides* the condition deciding the outcome: the two are different concerns, resource contention against exposure, and the guide is right about its own.

We record all three without complaint about their authors, who are doing something else and doing it well. The point is narrower and harder to argue with than any count of benchmarks: a document written in 2026 to enumerate the ways latency measurement goes wrong does not contain the ways this paper reports, which is what it means for a gap to be a gap.

S33.5. Where scheduling delay comes from, and what the queueing account gets right

A thread that is runnable does not always run. Work-conservation violations — a runnable thread left waiting while a core sits idle — are documented for CFS-era schedulers [S53]; ours is an EEVDF-era kernel, and we make no claim that those particular defects are present.

Queueing gives the leading-order account of waiting, and it is right about the thing that matters here: utilization alone does not fix a waiting time, because pooling and geometry matter at fixed ρ. This is why the geometry contrast of Section V refutes

a single-parameter law rather than queueing itself — a distinction we drew only after a referee pressed on it, since multi-server queueing is geometry-dependent at fixed ρ whether or not the scheduler conserves work.

Chandrasekar and Kramberger [S98] apply exactly such a single-parameter M/G/1 framing to benchmark bias. We adopt it as a leading-order account and refute it on the load axis. On a second axis they reach half of our mechanism concurrently and independently: their client's own interpreter lock "artificially inflates" the latencies it reports, charging the harness's own scheduling to the server under test. That is the inflation half. What a positive number cannot show — the negative span, and the sign channel that makes the displacement legible — is what remains ours.

Tail behavior under load [S51] is the phenomenon our stall distribution samples. That work also shows real-time priority collapsing a tail that renicing leaves alone, applied to the system under test; we apply the same lever to the harness, which is the manipulation of Section V.

REFERENCES

[S1] F. N. Bostanci, H. Luo, A. Olgun, M. Makeenkova, G. F. de Oliveira, A. G. Yaglikci, and O. Mutlu, "Cleaning up the mess: Re-evaluating the real-system modeling accuracy of Ramulator 2.0," arXiv preprint arXiv:2510.15744v4, May 2026, shows a MICRO best-paper runner-up with all three artifact badges, including Reproducible, reported incorrect simulator results through configuration errors and could not rebuild its own key results; argues for mechanisms to correct results and artifacts after publication. Read 2026-09-03.

[S2] Index.dev, "NATS vs Redis vs Kafka: message broker comparison," Practitioner comparison, 2026, https://www.index.dev/skill-vs-skill/nats-vs-redis-vs-kafka; no retained fraction, rejection count, timestamp resolution or send rate. Which broker gets the percentile, and what it is, is in Section III-A. S1 records what we first read here and why it was wrong. Read 2026-09-03.

[S3] Y. Gao, "Real-time event streaming: Kafka vs Redis Streams vs NATS in 2026," Practitioner comparison, 2026, https://dev.to/young_gao/real-time-event-streaming-kafka-vs-redis-streams-vs-nats-in-2026-34o1; ranks the three on "End-to-end latency (p99)" and gives Redis Streams 0.8 ms, NATS JetStream 3.2 ms and Kafka 12.5 ms, with a summary table reading "<1ms / 1-5ms / 5-20ms". Method given in full as "Tested on 3-node cluster, 8 vCPU, 32GB RAM each" and "benchmarks based on production deployments"; no tool, timestamp source, synchronization statement, resolution or discard count. Read 2026-09-04, and the source of the figures withdrawn from `indexdev2026brokers` above.

[S4] Apache Kafka, "KIP-526: Reduce producer metadata lookups for large number of topics," Apache Software Foundation, Kafka Improvement Proposal, 2020, documents the metadata fetch a producer performs before its first send to an uncached topic.

[S5] Joint Committee for Guides in Metrology, "Evaluation of measurement data — guide to the expression of uncertainty in measurement (JCGM 100:2008)," https://www.bipm.org/documents/20126/2071204/JCGM_100_2008_E.pdf, 2008, clause F.2.2.1, "The resolution of a digital indication". Read 2026-08-20.

[S6] J. W. Pratt, "Remarks on zeros and ties in the Wilcoxon signed rank procedures," *Journal of the American Statistical Association*, vol. 54, no. 287, pp. 655–667, 1959.

[S7] IEEE, "IEEE standard for terminology and test methods for analog-to-digital converters," IEEE Std 1241-2010, 2011, §6.4.1, Eq. (28): the histogram test's relatively-prime record condition; §5.4.1, Eq. (18): $f_i = (J/M)f_s$, named coherent sampling.

[S8] OpenMessaging Project, Linux Foundation, "The OpenMessaging benchmark framework," 2018, available at openmessaging.cloud/docs/benchmarks/; accessed 19 August 2026.

[S9] Linux Kernel Documentation, "KVM-specific MSRs: `MSR_KVM_SYSTEM_TIME_NEW`," https://docs.kernel.org/virt/kvm/x86/msr.html, 2026, flag bit 0, CPUID `0x40000001` bit 24. Read 2026-08-21.

[S10] Z. Zhou, "sched: Reduce the default slice to avoid tasks getting an extra tick," Linux kernel mailing list, commit 2ae891b82695, 2025, reviewed-by V. Guittot; reduces the constant to `0.70` in v6.15.

[S11] Hewlett-Packard, "Time interval averaging," Hewlett-Packard Company, Application Note 162-1, 1970, part no. 02-5952-0766; undated; the year is its *HP Journal* citation.

[S12] J. Rapp, R. M. A. Dawson, and V. K. Goyal, "Estimation from quantized Gaussian measurements: When and how to use dither," *IEEE Transactions on Signal Processing*, vol. 67, no. 13, pp. 3424–3438, 2019.

[S13] V. Raisanen, G. Grotefeld, and A. Morton, "Network performance measurement with periodic streams," Internet Engineering Task Force, RFC 3432, Nov. 2002.

[S14] A. Sharma, D. Shah, D. Lariviere, and H. ElBakoury, "Time, causality, and observability failures in distributed AI inference systems," *arXiv preprint arXiv:2604.21361*, Apr. 2026, open Compute Project, Unified Intelligent Infrastructure workstream.

[S15] StatsBomb, "Statsbomb open data," *GitHub Repository*, 2023. [Online]. Available: https://github.com/statsbomb/open-data

[S16] H. Liu, W. Hopkins, A. M. Gomez, and J. S. Molinuevo, "Inter-operator reliability of live football match statistics from opta sportsdata," *International Journal of Performance Analysis in Sport*, vol. 13, no. 3, pp. 803–821, 2013.

[S17] L. Pappalardo, P. Cintia, A. Rossi, E. Massucco, P. Ferragina, D. Pedreschi, and F. Giannotti, "A public data set of spatio-temporal match events in soccer competitions," *Scientific Data*, vol. 6, no. 1, p. 236, 2019.

[S18] M. Mohammad, "Analysis of design patterns and benchmark practices in Apache Kafka event-streaming systems," *arXiv preprint arXiv:2512.16146*, 2025.

[S19] T. Downs, "Grow buffer if histogram can't fit (pull request #398)," OpenMessaging Benchmark, commit 4eb2327c, Dec. 2023, fixes issue #369: truncated histogram serialisation silently discarded most latency samples. https://github.com/openmessaging/benchmark/pull/398; accessed 19 August 2026.

[S20] J. Vanlightly, "Kafka versus Redpanda performance — do the claims add up?" https://jack-vanlightly.com/blog/2023/5/15/kafka-vs-redpanda-performance-do-the-claims-add-up, May 2023, a second, independent silent-loss defect in the same harness.

[S21] S. Guo, "Avoid adding negative metric values (pull request #56)," OpenMessaging Benchmark, commit ebb5d6b8, Mar. 2018, https://github.com/openmessaging/benchmark/pull/56; accessed 18 August 2026.

[S22] HdrHistogram contributors, "Recorder — HdrHistogram API documentation," Project documentation, 2026, `recordValue` documents its exception as thrown "if value is exceeds highestTrackableValue" and does not mention negative values; the README describes the configurable range and the significant digits and is likewise silent on them. https://hdrhistogram.github.io/HdrHistogram/JavaDoc/org/HdrHistogram/Recorder.html. Read 2026-09-10.

[S23] franz1981, "The load generation need to fix coordinated omission (issue #247)," OpenMessaging Benchmark issue tracker, 2022, opened 2022-03-01, closed with pull request #248. Argues that a load generator failing to keep up "will pass unnoticed in the percentiles ocean" unless the publish delay is recorded against the scheduled send instant. Accepted; the admission condition was untouched by the repair. https://github.com/openmessaging/benchmark/issues/247. Read 2026-09-10.

[S24] SamBarker, "Export serialized HDR histogram to enable statistical significance testing of results (issue #452)," OpenMessaging Benchmark issue tracker, 2026, opened 2026-03-26. Observes that the harness "already maintains HDR Histograms internally throughout each run and discards them after extracting the summary scalars", and that summary scalars cannot say whether a difference is significant "or within the noise floor of the measurement". https://github.com/openmessaging/benchmark/issues/452. Read 2026-09-10.

[S25] Y. Virkar and A. Clauset, "Power-law distributions in binned empirical data," *The Annals of Applied Statistics*, vol. 8, no. 1, pp. 89–119, 2014.

[S26] Apache Software Foundation, "Apache Kafka tools — `EndToEndLatency.java` and `ProducerPerformance.java`," Source repository, 2026, https://github.com/apache/kafka; percentiles are computed from integer-divided milliseconds. Read 2026-08-31.

[S27] Apache Pulsar contributors, "`PerformanceConsumer.java`," Source repository, 2026, https://github.com/apache/pulsar. Read 2026-08-21.

[S28] Synadia, "NATS cli — `cli/latency_command.go`," Source repository, 2026, https://github.com/nats-io/natscli; appends the difference unconditionally. Read 2026-08-31.

[S29] Broadcom, "RabbitMQ PerfTest — `Consumer.java`," Source repository, 2026, https://github.com/rabbitmq/rabbitmq-perf-test; passes the difference unfiltered. Read 2026-08-31.

[S30] IOP Systems, "Rezolus: systems performance telemetry — `scheduler_discarded_samples`," Source repository, 2026, https://github.com/iopsystems/rezolus; counts samples discarded for out-of-order timestamps. Read 2026-08-20.

[S31] A. D. Brunelle, "`blktrace`/`btt` — `btt/inlines.h`," Source repository, 2026, `tdelta()` returns one nanosecond when the stage timestamps are out of order: `return (from < to) ? (to - from) : 1;`. Read 2026-08-20.

[S32] EMQ Technologies, "`emqtt-bench`: MQTT benchmark tool — `src/emqtt_bench.erl`," Source repository, 2026, https://github.com/emqx/emqtt-bench; the guard is on the counter, not the sample. Read 2026-08-21.

[S33] J. Axboe, "fio: Flexible I/O tester — `gettime.c`," Source repository, 2026, https://github.com/axboe/fio; the guard is in three timing functions. Read 2026-08-20.

[S34] G. Tene, "`wrk2` — `src/wrk.c`," Source repository, 2026, https://github.com/giltene/wrk2; the negative is detected, printed, and recorded unguarded; the vendored `hdr_histogram.c` bounds-checks the derived index behind live assertions. Read 2026-08-21.

[S35] HdrHistogram contributors, "`HdrHistogram_c` — `src/hdr_histogram.c`," Source repository, 2026, https://github.com/HdrHistogram/HdrHistogram_c; upstream's `hdr_record_values` refuses a negative value outright — the guard wrk2's vendored copy lacks. Read 2026-08-21.

[S36] Y. Mittal, "An empirical study of open-loop load generators for microservices and web workloads," Master's thesis, University of Illinois Urbana-Champaign, 2025, https://hdl.handle.net/2142/129222; behavioural evaluation of six open-loop generators. Read 2026-08-21.

[S37] Apache Kafka contributors, "KAFKA-19888: Clamp negative values in coordinator histograms," Issue tracker and pull request #20986, 2025, https://issues.apache.org/jira/browse/KAFKA-19888; merged 2025-11-26, shipped in 4.2.0, backported to 4.0.2/4.1.2. Read 2026-08-21.

[S38] L. Kleinrock, *Queueing Systems: Volume I - Theory*. Wiley-Interscience, 1975.

[S39] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. 14th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2009, pp. 265–276.

[S40] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*. ACM, 2007, pp. 57–76.

[S41] E. Barrett, C. F. Bolz-Tereick, R. Killick, S. Mount, and L. Tratt, "Virtual machine warmup blows hot and cold," *Proceedings of the ACM on Programming Languages*, vol. 1, no. OOPSLA, pp. 52:1–52:27, 2017.

[S42] A. Maricq, D. Duplyakin, I. Jimenez, C. Maltzahn, R. Stutsman, and R. Ricci, "Taming performance variability," in *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2018, pp. 409–425.

[S43] T. Kalibera and R. Jones, "Rigorous benchmarking in reasonable time," in *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*. ACM, 2013, pp. 63–74.

[S44] E. van der Kouwe, G. Heiser, D. Andriesse, H. Bos, and C. Giuffrida, "SoK: Benchmarking flaws in systems security," in *Proc. 4th IEEE EuroS&P*. IEEE, 2019, pp. 310–325.

[S45] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC)*. ACM, 2015, pp. 73:1–73:12.

[S46] H. Weyl, "Über die gleichverteilung von zahlen mod. eins," *Mathematische Annalen*, vol. 77, pp. 313–352, 1916.

[S47] V. T. Sós, "On the distribution mod 1 of the sequence $n\alpha$," *Annales Universitatis Scientiarum Budapestinensis, Sectio Mathematica*, vol. 1, pp. 127–134, 1958.

[S48] L. Schuchman, "Dither signals and their effect on quantization noise," *IEEE Transactions on Communication Technology*, vol. 12, no. 4, pp. 162–165, 1964.

[S49] S. McCanne and C. Torek, "A randomized sampling clock for CPU utilization estimation and code profiling," in *Proc. USENIX Winter Technical Conf.*, 1993, pp. 387–394.

[S50] J. Ousterhout, "Always measure one level deeper," *Communications of the ACM*, vol. 61, no. 7, pp. 74–83, 2018.

[S51] J. Li, N. K. Sharma, D. R. K. Ports, and S. D. Gribble, "Tales of the tail: Hardware, OS, and application-level sources of tail latency," in *Proc. ACM SoCC*. ACM, 2014.

[S52] J. Dean and L. A. Barroso, "The tail at scale," *Communications of the ACM*, vol. 56, no. 2, pp. 74–80, 2013.

[S53] J.-P. Lozi, B. Lepers, J. Funston, F. Gaud, V. Quéma, and A. Fedorova, "The Linux scheduler: a decade of wasted cores," in *Proc. 11th European Conf. Computer Systems (EuroSys)*. ACM, 2016, pp. 1–16.

[S54] P. B. Danzig and S. Melvin, "High resolution timing with low resolution clocks and a microsecond resolution timer for sun workstations," *ACM SIGOPS Operating Systems Review*, vol. 24, no. 1, pp. 23–26, Jan. 1990.

[S55] S. Herrnleben, M. Leidinger, V. Lesch, T. Prantl, J. Grohmann, C. Krupitzer, and S. Kounev, "ComBench: A benchmarking framework for publish/subscribe communication protocols under network limitations," in *Performance Evaluation Methodologies and Tools (VALUETOOLS 2021)*. Springer, 2021.

[S56] G. Tene, "How NOT to measure latency," 2015, talk, Strange Loop / QCon; coined "coordinated omission".

[S57] ——, "HdrHistogram: A high dynamic range histogram," 2015, used by the OpenMessaging Benchmark; available at hdrhistogram.org.

[S58] R. A. Wannamaker, S. P. Lipshitz, J. Vanderkooy, and J. N. Wright, "A theory of nonsubtractive dither," *IEEE Transactions on Signal Processing*, vol. 48, no. 2, pp. 499–516, 2000.

[S59] S. Yang, J. Jeong, B. Scholz, and B. Burgstaller, "Cloudprofiler: TSC-based inter-node profiling and high-throughput data ingestion for cloud streaming workloads," *arXiv preprint arXiv:2205.09325*, 2022.

[S60] Network-Based Computing Laboratory, Ohio State University, "OSU micro-benchmarks 7.5 — `osu_latency`," https://mvapich.cse.ohio-state.edu/benchmarks/, 2025, `c/mpi/pt2pt/standard/osu_latency.c` divides the round trip by two; the documentation reports "average one-way latency". Read 2026-08-20.

[S61] D. Turner and X. Chen, "NetPIPE: A network protocol independent performance evaluator," https://netpipe.cs.ksu.edu/, 2025, "Latencies are calculated by dividing the round trip time in half for small messages". Read 2026-08-20.

[S62] Intel Corporation, "Intel MPI benchmarks — `PingPong`," Intel MPI Library User Guide, 2021, "Reported timings — time $= \Delta t/2$"; the companion `PingPing` reports the unhalved time. Read 2026-08-20.

[S63] S. Hunold and A. Carpen-Amarie, "Reproducible MPI benchmarking is still not as easy as you think," *IEEE Transactions on Parallel and Distributed Systems*, vol. 27, no. 12, pp. 3617–3630, 2016.

[S64] S. Sundberg, A. Brunstrom, S. Ferlin-Reiter, J. D. Brouer, and T. Høiland-Jørgensen, "Waiting at the front door: Continuous monitoring of latency in the host network stack," in *Proc. ACM Internet Measurement Conference (IMC)*, 2026, an eBPF monitor timestamping at kernel hook points, deployed in a CDN. Its abstract names the sub-millisecond domain and application scheduling as host latency sources. It states its histogram resolution error —

$\pm 33.3\%$ from the bin midpoint on base-2 buckets, with $2^{1/8}$ costed at $\pm 4.33\%$ — and bounds its own observer effect below 6% tail inflation. Read 2026-09-10.

[S65] G. V. Rodríguez, A. Castañeda, and M. Piña, "On the limits of causal observation in shared-memory systems," arXiv preprint arXiv:2606.14093, 2026, formalizes the Causal Observability Problem and proves no solution at the observable boundary is both complete and sound; the placement of instrumentation relative to operation boundaries decides which of the two a monitor gets, "independently of the underlying timestamp mechanism". Shared-memory concurrent objects, not measurement: it reports neither failure mode studied here. Read 2026-09-10.

[S66] Confluent, "`librdkafka` introduction — delivery reports," https://github.com/confluentinc/librdkafka, 2026, the delivery-report callback `dr_msg_cb` is served by `rd_kafka_poll()` and other queue-serving calls, so it runs on the application's thread rather than asynchronously. Read 2026-08-20.

[S67] J. Axboe, "`fio` HOWTO — interpreting the output," https://fio.readthedocs.io/, 2026, for asynchronous ioengines completion latency runs "to when the I/O's completion was reaped by fio". Read 2026-08-20.

[S68] Advanced Micro Devices, "HIP runtime API: Event management," https://rocm.docs.amd.com/projects/HIP/en/latest/, 2026, under `hipEventElapsedTime()`: "the time that the event recorded may be significantly after the host calls hipEventRecord()". Read 2026-08-20.

[S69] NVIDIA Corporation, "CUDA C++ best practices guide, using CUDA GPU timers," https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/, 2026, section 9.1.2: "The device will record a timestamp for the event when it reaches that event in the stream." Read 2026-08-20.

[S70] Mellanox Technologies, "`sockperf`: network benchmarking utility," https://github.com/Mellanox/sockperf, 2026, the run summary prints `SentMessages`, `ReceivedMessages` and `SkippedMessages` beside the latency percentiles, and a dropped/duplicated/out-of-order line derived from sequence numbers. Read 2026-08-20.

[S71] OpenTelemetry Authors, "OpenTelemetry metrics data model — exponential histogram: Zero count and zero threshold," https://opentelemetry.io/docs/specs/otel/metrics/data-model/, 2026, `zero_count` holds values at or below a zero threshold, including "values that have been rounded to zero"; `count` is defined to include it. Read 2026-08-20.

[S72] A. Klenik and A. Pataricza, "Adding semantics to measurements: Ontology-guided, systematic performance analysis," *Acta Cybernetica*, 2022, arXiv:2112.11270; negative derived durations in a Hyperledger Fabric benchmark, traced to peer clock drift.

[S73] R. Durairajan, S. K. Mani, P. Barford, R. Nowak, and J. Sommers, "TimeWeaver: Opportunistic one way delay measurement via NTP," in *Proc. 30th International Teletraffic Congress (ITC 30)*, 2018, pp. 185–193, retains samples that earlier methods discarded, including negative latencies, by assigning them to lower accuracy tiers.

[S74] EEMBC, "CoreMark-PRO run rules," https://github.com/eembc/coremark-pro, 2026, "Each workload must run for at least 1000 times the minimum timer resolution." Plain CoreMark instead sets an absolute ten-second floor. Read 2026-08-20.

[S75] Standard Performance Evaluation Corporation, "SPEC power and performance benchmark methodology, version 2.3," https://www.spec.org/power/docs/SPEC-Power_and_Performance_Methodology.pdf, 2025, clause 8.2.2, Power Reporting: the benchmark "should report the number of samples that may have a return value 'Unknown"' and those exceeding the maximum allowed uncertainty. Read 2026-08-20.

[S76] S. Shalunov, B. Teitelbaum, A. Karp, J. W. Boote, and M. J. Zekauskas, "A one-way active measurement protocol (OWAMP)," Internet Engineering Task Force, RFC 4656, Sep. 2006.

[S77] ETSI, "Speech and multimedia transmission quality (STQ); QoS aspects for popular services in mobile networks; part 4: Requirements for quality of service measurement equipment," European Telecommunications Standards Institute, Technical Specification TS 102 250-4 V2.3.1, Nov. 2019, clause 5.1, General requirement for data logging.

[S78] G. Almes, S. Kalidindi, M. Zekauskas, and A. Morton, "A one-way delay metric for IPPM," Internet Engineering Task Force, RFC 7679, Jan. 2016, obsoletes RFC 2679. Section 3.5 keeps negative values as distribution data rather than discarding them.

[S79] Standard Performance Evaluation Corporation, "SPECjms2007 run and reporting rules, version 1.0," https://www.spec.org/jms2007/docs/RunRules.html, 2014, clause 3.6.2 records delivery time as a histogram of default granularity 100 ms; the only timing-quality requirement is clause 3.2's 100 ms bound on inter-node clock agreement. Read 2026-08-20.

[S80] K. Sachs, S. Kounev, J. Bacon, and A. P. Buchmann, "Performance evaluation of message-oriented middleware using the SPECjms2007 benchmark," *Performance Evaluation*, vol. 66, no. 8, pp. 410–434, 2009.

[S81] zoxy-io, "`zrk`: a Zig rewrite of `wrk2`," https://github.com/zoxy-io/zrk, 2026, "At the sub-millisecond latencies a fast service actually produces, that's not a rounding error — it's frequently larger than the number being measured"; the overshoot is recorded rather than discarded. Read 2026-08-21.

[S82] G. Tene, "`wrk2`: a constant-throughput, correct-latency variant of `wrk`," https://github.com/giltene/wrk2, 2019, "measured latencies are only accurate to a +/− ~1 msec granularity, due to OS sleep time behavior" — disclosed, not discarded. Read 2026-08-21.

[S83] M. Kuperberg, M. Krogmann, and R. H. Reussner, "TimerMeter: Quantifying properties of software timers for system analysis," in *Proc. 6th Int. Conf. Quantitative Evaluation of Systems (QEST)*, 2009, pp. 85–94.

[S84] Jaeger contributors, "`model/adjuster/clockskew.go`," https://github.com/jaegertracing/jaeger, 2026, at v1.57.0; adjusters run in the query service, and the default has been off since v1.20.0. Read 2026-08-21.

[S85] ——, "Clock skew adjuster considered harmful (issue #1459)," Issue tracker, 2019, https://github.com/jaegertracing/jaeger/issues/1459; the adjustment has shipped disabled since v1.20.0 (2020), which set `-query.max-clock-skew-adjustment` to `0s`. Read 2026-08-21.

[S86] A. Swami and N. Chougule, "Architecture dependent temporal observability under deployment interference in edge inference systems," *arXiv preprint arXiv:2605.17701*, 2026.

[S87] T. C. Paul, P. Lertpongrujikorn, H. D. Nguyen, and M. Amini Salehi, "Benchmarking message brokers for IoT edge computing: A comprehensive performance study," arXiv preprint arXiv:2603.21600, 2026, introduces mq-bench: nanosecond send timestamps, one host, no positivity filter. Read 2026-08-25.

[S88] W. Yue, M. Boissier, and T. Rabl, "A survey of stream processing system benchmarks," in *Performance Evaluation and Benchmarking (TPCTC)*. Hasso Plattner Institut, 2024, reviews 27 benchmark efforts across five dimensions, one of them "tracked metrics", and closes on the aspects that have been overlooked. Over 21 pages it says benchmark 169 times and latency 33; timestamp, clock, resolution and "measurement error" do not occur. Counts recomputed by scripts/literature_census.py. Read 2026-09-09.

[S89] M. Fruth, S. Scherzinger, W. Mauerer, and R. Ramsauer, "Tell-tale tail latencies: Pitfalls and perils in database benchmarking," in *Performance Evaluation and Benchmarking (TPCTC)*. Springer, 2021, pp. 119–134.

[S90] Y. Zhang, D. Meisner, J. Mars, and L. Tang, "Treadmill: Attributing the source of tail latency through precise load testing and statistical inference," in *Proc. 43rd ACM/IEEE Int. Symp. Computer Architecture (ISCA)*, 2016.

[S91] F. Cerqueira and B. B. Brandenburg, "A comparison of scheduling latency in Linux, PREEMPT_RT, and LITMUS^RT," in *Proc. 9th Annual Workshop on Operating Systems Platforms for Embedded Real-Time Applications (OSPERT)*, 2013, pp. 19–29, footnote 7 bounds worst-case scheduling latency below by the quantum length under a quantum-driven scheduler.

[S92] M. R. C. Mukkolakkal, "1.5 million messages per second on 3 machines: Benchmarking and latency optimization of Apache Pulsar at enterprise scale," arXiv preprint arXiv:2603.29113, 2026, reports 1,499,947 msg/s at a 3.88 ms median publish latency on three bare-metal nodes. The median sits above the quantum, so the deletion arithmetic does not bind there and this is a comparison FOR the main text's concession rather than an exception to it. Over four pages it names that median seven times; the words timestamp, clock, resolution, discard, negative and synchronization do not occur. Read 2026-09-10.

[S93] Openbenchmarks, "How voice agent latency is measured," Methodology page, 2026, https://openbenchmarks.com/voice-agent-latency/how-voice-agent-latency-is-measured; both endpoints on one clock, discard counts published per platform and per reason, and differences below the detector's resolution not claimed. Read 2026-08-31.

[S94] A. Klenik and I. Kocsis, “Porting a benchmark with a classic workload to blockchain: TPC-C on Hyperledger Fabric,” in *arXiv preprint arXiv:2112.11277*, 2021, instruments the load generator itself: the “scheduling precision reserve” is the gap between when a transaction is picked and when it must be submitted, published as boxplots against terminal count. It “can drop below zero in the case of hundreds of terminals”, and the authors weigh that against the workload’s own timescale — the violation “never exceeds half a second” where TPC-C’s constraints “are in the order of magnitude of seconds”. The invalidated fraction is published too. Read 2026-09-04.

[S95] B. Krishnamachari, “How to do statistical evaluations in ECE/CS papers: A practical playbook for defensible results,” arXiv preprint arXiv:2605.00428, 2026, section 14, “Latency-Specific Pitfalls”, names three: the mean as a summary, coordinated omission, and warmup effects; section 18, “Outliers, failed runs, and exclusion rules”, is the nearest it comes to a discard rule. Over 31 pages the words timestamp, resolution, quantization and retention do not occur. Read 2026-09-03.

[S96] AutoMQ, “Kafka benchmark methodology: run fair comparisons,” Vendor methodology guide, May 2026, https://www.automq.com/blog/kafka-benchmark-methodology-run-fair-comparisons; covers workload profile, hardware normalization, durability contract and failure scenarios. Its whole treatment of what the instrument discards is the phrase “with outlier handling described”. Read 2026-09-03.

[S97] meshIQ, “ActiveMQ performance benchmarking: the complete methodology guide,” Practitioner guide, 2026, https://www.meshiq.com/blog/activemq-performance-benchmarking-methodology/, 21 July 2026. “The Seven Performance Benchmarking Errors” are co-locating the load generator with the broker, skipping JVM warmup, measuring only throughput, not running long enough, non-representative message sizes, triggering producer flow control, and not matching production configuration. Verified on two independent reads that *clock*, *timestamp*, *resolution* and *negative* do not occur on the page. Read 2026-09-04.

[S98] A. Chandrasekar and J. Kramberger, “Identifying and mitigating systemic measurement bias in production LLM inference benchmarks,” *arXiv preprint arXiv:2605.24217v2*, 2026.